

プログラミング演習 (学際科学科)
広域システム特論 III (3) (学際科学科)
総合分析情報学基礎 IX (情報学環 学際情報学府)

金子 知適 (kaneko@acm.org)

2026 年度 夏学期

Time-stamp: <2026-06-24 16:31:13 tk>

目次

第 1 章	ガイダンス・文の実行と制御構造	7
1.1	ガイダンス	7
1.2	環境整備	7
1.2.1	UTOL への登録	7
1.2.2	jupyter-notebook の起動と操作	7
	UV	8
	Anaconda	9
1.2.3	Google Colaboratory	10
1.3	プログラムの作成と実行	10
1.4	順次の実行	11
1.5	整数と繰り返し	12
1.6	条件分岐	16
1.7	総合問題	17
1.8	この章の課題	20
第 2 章	式と評価, 関数	21
2.1	式の評価	21
2.2	関数的な計算: 式と評価	22
2.2.1	原始的な式	22
2.2.2	式の組み合わせ	22
2.2.3	変数: 値に名前をつける	23
2.2.4	関数: 計算に名前をつける	24
2.2.5	関数内の変数	26
2.2.6	様々な型と演算	27
2.2.7	条件判断	30
2.2.8	計算の順序	32
2.3	参考: 命令的な計算	33
2.4	発展: 関数を返す関数	34
2.5	この章の提出課題	39
2.A	毎週の課題の提出方式について	40
2.A.1	授業開始時: テンプレートの入手	40
2.B	ターミナルでの Python3 の利用	41
2.B.1	整数除算	41
2.C	参考資料	42

第 3 章	ユニットテスト, 配列, ループ不変条件	43
3.1	テストの自動化: ユニットテスト	43
3.1.1	テストケースの作成	46
3.2	配列と関数	47
3.2.1	関数内の変数 (前半は再掲)	47
3.2.2	配列	48
3.3	変数の値の書き換えとループ不変条件	50
3.4	配列とループ	52
3.5	配列や文字列の変更	53
3.6	発展: 関数のパラメータ化	57
3.7	この章の提出課題	59
3.A	テスト駆動開発★	60
3.A.1	閏年の判定	60
	失敗するテストの作成	60
	実装とテストの拡充	61
	実装の変更	61
	仕様の変更	61
3.B	C++でのテスト	62
第 4 章	参照と二次元配列	64
4.1	配列の要素の書き換え	64
4.2	二次元配列	66
4.3	成長スライドパズル	69
4.3.1	1 行の移動	70
4.3.2	正方形の盤面	71
4.4	有限ライフゲーム	73
4.5	この章の提出課題	79
4.A	実体と参照★	80
4.B	先週の補足	85
第 5 章	再帰	86
5.1	帰納的定義と線形再帰	86
5.2	範囲を絞る: 二分法・二分探索	87
5.3	枝分かれを伴う再帰	91
5.4	併合整列 (Merge Sort)	95
5.5	再帰と反復の応用	97
5.6	この章の提出課題	99
5.A	関数と実行状態の管理	100
5.B	ユークリッドの互除法と応用	102
第 6 章	木構造, 複合データ (クラス)	103

6.1	階層的構造	103
6.1.1	数値の二分木	103
6.1.2	木の頂点の訪問	105
6.1.3	規則を持つ木: 二分探索木	107
6.1.4	決定木	108
6.1.5	四則演算の構文木	110
6.1.6	内部表現の世界と使用する世界の分離	111
6.2	クラスの利用の初歩	111
6.3	発展問題	113
6.4	この章の提出課題	114
第7章	状態管理, 抽象インターフェースと継承	115
7.1	内部状態を持つオブジェクト	115
7.1.1	内部状態	115
7.1.2	カプセル化	116
7.1.3	メソッドの再帰呼出し	120
7.1.4	状態と複製	122
7.1.5	クラスの図表現	124
7.2	インターフェース	125
7.2.1	囚人のジレンマゲーム	125
7.2.2	ゲームとプレイヤーのモデル化	125
7.2.3	繰り返し囚人のジレンマゲーム	128
7.2.4	インターフェースの拡張と継承	128
7.3	この章の提出課題	131
第8章	クラスライブラリ・デザインパターン	133
8.1	クラスとオブジェクト	133
8.1.1	C++のクラス	134
8.1.2	例外処理	135
8.1.3	複数の方を持ちうる型	137
8.2	Python の help と docstring	137
8.3	クラスとインターフェースの継承	139
8.3.1	C++とインターフェースの継承	141
8.4	デザイン・パターン	144
8.4.1	パターンの例: Observer パターンと時計プログラム	144
8.4.2	Subject と Observer	145
8.4.3	時を刻む部分の変更	146
8.4.4	時刻を表示する部分	146
8.5	C++で class を扱う注意点	148
8.5.1	複製と参照, スマートポインタ	148

8.6	この章の提出課題	154
第 9 章	状態の保存と通信	155
9.1	モジュールとパッケージ	155
9.2	データの扱い	156
9.2.1	文字列とバイト列	156
9.2.2	テキストファイルの読み書き	157
9.2.3	path	159
9.2.4	CSV	159
9.2.5	階層データと dict	160
	配列や参照を含む dict	160
9.2.6	JSON: 階層データの文字列表現	162
9.3	HTTP	163
9.3.1	HTTP での読み込み	163
9.3.2	データの配布	164
9.3.3	geojson	164
9.4	numpy	164
9.4.1	線形代数	165
9.4.2	numpy データの保存	166
9.5	乱数の管理	166
9.5.1	seed	167
9.5.2	状態管理	167
9.6	ファイルからの実行	168
9.6.1	ソースファイルの作成と実行	168
9.6.2	引数の指定	169
9.6.3	テストの実行	170
9.6.4	総合問題	170
9.7	この章の提出課題	171
9.A	Microsoft Windows での環境設定	171
9.A.1	Gitbash	171
9.A.2	WSL	172
9.A.3	MSYS2	172
9.B	Jupyter で書いたプログラムの実行	172
9.C	ファイルの分割	173
9.D	reload	174
第 10 章	最終課題	175
10.1	最終課題の選択肢	175
10.2	Schedule	175
10.2.1	提出期限	176

第 11 章	最終課題作成	177
第 12 章	最終課題発表	178
付録 A	開発ツール	179
A.1	バージョン管理システムと最小限の git	179
A.1.1	さいしょの目標	179
A.1.2	初期設定	180
ユーザ情報の設定		180
リポジトリの初期作成		180
既存のリポジトリを複製する場合		181
新規リポジトリを作成する場合 (参考)		181
A.1.3	ファイルの変更と登録	181
git add と commit の関係		182
A.1.4	履歴の活用	184
A.2	リモートコンピュータの利用 — SSH	184
A.2.1	公開鍵暗号システムを用いた認証	184
鍵ペアの作成		185
公開鍵の登録		185
ログイン		186
A.3	リモートリポジトリの利用と共同開発	187
A.3.1	上流リポジトリの作成	187
A.3.2	上流リポジトリの活用	188
A.3.3	複数人での開発	189
付録 B	最終課題: 潜水艦ゲーム	191
B.1	目標	191
B.2	準備	191
B.3	動作確認と対戦	192
B.4	独自の思考プレイヤー作り方	194
B.4.1	サンプルプログラムを読む	194
B.4.2	ヒント	194
B.5	Python パッケージの読み方	195
B.5.1	フォルダ構成	195
B.5.2	よく使うモジュール	195
pytest		196
argparse		196
logging		197
B.5.3	関数と引数	199
B.6	2025 年度の変更	200
付録 C	最終課題: 強化学習 Gymnasium	201

C.1	インストール	201
C.2	動作確認	201
C.3	ランダムプレイ	202
C.4	賢い行動をさせる	202
C.4.1	初めの一步 (案)	203
C.4.2	強化学習	203
C.4.3	環境の選択	203

第 1 章 ガイダンス・文の実行と制御構造

Introduction, Statement and Control structure
2026-04-08 Wed

事務連絡

ハイブリッド (オンライン Zoom 併用) で行うが、原則として学部生は対面、大学院生はオンライン参加を想定している。ソースコードの質問・コメントのために、Zoom で画面共有機能を用いる。OS の設定と再起動が必要な場合もあるので、事前に方法を確認しておくが良い。学際情報学府/他学部聴講の受講者への注意: この授業は教養学部的时间割で行う。



Notice: 環境に関する注意

各自の PC でも参加できるように、jupyter を各自でインストールすることを勧める。インストールに困難がある場合などは、当面 1.2.3 節に紹介する Google Colaboratory を利用して良い。その場合、提出方法の指示などの jupyter は、適宜 Google Colaboratory と読み替える。最終課題の標準タスクは、Colaboratory では動かないことに注意。

1.1 ガイダンス

- 演習の形式・評価: 毎週の提出 + 最終課題
- 使用言語 Python3 (C++は指定の一部課題のみ許可)
- 標準環境 jupyter-notebook

1.2 環境整備

1.2.1 UTOL への登録

水曜 5 限, プログラミング演習 (学際科学科, 広域システム特論 III(3) も可), 総合分析情報学基礎 IX (情報学環総合分析情報学コース) で登録する。「教材」の欄に本資料が PDF においてあるはずである。

1.2.2 jupyter-notebook の起動と操作

Python の管理システムはいろいろとあるが、授業担当者は uv をお勧めする。

UV

まず uv をインストールする。インストール方法は homebrew や winget, scoop などパッケージマネージャを使っていればそれ経由がお勧め。 <https://docs.astral.sh/uv/getting-started/installation/#homebrew>

```
 $ brew install uv 1
```

インストールに成功したら、‘uv self version’ でバージョンを確認する。0.11.3 が 2026 年 4 月時点の最新の模様。

```
 $ uv self version 1
uv 0.11.3 (Homebrew 2026-04-01 aarch64-apple-darwin) 2
```

授業のディレクトリを作成する。たとえば、水曜五限なので W5 とするなら

```
 $ mkdir W5 1
$ cd W5 2
```

Virtualenv を設定する。‘uv venv’ だけでそれ以上の引数を与えなければ、フォルダは .venv になる。

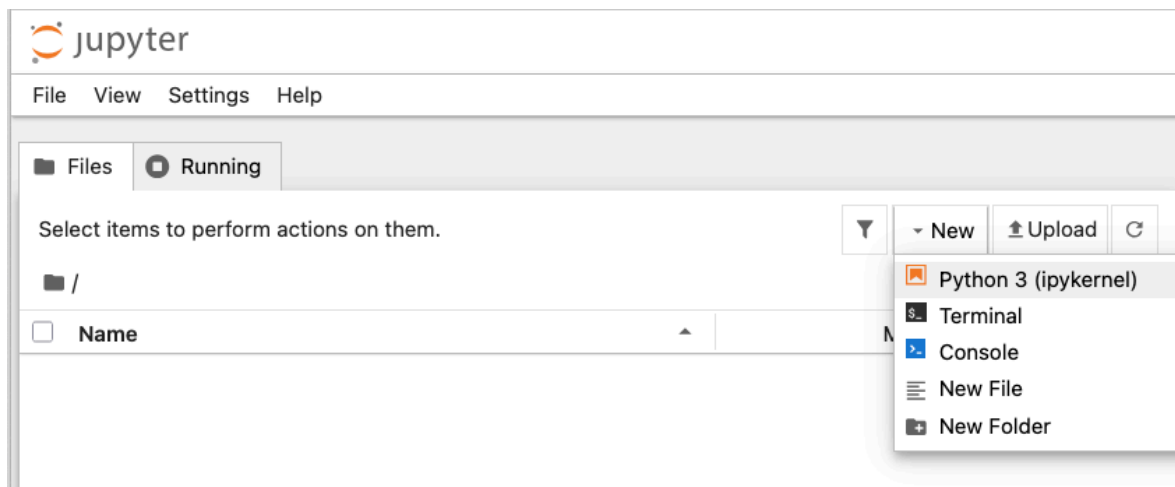
```
 $ uv venv 1
Using CPython 3.12.5 2
Creating virtual environment at: .venv 3
Activate with: source .venv/bin/activate 4
```

その環境に jupyter をインストールする。

```
 $ uv pip install jupyter 1
Using CPython 3.12.5 2
Creating virtual environment at: .venv 3
Activate with: source .venv/bin/activate 4
```

jupyter を起動する。

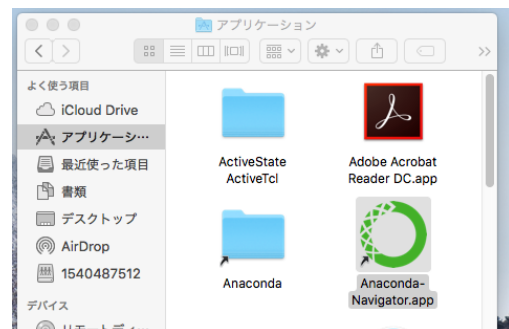
```
 $ uv run jupyter notebook 1
```



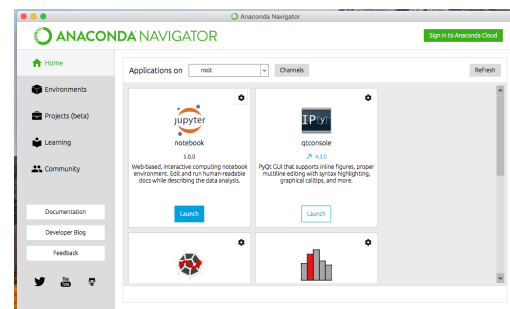
Anaconda

Anaconda をインストールしてあった場合

- アプリケーションフォルダを開く
- Anaconda-Navigator を起動



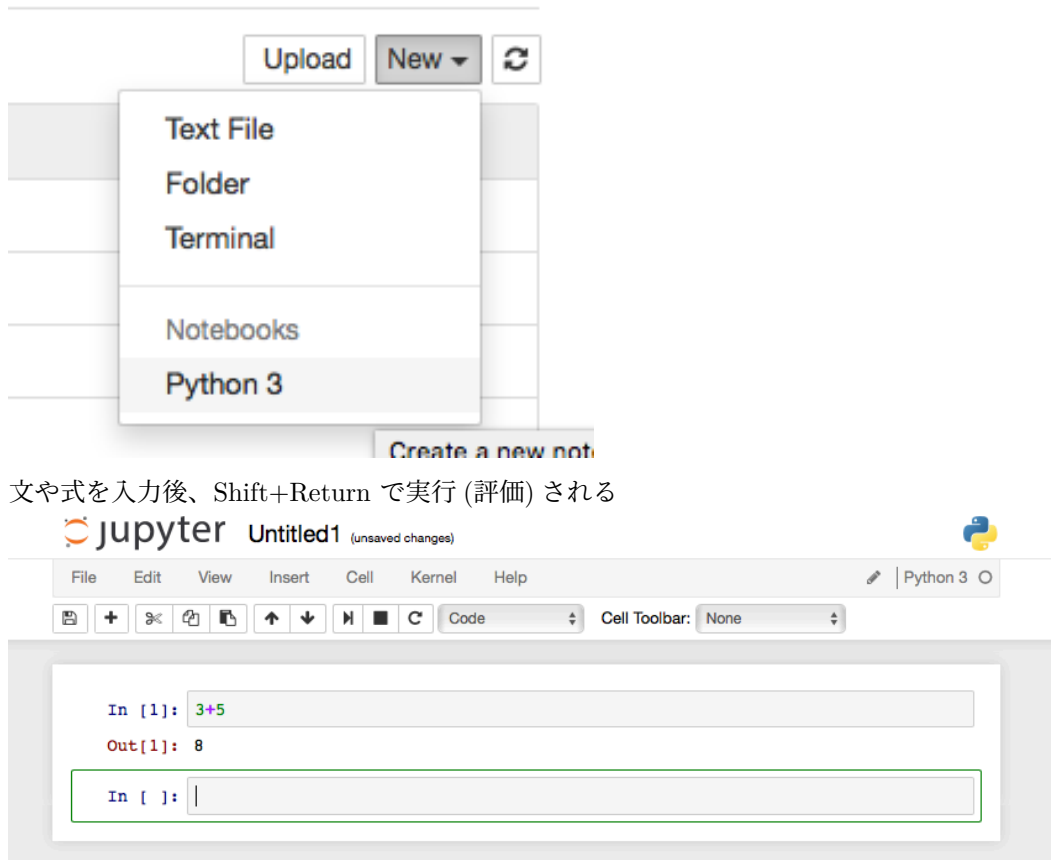
Jupyter Notebook を起動



ブラウザに *jupyter* が起動する



右上の New というメニューから Python3 を選択



文や式を入力後、Shift+Return で実行 (評価) される

1.2.3 Google Colaboratory

jupyter のインストールに困難があった場合は、当面の代替手段として *Google Colaboratory* を使っても良い。ほぼ同様に動作する。ただし、トレードオフがあるので、両方使えることが望ましい。授業としては、早めに jupyter に移行することを推奨する。

<https://colab.research.google.com/>

1.3 プログラムの作成と実行



プログラミングをどこから説明すると分かりやすいかは議論があるが、今年のこの演習では文の列から開始する:

- プログラムは「文」 (*statement*) の列から構成される
- 文は原則として前から順に実行される
- 「文」としては `print` (表示) のみを扱う。「実行」とは表示が行われることを意味すると

する。

- 文の実行順序を変更する仕組みである制御構造を2つ導入する。ひとつは繰り返し `for`，もうひとつは条件分岐 `if` である

プログラムを構成する重要な概念である，式，変数，関数については次章以降に導入する。なるべく授業時間中に例題や問題に多く取り組んで，教員にチェックを受けること。

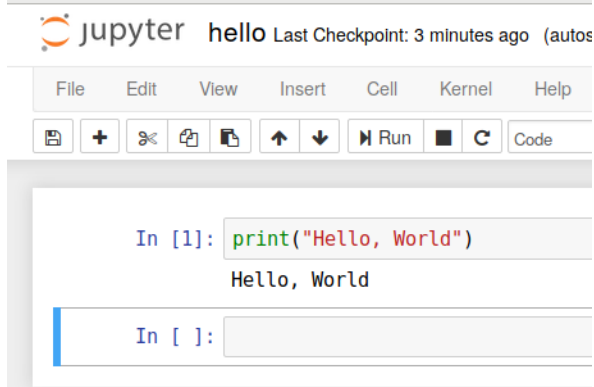
慣例に則り `Hello, world` と表示するプログラムを作成しよう。

Python3

```
print("Hello, world")
```

1

jupyter-notebook 上で上記を入力し，実行ボタンを押し，表示を確認する。



以降の教材では，実行結果をスクリーンショットに代えて，次のようなテキストで表記する。行頭に `>>>` がある行はプログラムの入力を示し，他の行はシステムの応答を示す。これは端末で Python インタープリタを使う場合の表記と同じである。



```
>>> print("Hello, world")
Hello, world
```

1

2

1.4 順次の実行

プログラムは「文」の列から構成される。この資料では(まずは)文として `print("xxx")` という形式の Python の表示 (`print`) 文を扱う。この文が実行されると二重引用符で囲まれた文字列を表示するという効果を持つ。演習環境では，日本語など ASCII に含まれない文字も表示可能だが，文字コードを `utf-8` にしておくこと (<https://hwb.ecc.u-tokyo.ac.jp/hwb2023/information/text/character/>)。

 型: 文字列 str

1 重引用符, または, 2 重引用符で囲まれた文字の列. 他の多くの言語 (例えば C++, Java, sh) では, 1 重引用符と 2 重引用符の機能には差があるが, Python では等価である.

例 'Hello', "Hello", '3+5', 'abcXyZ+', '_'

Python3

```
print("春眠不觉晓") 1
print("处处闻啼鸟") 2
print("夜来风雨声") 3
print("花落知多少") 4
```

改行文字は任意の場所に入れられるので, 以下のように文との対応を変えても結果は同じである. また#から行末までは, コメントとして扱われ, 実行に影響を与えない. さらに, **改行**を表す文字 \n を用いて, 文字列の途中で改行を行うことができる. なお, 改行文字を用いなくても print 文の終了時には改行が自動で行われる. この自動的な改行を抑制する手法は後で述べる.

Python3

```
print("春眠不觉晓") 1
# ここで息継ぎ 2
print("处处闻啼鸟\n夜来风雨声\n花落知多少") 3
```

次の分は, 改行を 2 回行う.

Python3

```
print("\n\n") 1
```

例題 改行 (newline)

改行を適切に増やして, 春曉の各行の間に空行が表示されるようにして, 読みやすくせよ.

1.5 整数と繰り返し

文字列の他に 123 のような**整数**を扱うことにし, print(123) という表示を導入する.

 型: 整数 int, 浮動小数 float

int はほぼ数学の整数と同等. float は実数に相当するが, 有限精度の近似表現である. 2 項演算子 +, -, *, // を用いた算術演算ができる.

int 1, -352, 777777

float 0.5, -3e7, 777777.0

以下のコードは各行に 1,2,3 の数字を一つずつ表示する。

Python3

```
print(1) 1
print(2) 2
print(3) 3
```

さて、3 までならば容易だったが、100 まで表示する場合に同様に記述すると 100 行も必要である。これは苦痛である¹⁾。そこで次のような for という略記法が用意されている。

Python3

```
for i in range(1,4): 1
    print(i)          2
```

これは 2 行目の**制御変数** `i` の部分を 1,2,3 と次々と置き換えながら、for 文以降の一定の範囲を繰り返し実行する文法である。



定型パターン for

```
for 制御変数 in range(始点, 終点): 1
    ... # 繰り返し実行する文 または ブロック 2
```

`range(始点, 終点)` の区間は、始点を含み終点を含まない。制御変数は、始点、始点+1, ..., 終点-1 に順に置き換わる。繰り返す範囲は、**インデント**と呼ばれる行の先頭の空白文字で制御する。インデントの揃った 1 つ以上の文を**ブロック**と呼ぶ。上記の 2 行目には、先頭に空白文字を 2 つ入れてある。Python ではこれらの空白文字の有無がプログラムの動作に影響を与える。

今後、プログラムの意味を明確に保ったまま短く書くための概念や手法がしばしば登場する。計算機の気持ちになって実行内容をシミュレートする能力と、複雑な動作を簡潔に表現する能力の両方を意識すると良い。

例題 100 (p100)

1 から 100 まで表示せよ。

例題 棒 (bar)

以下のような棒を表示せよ。以下の例では長さ 4 だが 70 など長いものも簡単な変更で書けるように作成し、実際に 70 で表示してみよ。

1) もしかすると数百行ならものともしないという人もいるかもしれないが、苦痛に思ってください

長さ 4 のものは

Python3

```
print("****")
```

1

とすれば十分だが、70 にするには ‘*’ を 70 個正確に入力する必要が生じて苦痛である。そこで `for` の繰り返しを応用する。

Python3

```
for i in range(70):
    print("*", end="")
print()
```

1

2

3

このように書くと ‘70’ の部分を変更するだけで任意の長さに変更可能である。ここで `range(70)` は `range(0,70)` の省略記法である。一般に `n` 回繰り返したい場合は `range(n)` と書くと良い。

なお `for` の繰り返しの意味は先ほどと変わらないが、繰り返しの内部で `i` が使われていないので、「`i` の部分を 1,2,3 と次々と置き換えながら」という部分は意味を持たず、単に 70 回繰り返す指示と等価である。また `print("*", end="")` の `end=""` は、`print` 実行終了時に行われる改行を抑制するという意味である。

例題 筋トレ (exercise)

腕立てと腹筋を交互に繰り返すコードは、左右どちらか?

```
for i in range(3):
    print('腕立て')
for i in range(3):
    print('腹筋')
```

1

2

3

4

```
for i in range(3):
    print('腕立て')
    print('腹筋')
```

1

2

3

例題 長方形 (rectangle)

以下のような長方形を表示せよ。以下の例では 4x3 だが 70x20 など巨大なものも簡単な変更で書けるように作成し、実際に 70x20 で表示してみよ。

長さ 4 を 3 行であれば

Python3

```
print("****")
print("****")
```

1

2

```
print("****")
```

3

と書くこともできるが、まず行数を簡単に変更可能とするために、1行ずつ for の繰り返しを応用する。

Python3

```
for i in range(3):
    print("****")
```

1

2

これは改行部分を別にして以下のように書くことができる。

Python3

```
for i in range(3):
    print("****")
```

1

2

さらに内側の星の表示部分を「棒」の例題のように、for の繰り返しを応用する。

Python3

```
for i in range(3):
    for j in range(4):
        print("*", end=" ")
    print()
```

1

2

3

4

このような二重の繰り返しを**二重ループ**と呼ぶ。二重ループでは、内側のループがまず実行されてから外側のループがひとつ進む。

例題 三角 1 (triangle)

以下のような三角形を表示するものを定義せよ。例では大きさが3であるが、大きさ5,6なども少しの編集で簡単に試せるように作成せよ。

```
*
**
***
```

考え方: 長方形との差は、行ごとに長さが異なる点である。行の長さは1..4の4で決まっているので、この4を行に対応させれば良い

Python3

```
for i in range(3):
    for j in range(i+1):
        print("*", end=" ")
    print()
```

1

2

3

4

この例のように、range(i) の i の部分は数に対応するものを書くことができる。range(i) に代え

て, `range(i*4-3)` とするとどうなるだろうか. 実行して説明せよ.

1.6 条件分岐

別の制御構造として, 文を実行したりしなかったりする文法 `if` を導入する.



条件分岐 `if`

```
if 条件:
    条件が成り立ったら実行する文 またはブロック
else:
    条件が成り立たなかったら実行する文 またはブロック
```

1
2
3
4

`else` 以降は省略可能

例題 偶数奇数 (oddeven)

1 から 10 までの整数について, 偶数であれば `even`, 奇数であれば `odd` と表示するプログラムを表示せよ.

```
1 odd
2 even
(中略)
10 even
```

Python3

```
for i in range(1,11):
    print(i, end=" ")
    print(" ", end=" ")
    if i % 2 == 0:
        print("even")
    else:
        print("odd")
```

1
2
3
4
5
6
7

(条件分岐は次週以降も取り扱うので, 困難を感じた場合は次週以降にもう一度取り組んでも良い.)

1.7 総合問題

問題 九九 (p99)

9 列 9 行の整数で、かけざん九九の表を表示するプログラムを作成せよ。整数 i と j の積は $i*j$ で得られる。その表示は `print(i*j, end=" ")` で得られる。空白は `print(" ", end=" ")` と表示する。

(桁をそろえたい場合は `print(f'{i*j:3d}')` などのように、`f'...'` という波括弧内に式と書式指定を書く文字列表現を使うことができるが、揃えなくても良い。)

問題 筋トレ 3 セット (train)

腕立てと腹筋を交互に 2 回ずつ行うトレーニングを 3 セット行うメニューを考える。セット終了時に 5 分休憩するとする。print 文の使用は、4 行に抑えつつ、右の表示を得るコードを記述せよ。

トレーニング始め!

腕立て

腹筋

腕立て

腹筋

(5 分休憩)

トレーニング始め!

腕立て

腹筋

腕立て

腹筋

(5 分休憩)

トレーニング始め!

腕立て

腹筋

腕立て

腹筋

(5 分休憩)



文法: 定数

整数や文字列の値の別名を定数と呼ぶ。この資料では、**定数**には大文字で始まるアルファベット列を用いる。別名には、主にアルファベットを用いる。アンダースコア_と 2 文字目以降には数字も利用可能である。定数とは、プログラム中からは変更できない固定値、という概念である。実際のところ、Python では変更できてしまうので、プログラム作成者が注意しなければならない。多くの言語では、変更不可の定数を言語機能で実現している。たとえば C++ の場合

は `const` 修飾子を用いる.

```
N = 3
u_tokyo_id = 987654321
```

1
2

問題 三角 2 (triangle2)

右図のような三角形を表示するものを定義せよ。例では大きさが4であるが、大きさ5,6なども少しの変更で簡単に試せるように作成せよ。

```
****
***
**
*
```

問題 矢印 (arrow)

右図のような矢印を描くプログラムを作成せよ。前の問題同様に、大きさを簡単に換えられるように作成すること。矢印は、整数 N に対して、幅 $2N+1$ 高さ $2N+1$ を取る。例は $N=3$ の結果である。

```
*
***
*****
*****
*
*
*
```

なお、アスキーアートを想定通りに表示するためには、等幅 (とうはば) フォントをウェブブラウザで用いる必要がある。²⁾

例題 外枠 (frame)

しばらく、以下のパターンで $N \times N$ の正方形に類する形を扱うことを考える。 N は3以上の整数とする。

```
N = 5
for r in range(N):
    for c in range(N):
        print('#', end='')
    print('')
```

1
2
3
4
5

2) たとえば、担当教員のウェブを参照 <https://www.graco.c.u-tokyo.ac.jp/~kaneko/lecture/colab/#ascii-art>

このコードを変形して，次のような外枠を表示するプログラムを作成せよ．なお以下は $N=7$ の場合である．

```
#####
#      #
#      #
#      #
#      #
#      #
#####
```

問題 本棚 (shelf)

次のような本棚を表示するプログラムを作成せよ．以下は $N=9$ の例である．

```
#####
#          #
#####
#          #
#####
#          #
#####
#          #
#####
```

問題 扉 (door)

次のような扉を表示するプログラムを作成せよ．以下は $N=9$ の例である．

```
#####
#          #
# ##### #
# ##### #
# ##### #
# ##### #
# ##### #
#          #
#####
```

問題 森 * (forest)

[3,5,7,8] などの数値列を与えられて，対応する大きさの木を横に並べて描くプログラムを作成せよ．ただし，1 本の木は上の arrow とする．条件式を頑張ると，for と if だけで実装できる。^a

([3,5,7,8] などの配列は次回扱うので，スキップ可能)

a 実用的には、画用紙に相当する 2 次元配列を扱うと実装が楽だが、現時点では範囲外.

1.8 この章の課題

提出対象: 以下のいずれか

- 問題 (★なし) を全て Python で解く. ただし, 1.7 節 総合問題の正方形シリーズはどれか一題で良い. (通常コース)
- ★つきの問題を解く (通常コースの内容を概ね習得済みの人)

提出先: UTOL

提出時期:

- 標準提出期限 (全員): 4 月 13 日 (月) までになるべく完成させて提出する. スタッフができ具合をコメントする. (場合によっては再提出になる)
 - 授業中に完成した小問については, 授業時間中にスタッフ (教員または TA) にチェックを受けて OK をもらう. スタッフ名をコメントに付記する (出席の確認にも用いる).
 - 一部途中の場合は, 作成途中と付記する.
- 学期終了時: 最終的な成績は最後に提出されたものを (改善コメントを受けて改善した提出があればそちらを) 用いる.

提出条件:

- 提出ファイルは, *jupyter から作成した* .ipynb ファイルとする. ファイル名を weekN-(学生証番号).ipynb としておくこと. Colab で演習した場合は, ブラウザ画面左上の File メニューから Download, Download .ipynb と選択して, 得られたファイルの名前を整えて提出する.
- 日本語 (または英語) でコメントが付記されていること: { 途中か完成品か, 授業中に既に OK を得ていればその教員名 } 学んだこと, (あれば) 他に参考にした資料や他の人の回答など, (途中で提出する場合は何がうまくゆかずに困っているなど) をコメントで盛り込む. (何も苦労がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること — 授業中に教員の OK を得た場合は簡潔で良い)

第2章 式と評価, 関数

Expression, Evaluation, and Function
2026-04-15 Wed

事務連絡

第1, 2週の授業への参加に困難がある(あった)場合は, 状況を担当教員に相談してください. Zoomについて, (1) なるべく profile で本名 (full name) を登録, (2) 大学のアカウントでログイン, としてから授業に参加してください.

概要

(前章で説明した「プログラムは文の列である」という理解の仕方とは別に) プログラムは式 (expression) を組み合わせたものであり, 式を評価する (evaluate) ことで計算が進行するという考え方を紹介する.

学部生: なるべく授業時間中に例題や問題に多く取り組んで, スタッフ (教員または TA) にチェックを受けること (必修授業としての履修者は特に).

大学院生: これまでの提出に問題なければチェックを義務としない. ただしチェックを受けなかった場合は, 特に気をつけて UTOL のコメントを確認すること.

2.1 式の評価

jupyter notebook で, 式 (expression) を入力すると評価 (evaluate) した結果を返す:

```
In [1]: 1+1
```

```
Out[1]: 2
```

これを本資料では, これ以降, 以下の書式で書く. >>> の部分はユーザの入力を示す記号であり, 実際には入力しない. 同様に##から行末までは注釈であり, 実際には入力しないし, 表示もされない. また, 1+1 のあとに Shift+Enter などセルを実行しているが, その手順は表記していない.



```
>>> 1 + 1 # ユーザの入力
2 # Python の返答
```

1
2

ここで 1+1 を式, 2 を値 (あたい, value) という. また値は型 (type) を持ち, 2 の型は整数である.

! Notice: 重要: 評価 v.s. print

前章の `print` の結果の表示と見た目はほぼ同一であるが、内容が大きく異なることに注意。式はあとで紹介するように別の大きな式の部分式として利用できるが、`print` したものはプログラムの外に出力された扱いとなりその内容を利用することはできない。

```
>>> (1 + 2) + 3 # OK 評価      1
6                                2
>>> print(1 + 2) + 3 # NG ?!  3
3                                4
Traceback (most recent call last): 5
  File "<stdin>", line 1, in <module> 6
TypeError: unsupported operand type(s) for +: 'NoneType' and 7
      ↪ 'int'
```

評価して値を得ることと、表示 (`print`) することを、自身で区別すること。

2.2 関数的な計算: 式と評価

📌 約束

本日は、変数への値の代入 ($\neq$ 初期化) は用いないこと。また、先週説明した `for` も用いないこと。`print` はデバッグの表示用途のみ使って良い。

📌 式 (最初の定義)

式は、

- 原始的な式 (atomic expression) と、
- 式を演算子で適切に組み合わせたもの (compound expression)

である。

2.2.1 原始的な式

数は原始的な式である。(例: 345, 3.1415)

真 (True) と偽 (False) の真偽値 (bool) や、"Hello"などの文字列 (str) も原始的な式である。

2.2.2 式の組み合わせ

二項演算子 (binary operator) を使って式を組み合わせることができる。Python や C++の多くの

式は,

(式 演算子 式)

と演算子を中央に配置する (中置記法). 例: $450 * 1.05$

このような式をさらに組み合わせたものもまた、式である。式を組み合わせてできた式について、部品となる式を**部分式**という。たとえば、整数と、四則演算 $+$, $-$, $*$, $/$, 剰余 $\%$, 単項 $-$, $()$ を自然に組み合わせたものは式である。

通常は括弧を省略する。1-2-3-4-5 は $((((1-2)-3)-4)-5)$ という意味である。一意に括弧を復元できるように、演算子の優先順位 (e.g., 乗算は加算に優先する) や同じ順位の場合は左から評価するなどの規則が、プログラミング言語ごとに定められている。

同じ名前の演算であっても、対象となる式の持つ型 (type) 毎に異なる意味を持つ。たとえば、Python を含む多くの言語で、 $+$ 演算子は、整数型については和を、文字列型については連結の効果を持つ。また、C++などのプログラミング言語では**整数の除算は整数を返す**ことがある (例えば、C++では $5/3$ と $5/3.0$ で異なる値を返す)。Python3 では、整数と浮動小数点数の両方で除算演算子 $/$ は浮動小数点数の演算として行われる。整数の除算の商を求めたい場合は、演算子 $//$ を用いて $5//3$ という式を書く。

2.2.3 変数: 値に名前をつける



式 — 変数の導入 —

式は,

- 原始的な式 (atomic expression),
- (定義済みの) 変数 (定数) — new! —
- 式を演算子で適切に組み合わせたもの (compound expression)

である。

変数 (variable) を使うことで計算された**値** (value) に名前をつけることができる。名前に使用可能な文字は言語によって異なる。この資料では特別な場合を除いて、先頭を英字 (アルファベット), 2 文字目以降には英字の他に数字とアンダースコア ($_$) を用いる。

変数を用いることで、次のような利益がある。

- 意図が不明瞭な値に名前を付けることができるため、プログラムの意味がより分かりやすくなる。
(具体的な値を使っている場合よりも、より**抽象化**されたプログラムにすることができる)
- 同じ計算式が何度も使われる場合に、変数によって置き換えることで式が簡単になる
- 何度も使われる値を変更する際に、変数の定義だけを変更すればよくなる

値が変更されない変数のことを特に**定数** (constant) と呼ぶ。この資料の Python ではアルファベット大文字で始める。C++では `const` 修飾子を付与する。

Python3

Size = 2

1

C++

const int Size = 2;

1

と書くと、Size という名前の変数と 2 という値が結びつけられる。以降、2 と書くかわりに Size と書いても全く同じ値が得られる。つまり、変数もまた式である。

値に名前を付けることは**抽象化**の第一歩である。例えば、

Python3

```
Pi = 3.1415
Radius = 2
```

1

2

のような変数定義をしたとする。すると、

Python3

```
3.14 * 2 * 2      # (円周)
3.14 * 2 * 2      # (面積)
```

1

2

と書いたのに対して

Python3

```
Pi * 2 * Radius   # (円周)
Pi * Radius * Radius # (面積)
```

1

2

と、より意味を明瞭に書くことができる。

後者の書き方はプログラムの保守性を向上させる。たとえば半径を 2 から 10 に変更したい場合を考えよう。

2.2.4 関数: 計算に名前をつける



式 — 関数の導入 —

式は、

- 原始的な式 (atomic expression),
- (定義済みの) 変数 (定数)
- (定義済みの) 関数 — new! —
- 式を演算子で適切に組み合わせたもの (compound expression)

である。

計算の手順にも名前を付けることができ、これを**関数** (function) と呼ぶ。関数の基本形を次のように導入する。



基本パターン: 関数

まず関数を以下のパターンで定義する。

```
def 関数名 (仮引数 1, 仮引数 2, ..., 仮引数 n):  
    return 式
```

1
2

上記のように定義した関数は

```
関数名 (引数 1, 引数 2, ..., 引数 n)
```

1

のように式として使うことができる。キーワード `return` の右に書いた式が、関数を評価した際の値となる。

例えば、3次元空間での距離の自乗を計算する $(x * x) + (y * y) + (z * z)$ という式を考える。この式には、「何かを二乗する」計算が頻出している。つまり、自乗の対象は x や y などと異なるが、自乗という手順は共通である。そこで、変化する部分を **引数 a** とおき、残る共通部分である「何かを二乗する」部分を関数としてまとめてみよう。

Python3

```
def square(a):  
    return a * a
```

1
2

すると、先の式は `square(x) + square(y) + square(z)` と書くことと等価である。つまり、関数も式を構成する要素として利用可能である。

関数を定義すれば、その関数の名前と「何を計算するか」さえ覚えておけば、複雑な定義の詳細を忘れてしまうことができる。これは、**計算手順の抽象化** (abstraction) の第一歩である。

C++

```
int square(int x) {  
    return x * x;  
}
```

1
2
3

例題 長方形の面積 (rectarea)

二辺の長さがそれぞれ a, b である長方形の面積をもとめる関数 `rectarea(a, b)` を定義せよ

関数を作成したら、具体的な値で実行結果を確認しよう



```
>>> rectarea(2, 3)  
6
```

1
2

```
>>> rectarea(10, 5)
50
```

3
4

例題 距離 (distance)

座標 (x, y) 上の点の原点からの距離を返す関数 `distance(x, y)` を定義せよ。

平方根は `math.sqrt()` という関数が用意されている。ただし事前に `import math` という使用宣言を行う。あるいは、べき乗を表す `**` 演算子を用いて、`0.5` 乗しても良い。



```
>>> import math
>>> math.sqrt(3)
1.7320508075688772
```

1
2
3

これ以降の問題も、自身で同様のテストを行って確認せよ。

2.2.5 関数内の変数

関数の中でも、値に名前をつける目的で変数を使うことができる。



基本パターン: 内部で変数を使う関数

```
def 名前(仮引数 1, 仮引数 2...):
    変数 1 = 式
    変数 2 = 式
    ...
    return 式 # 式には、仮引数に加えて変数 1, 2.. を利用可能
```

1
2
3
4
5

例題 三角形の面積 (triangle_area)

3 辺の長さ a, b, c に対応する三角形の面積を計算する関数 `triangle_area(a, b, c)` を作成せよ。面積は数式 $\sqrt{s(s-a)(s-b)(s-c)}$ で与えられる。ただし $s = (a + b + c)/2$ である。

回答例

Python3

```
def triangle_area(a, b, c):
    s = (a + b + c) / 2.0
    return # ここに s, a, b, c を使った面積の式を書く
```

1
2
3

2.2.6 様々な型と演算



型: 真偽値

真偽値 (Boolean) は **True** または **False** のどちらかをとる. 二つの整数 (あるいは文字列) を比較演算子 (等しい`==`, 等しくない`!=`, 大きい`>`, 大きいか等しい`>=`, 小さい`<`, `<=`) で結んだ式を評価すると真偽値を得られる.

```
>>> 3 < 5
True
>>> (3 + 5) < 7
False
```

また真偽値同士の演算として, 論理積 **and**, 論理和 **or**, 単項演算として否定 **not** が用意されている. なお C++ などではそれぞれ `&&`, `||`, `!` の記号を用いる.

例題 除算 (`is_divisible`)

整数 x が y で割り切れる場合真, そうでなければ偽を返す関数 `is_divisible(x, y)` を定義せよ.

例題 Range (`in_range`)

3つの整数 a, b, c に対して, それらが $a < b < c$ の条件を満たすならば **True** を, 満たさないならば **False** を返す関数 `in_range(a, b, c)` を作成せよ

ヒント: Python では, 比較演算子を chain することが出来る (!)¹⁾. すなわち, $a < b < c$ と書くと期待通りの真偽値を得られる.

一方, 他の言語 Ruby や C++ では, 二項演算子では3つの数を同時に比較することができない. その場合は, $a < b$ と $b < c$ をそれぞれ判定してから, 論理積を取る.

例題 ほぼ一致 (`close_enough`)

2つの数 (x, y) が十分に近い場合に真, そうでない場合に偽を返す関数 `close_enough(x, y)` (十分に近いとは, 定数 $\Delta > 0$ を使って $|\frac{x-y}{y}| < \Delta$ である場合とする. $|y| > \Delta$ は仮定して良い. Δ は自分で適当な値に定める)

1) <https://docs.python.jp/3/reference/expressions.html#comparisons>

例題 平方根判定 (is_sqrt)

数 x, y を受け取り, x が y の平方根になっていれば真, そうでなければ偽を返す `is_sqrt(x, y)` (ただし, 数値誤差を考慮して, x^2 と y が十分に近いことを判定することにする. 直前の例題参照)

例題 倍数 (multiple_of_two_or_three)

与えられた数が 2 または 3 の倍数であるときに真, そうでないときに偽となる関数 `multiple_of_two_or_three(x)`

問題 Circle in a Rectangle (in_rectangle)

長方形の中に円が含まれるかを判定する関数 `in_rectangle(w, h, x, y, r)` を作成せよ. 長方形は左下の頂点を原点とし, 右上の頂点の座標が (w, h) である. 円は中心の座標が (x, y) 半径が r である. 円が含まれるなら True を, そうでなければ False を返すとする.

図解 http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ITP1_2_D

文字列 "Hello"のように二重引用符で囲まれたものは**文字列**である. 文字列に対しては, +演算子が結合の意味を, *演算子が+演算子を与えられた数だけ繰り返す意味を持つ.

```
>>> "Hello" + "World" 1
'HelloWorld' 2
>>> "Hello"*3 3
'HelloHelloHello' 4
```

```
>>> "Hello".upper() 1
'HELLO' 2
```

ドットは**メソッド呼び出し**を表す. この演習内では `upper("Hello")` のように関数の特殊表記として理解してよい.

問題 Watch (watch)

0 時からの経過秒数 s に対応する hh:mm:ss 形式の文字列を返す関数 `watch(s)` を定義せよ. 数値が 1 桁の場合, 0 を付けて 2 桁表示をする必要はない. (後の方を参考にすれば 0 をつけてもよい)

`watch(46979)` は 13:2:59 という文字列を返す.

実行例補足: `"lunch time --- "+watch(46979)+"!?"` を評価すると `"lunch time --- 13:2:59!?"` となる.



文法: 型変換

整数の 1 と 2 をコロンでつなげた文字列は `str(1)+":"+str(2)` で得られる。整数は `str` という関数により文字列に変換される。逆方向の変換は、`int` で行う。

```
>>> 1
1
>>> str(1)
'1'
>>> str(3 + 5)
'8'
>>> int('8')
8
```

C++の場合は、`std::to_string` という標準関数が、整数を文字列 (`std::string`) に変換するために用意されている。ただし、古い環境では、コンパイルオプション `-std=c++11` が必要かもしれない。

C++11

```
#include <string>
#include <iostream>
using namespace std;
// 整数 a と b をコロンでつなげた文字列を返す関数
string concat(int a, int b) {
    return to_string(a)+":"+to_string(b);
}
// 実行例
int main() {
    cout << concat(3, 5) << endl;
}
```

問題 Abbreviation (abbreviation)

与えられた文字列の (1) 先頭 1 文字, (2) 「(文字列全体の長さ) - 2」という値の整数 (を文字列に変換したもの), (3) 最後の 1 文字をつなげたものを返す関数 `abbreviate(word)` を定義せよ。例えば, `"internationalization"` であれば `"i18n"` を返す。

文字列 `X` に対して, 先頭の文字は `X[0]`, 末尾の文字は `X[-1]` で得られる。また, `len(X)` は文字列の長さを与える。

C++では, `string` 型の文字列 `X` に対して, `X.substr(n,1)` が `s` の `n` 文字目 (からはじまる 1 文字の) 文字列を, `X.size()` が `X` の文字数を返す。

実行例補足: `abbreviate("internationalization")+"!!!"` を評価すると `"i18n!!!"` となる。

2.2.7 条件判断

条件によって計算の内容を変えることによって、より複雑な計算をさせることができる。if と else は条件によって式を切り替える文法である。



基本パターン: if を使う関数

関数の基本パターンの最後の文である `return` 式 を次の 4 行で置き換える。

```

if 条件 (真偽値になる式):           1
    return 条件が真の場合の式       2
else:                                3
    return 条件が偽の場合の式       4

```

例えば、絶対値を求める関数は数学的には

$$|x| = \begin{cases} x & (x \geq 0) \\ -x & (\text{otherwise}) \end{cases}$$

であるが、これが次のように書ける:

Python3

```

def abs(x):                           1
    if x >= 0:                         2
        return x                      3
    else:                              4
        return -x                     5

```

この `return` をさらに置き換えることで、条件を入れ子にすることもできる。

Python3

```

if 条件 (真偽値になる式):           1
    # 以下の 4 行全体が、条件 1 が真の場合の return 式 に相当 2
    if 条件 2(真偽値になる式):       3
        return 条件 1 も 2 も真の場合の式 4
    else:                             5
        return 条件 1 が真で条件 2 が偽の場合の式 6
else:                                 7
    return 条件が偽の場合の式       8

```

条件が成り立たない場合に更に `elif` を用いて他の条件で場合分けすることもできる:

Python3

```

if 条件 1:                                1
    return 条件 1 が真の場合の式          2
elif 条件 2:                              3
    return 条件 2 が真の場合の式          4
elif 条件 3:                              5
    return 条件 3 が真の場合の式          6
...                                       7
else:                                     8
    return どの条件も偽の場合の式          9

```

C++

```

int abs(int x) {                          1
    if (x >= 0)                            2
        return x                          3
    else                                    4
        return -x                         5
}                                          6

```

例題 点数を成績に (grade)

関数 `grade(g)` として, g が 80 以上なら "A", g が 65 以上なら "B", g が 50 以上なら "C", それ以外なら "D" を返す関数を定義せよ.

例題 AMPM (meridiem_hour)

与えられた時刻が午前 (0~11) だった場合 "am" という文字列を, 午後 (12~23) だった場合 "pm" という文字列を返す関数 `meridiem_hour(h)` を定義せよ.
(それ以外の値には "error" という文字列を返すとする.)

例題 $3n+1$ (collatz_step)

n が偶数の場合に $n/2$ を, n が奇数の場合 $3n+1$ を返す関数 `collatz_step(n)` を定義せよ.

問題 信用 (issueCC)

(年収に対する) 借金返済額 dt , 返済遅延回数 dl , 年収の額 ic に対して, 以下の決定木に従って, クレジットカードを発行するかどうかを真偽値 (True か False) で返す関数 `issueCC(dt,dl,ic)` を定義せよ.

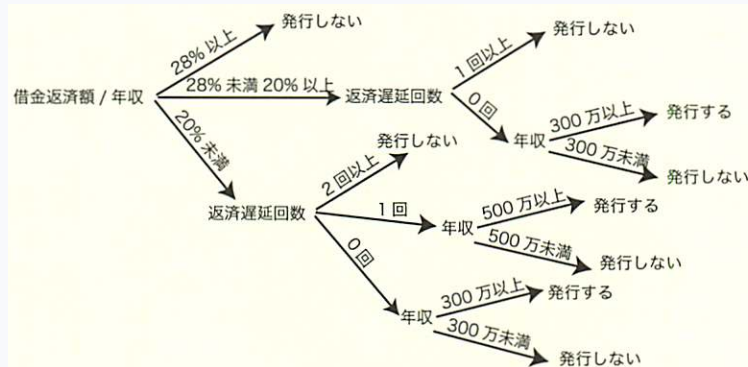


図 4.26 決定木

注: True は、文字列"True"とは異なる。

問題 Simple Calculator (calculator)

二つの整数 a, b と四則演算 op に対応する値を返す関数 $calculator(a, op, b)$ を作成せよ。たとえば $calculator(1, "+", 2)$ は 3 を返し、 $calculator(3, "/", 2)$ は 1 を返す。

注記: $calculator(calculator(3, "*", 10), "+", 7)$ は、 $3 \cdot 10 + 7$ を計算し 37 を返す。

2.2.8 計算の順序

複雑な式を評価するとき、実際にはどのようにして計算が行われているのだろうか？

例えば「 $(3 * 3) + (4 * 4)$ 」のような式からどうやって 25 が求められたり、 $square(8)$ という式からどうやって 64 が求められているのだろうか。

実際には、プログラミング言語毎に様々な規則があり、簡単には説明できない。しかし、本章で扱ったような式を中心とした関数的な(副作用を含まない)プログラムの場合は、以下のように、比較的簡単なモデルで演算した場合と一致するように作られている。

原始的な演算子の場合: 式を評価する手順は、

- まず部分式を全て評価する。
- 演算子に部分式を評価した値を適用 (apply) する。

「適用する」とは、+ だったら実際に加算を行うことであり、計算機の内部では「加算をする機械命令」が実行される。

のように定義できる。この定義は(後で述べるように)再帰的なものである。

関数を含む場合: 関数を部分式として含む式の場合、部分式全体を関数の定義部分で置き換えて評価を進める。その際に、仮引数の変数は実引数によって置き換える。

例えば、

Python3

```
def double(x): 1
    return x + x 2
```

という定義があったときに、`double(3)` という式を評価すると、「`double(3)`」を、 $x = 3$ に注意して、 $3 + 3$ という式に置き換えて評価が続けられる。

こみいった式の場合は、どの式から置き換えをしてゆくか選択肢がある。Python や C++ では内側の式、関数の内側の引数や複式の評価が行われてから外側の式が評価される。

```
square(double(3))
```

とした場合に

```
square(double(3))
```

```
square(3+3)
```

```
square(6)
```

```
6*6
```

のように計算が進む。

短絡評価: 効率的な評価のために、論理演算では「左から評価を始めて、答えが決まったらそこで打ち切る」というルールが多く言語で導入されている。たとえば (例のための例で現実感がないが)

```
x = (square(5) == 30) and (square(6) == 36)
```

と書いた場合、`square(5)` の計算は行われるが、`square(6)` の計算は行われない。これは論理積演算子の左側の項 (`square(5) == 30`) が偽となったため、右の項の真偽値によらず式全体の値が偽となることが分かったためである。これにより、無駄な計算を省くことができる。

比較として

```
x = square(5) + square(6)
```

という演算では、`square(5)` と `square(6)` がまず計算されてから、`+` の計算が行われる。

2.3 参考: 命令的な計算

前の section で見てきた計算は、四則演算のように何度計算しても結果が変わらないようなものであった。このようなものを関数的 (functional) な計算という。

現実世界のプログラムでは、計算をする度に結果が変わるものや、計算をした結果、外界に影響を与えるものごとくも扱う必要がある。このようなものを命令的 (imperative) な計算と言う。

例:

- キーボードから 1 行分の文字列を入力をする: `input()`
- 画面へ文字列を出力する: `print("To be or not to be ...")`
- 乱数の発生: `random.random()`

このような計算を扱う場合には、実行の順番に依存して結果が変わる。そのために命令的な実行を扱うプログラミング言語では、前章で紹介した「文」という実行順序に関する概念を設けて、「原則として前から順に実行される」というルールを導入する。今後この資料では、文とは、`print` だけでな

く、すべての式、変数の初期化、代入を含む。

関数的に記述したプログラムのほうが、動作確認の手間が圧倒的に少ない。そこで命令的な計算の使用は小規模に留めて、なるべく関数的な計算でプログラムを構成することがお勧めである。

命令的なプログラムを扱う場合は、プログラミング言語の仕様に対する正確な理解が必要となる場面が多い:

C++

```

1 int a() {
2     cout << "a" << endl; // 1 を返すついでに a と表示
3     return 1;
4 }
5 int b() {
6     cout << "b" << endl; // 1 を返すついでに b と表示
7     return 1;
8 }
9 int sum(int x, int y) {
10     return x + y;
11 }
12 int main() {
13     int v0 = a() + b();
14     int v1 = sum(a(), b());
15     cout << v0 << endl;
16 }
```

たとえば部分式の評価順序は C++ では規定されていないので、`a() + b()` であっても `sum(a(), b());` であっても、`a()` と `b()` がどちらが先に評価されるかは環境依存である。式の値は評価順序によらず `1+1=2` であるが、`a` と `b` の表示順序は異なりうる。

C++

```

1 int p = a();
2 int q = b();
3 cout << p+q << endl;
```

予期しない事態を避けるためには、1 文に含まれる式を単純に保つことは有効である。

2.4 発展: 関数を返す関数

比較的新しいプログラミング言語では関数自身を関数の引数に渡したり、関数の返り値とする機構が用意されている。

説明の簡潔さのため、単独の引数 `x` をとる関数を以下では考える。

Python3

```

1 def xplus3(x):
2     return x + 3
```



```
>>> xplus3(1)
4
```

1
2

変数は、関数を値として持つことができる。



```
>>> f = xplus3
>>> f(10)
13
```

1
2
3

関数を、`lambda` を用いて実行時に作成することもできる。これを匿名関数と呼ぶことにする。関数の呼び出しの際には (通常関数と同様に呼び出せる。



```
>>> f = lambda x: x+3
>>> f(1) # 1+3
4
>>> f(2) # 2+3
5
```

1
2
3
4
5

以下のように複数の引数を取ることもできる。



```
lambda 引数 1, 引数 2, ... : 単一の式
```

1

`lambda` で作成した関数を、関数の返り値にすることができる。次の関数 `plusn` は、『「引数 `x`」に `n` を足して返すような匿名関数』を作成して返す。

Python3

```
def plusn(n):
    return lambda x: x+n
```

1
2



```
>>> f = plusn(10)
>>> f(1)
11
>>> g = plusn(-10)
>>> g(1)
-9
```

1
2
3
4
5
6

`lambda` では、単一の文しか書くことができない。複数行の文を書きたい場合は、`def` による関数定義を使う。

Python3

```
def xplus3():
    def inner_plus3(x):
        y = x + 3
        return y
    return inner_plus3
```

匿名関数を、別の関数に渡して組み合わせた関数を作成することもできる。

Python3

```
def compose_product(f, g):
    # h(x) = f(x)*g(x)
    return lambda x: f(x) * g(x)
```



```
>>> h = compose_product(f, g)
>>> h(1)
-99
>>> f(1)
11
>>> g(1)
-9
```

問題 変数を含む電卓 ★ (calcvar)

変数 x を含む四則演算の式を表す文字列 s を受け取って、引数 x に応じた値を計算する関数を返す関数 `parse(s)` を作成せよ。

簡単のため文字列 s は以下の条件を満たす

- 演算は、Simple Calculator で扱う四則演算のみである
- 数字は、0 から 9 までの 1 文字の整数のみが登場する。
- 変数は、現れない場合もある。現れる場合も x のみである。
- 演算子一つにつきカッコが一組つく。余計なカッコはつかない

この課題は、全員の義務ではない。以下にヒントも用意されているが、再帰や多重代入の利用が想定されるので、経験がない場合は第 5 週以降に考えるのが適しているかもしれない。この課題に限っては C++ を用いても良い。ただし Python の `eval` は使用禁止とする。



```
>>> f = parse("((x+3)+3)")
>>> f(0)
6
>>> f(1)
7
>>> f = parse("((x+3)*(x-3))")
>>> f(1)
```

-8

8

部品ごとに分けて作ると良い。入力のような式も、引数 x を受けて値を返すような匿名関数の組み合わせで、対応できることに注意する。

1. 引数 c が x 一文字か数字位置文字だと仮定する。 c に対応する関数を返す関数 `make_term(c)` を作成せよ。文字が x の場合は identity function を、数値の場合は定数関数を返すとする。



```
>>> f = make_term("x") # f(x) = x      1
>>> f(3)                        2
3                                3
>>> f(5)                        4
5                                5
>>> f = make_term("9") # f(x) = 9      6
>>> f(3)                        7
9                                8
```

2. 一つの演算子を含む長さ 3 の文字列を引数として、対応する関数を返す `parse_one(s)` を作成せよ。以下のような構造になるはずである。また問題 Simple Calculator で作成した関数を呼び出すと簡潔になる。

Python3

```
def parse_one(s):                        # "x+3" => xplus3      1
    lf = ... # s[0] に対応する関数      2
    op = s[1] # operator                 3
    rf = ... # s[2] に対応する関数      4
    def func(x):                        5
        ... # lf, rf, op と x から値を計算する, 組み合わせた関数を定義 6
    return func                          7
```



```
>>> f = parse_one("x+3")              1
>>> f(100)                            2
103                                    3
```

3. (`parse_one` を複製改変して)1 重のカッコを含む式 (例 `"(x+3)"`) に対応する匿名関数を返す関数 `parse_term(s)` を作成せよ。戻り値は長さ 2 の配列とし、先頭要素が対応する関数、次の要素が閉じカッコの位置とする。



```
>>> s = "(x+3)blahblahblah"          1
>>> f,pos = parse_term(s) # 結果を, f と pos の二つの変数で受け取る 2
>> f(3)                                3
6                                        4
```

```
>>> pos
5
>>> s[pos:] # 部分文字列, "(x+3)"を解析した残りに相当
'blahblahblah'
```

4. `parse_term(s)` を変更し, ネストするカッコを含む式に対応させよ. たとえば以下のような再帰で作ることができる.

Python3

```
def parse_term(s):
    if s[0] == "(":
        # カッコを含む式
        # 再帰で, 開きカッコの右から演算子の左側までを解析
        lf, lsize = parse_term(s[1:])
        op = s[lsize+1] # 演算子
        rf, rsize = parse_term(s[...]) # 再帰で, 演算子の右側を解
        ↪ 析
        f = lambda x: ...
        return f, ...
    else:
        # 1文字の式
        f = make_term(s[0])
        return f, 1
```

5. `parse_term(s)` を利用して, 問題に対応する関数 `parse(s)` を定義する

Python3

```
def parse(s):
    f, size = parse_term(s)
    return f
```

C++の場合も C++11 の機能を用いると, 以下のように匿名関数を扱うことができる (コンパイルオプションに `-std=c++11` をつける). `double` を引数にとり `double` を返す関数の型は, `function<double(double)>` である. 匿名関数の作成は, `[] (引数) { 本体 }` という文法に従う. 冒頭の `[]` 内では, 本体の式で扱う外側の変数を列挙する. 詳しくは文法書を参照のこと. `auto` は型推論を利用した型の略記法である. 関数呼び出しの際は, 引数をカッコで直接渡す.

C++11

```
#include <functional>
#include <iostream>
using namespace std;
function<double(double)>
compose_product(function<double(double)> f,
    ↪ function<double(double)> g) {
    return [f, g](double x){ return f(x) * g(x); };
}
int main() {
```

```

auto f = [](double x){ return x + 3; };          9
auto g = [](double x){ return 2 * x; };          10
auto h = compose_product(f, g);                 11
cout << h(3) << endl;    // 6*6 = 36             12
cout << h(5) << endl;    // 8*10 = 80            13
}                                                  14

```

2.5 この章の提出課題

提出対象: 以下のいずれか

- 問題(★なし)を全て Python で解く (通常コース)
- ★つきの問題を解く (通常コースに苦勞がない場合)

提出先: UTOL

提出時期:

- 授業終了時: 学部生は授業終了時に出席代わりに、「参加報告」に出来具合を回答する
- 標準提出期限 (全員): 4 月 20 日 (月) までになるべく完成させて提出する. スタッフができ具合をコメントする. (場合によっては再提出になる)
 - 授業中に完成した小問については, 授業時間中にスタッフ (教員または TA) にチェックを受けて OK をもらう. スタッフ名をコメントに付記する (出席の確認にも用いる).
 - 一部途中で一旦提出する場合は, 作成途中と付記する.
- 学期終了時: 最終的な成績は最後に提出されたものを用いる.

提出条件:

- 指定した課題が, 指定した関数名で作成されていること
- 提出ファイルは, 2.A 節を参考にして, *jupyter から作成した* weekN-(学生証番号).ipynb とする
- 日本語 (または英語) でコメントが付記されていること: { 途中か完成品か, 授業中に既に OK を得ていればその教員名 } 学んだこと, (あれば) 他に参考にした資料や他の人の回答など, (途中で提出する場合は何がうまくゆかずに困っているなど) をコメントで盛り込む. (何も苦勞がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること – 授業中に教員の OK を得た場合は簡潔で良い)

2.A 毎週の課題の提出方式について

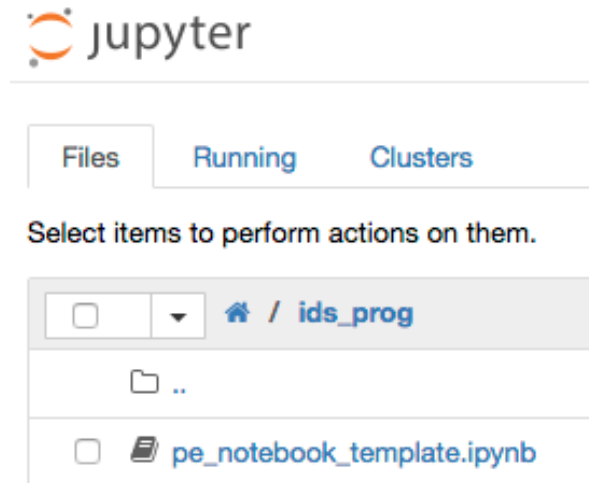
2.A.1 授業開始時: テンプレートの入手

2 週目以降は, UTOL の本授業のページにアクセスし, 「教材」に置いてある `pe_notebook_template.ipynb` をダウンロードする. 提出用テンプレートを適当な場所 (例えば, ホームディレクトリ直下に `ids_prog` などの名前のディレクトリを作成する) へ移動する.

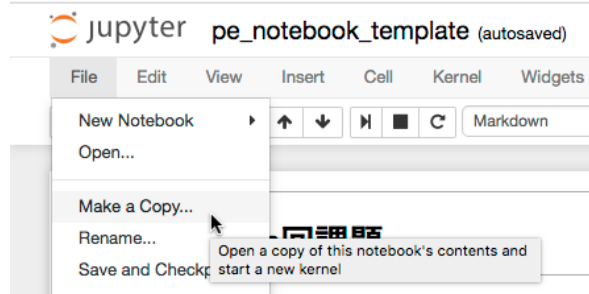
テンプレートから新しい notebook (作業用) を作る

授業中にプログラムを実行したり作業するための作業用 notebook を作成する.

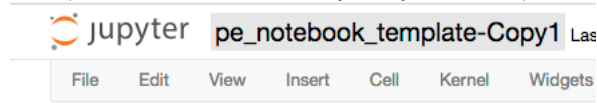
jupyter notebook 上でテンプレートファイルを開く. (jupyter notebook を起動した時に現れるブラウザ上で, 適切なフォルダを選択して該当ファイルをクリックする)



File > Make a Copy でテンプレートファイルをコピーしたファイルを作成する.



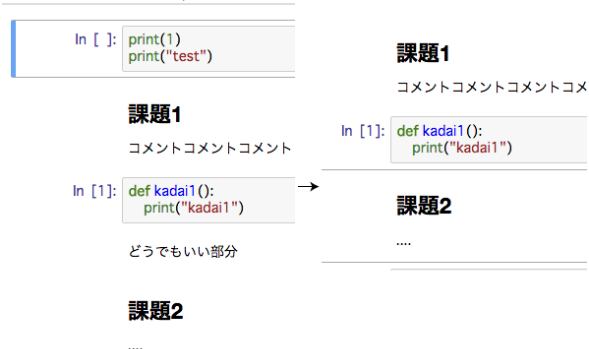
画面上の `pe_notebook_template-Copy1` をクリックし, 新しいファイル名 (`weekN-work`) へと変更する. (ただし `N` は章の番号= $\{2,3,\dots\}$ とする.)



以降はここで作成したノートブック `weekN-work` でプログラムを実行する.

提出時: 作業用 notebook から提出用ノートブックを作成する

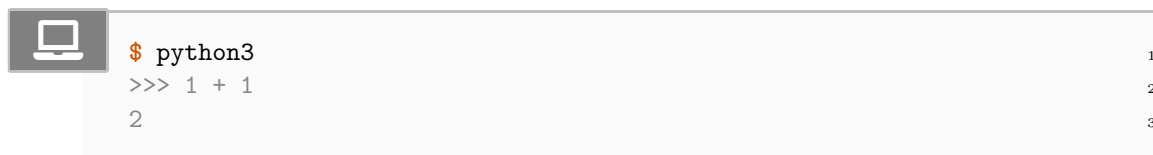
先程と同様に、File > Make a Copy で weekN-work をコピーし、weekN-(学生証番号) へと名前を変更する。“(学生証番号)”の部分自分の番号に置き換える。自分が提出したい部分以外を削除する「課題1」などの見出しなどは、ダブルクリックで編集できる。文法は Markdown 記法である (必要に応じて調査せよ)。



以上の操作で作成した jupyter notebook ファイル (weekN-(学生証番号).ipynb) を UTOL に提出せよ。

2.B ターミナルでの Python3 の利用

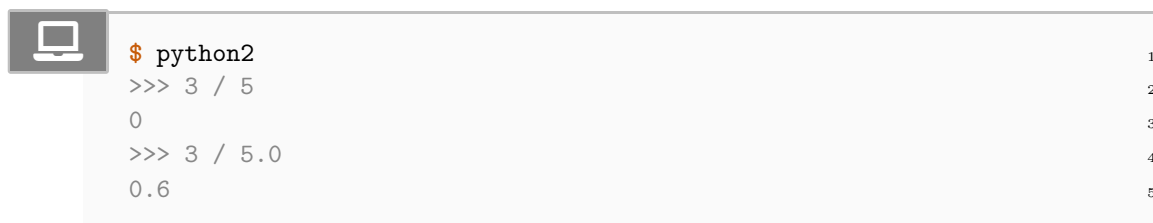
インタプリタは python3 として起動する。



本資料では、基本的に jupyter 上での実行を勧めるが、こちらのインタプリタが役に立つこともある。

2.B.1 整数除算

python2 と 3 で挙動が異なるので注意。





```
$ python3
>>> 3 / 5
0.6
>>> 3 // 5
0
```

1

2

3

4

5

2.C 参考資料

プログラミングのパラダイムに関しては, Concepts, Techniques, and Models of Computer Programming [2], 計算機プログラムの構造と解釈 [3], などを参照. 前者は学内あるいは東大図書館のEZproxy からPDF をダウンロード可能. 後者はウェブで公開されている. 今回の資料には1997年と1999年に増原先生によって行われた演習資料から拝借して金子が改変した部分を含む.

第3章 ユニットテスト，配列，ループ不変条件

Unit test, Array and Loop invariant

2026-04-22 Wed

概要

前週は「関数的に」(functional) プログラムを書くことをおすすめした。今週は、「命令的」(imperative) な文法を再び導入する。これによりプログラムを書く自由度が向上するが、一方で、プログラムが複雑になり理解が困難となるリスクもある。そこで、プログラムを分かりやすく保つための方法を合わせて身に付けよう。最初に紹介する「**ユニットテスト**」(unit test) は、プログラムを意図通り作成できたかを自動で検査する方法を提供する。続いて、配列と変数 (値の変更を許す) を題材にループ不変量 (loop invariants) という概念を学ぶ。

3.1 テストの自動化: ユニットテスト

先週は、関数を作成した後、jupyter 上でテストすることを行った。

(再掲)

例題 長方形の面積

二辺の長さがそれぞれ a, b である長方形の面積をもとめる関数 `rectarea(a, b)` を定義せよ

関数をデバッグしたり、あるいは完成したあとも高速化など変更を加える際など、同じテストを繰り返し**確実に**行いたい。そこで、テストの作業を自動化しよう。

この資料では Python の `pytest` というフレームワークを紹介する。¹⁾

まずは、`assert` の基本機能を紹介し、その後、`pytest` と jupyter 用の `ipytest` を紹介する。別途指示をするまで今週以降は、`ipytest` を用いること。

Python3

```
def rectarea(a,b):
    return a*b
# 名前を "test_" で始める
def test_area():
```

1) 2020 年までは `unittest` を使っていたが変更した

```

assert rectarea(7,5) == 35
assert rectarea(10,20) == 200

```

関数 `test_area` 内の 2 つの `assert` 文が本質である。それぞれの行を、具体的な値で計算結果を検証する、**テストケース**とよぶ。

テストを確認するもっとも単純な方法は、この関数を実行すれば良い。

Python3

```
test_area()
```

実行が正常に終了した場合は何も表示されない。失敗した場合は、以下のような `AssersionError` が表示される。

```

>>> test_area()
Traceback (most recent call last):
AssertionError:
... 1 def test_area():
... 2     assert rectarea(7,5) == 135

```

以上が、`assert` の基本的な機能である。テストケースに限らず、デバッグ等でも有用である。

`pytest` は、これらのテストケースを自動で発見して実行するなど、テスト実行や管理をより便利にするツールである。`ipytest` は、`pytest` を `jupyter` に対応させたもので、以下のように用いる

まず、`pip` module をインストールする。この作業は一度だけ行えば良い。

```
$ pip3 install ipytest
```

インストールの確認は、たとえば以下のように行う

```

$ pip3 list
ipytest      0.9.1
pytest      6.2.3

```

Microsoft Windows で `anaconda` を使っている場合、スタートメニューから “Anaconda Prompt” でシェルを起動し、`conda` コマンドを利用してインストールすることができる。²⁾

```
conda install -c conda-forge ipytest
```

続いて、`jupyter` を再起動し、冒頭で以下を実行する。この作業は各 notebook 毎に行う。

2) <https://anaconda.org/conda-forge/ipytest>

Python3

```
import ipytest
ipytest.autoconfig()
```

1
2

正常実行された場合は、特に表示はない。



もしインストールに失敗していると `ModuleNotFoundError` が発生する

```
>>> test_area()
Traceback (most recent call last):
ModuleNotFoundError:
...
```

1
2
3
4

これ以降、各 jupyter の cell では、テスト関数の自動実行が可能である。そのためには、セルの先頭で `%%ipytest -qq` を記述し (この文法の紹介は割愛する)、テスト関数を `test_` で始まる名前で定義する

Python3

```
%%ipytest -qq
def test_area():
    assert rectarea(7,5) == 135
    assert rectarea(10,20) == 200
```

1
2
3
4

正常に実行された関数の数だけ . が表示される。



.

[100%]

1

もしテストに失敗すると、以下のように出力される。



```
F
===== FAILURES =====
----- test_area -----

def test_area():
>     assert rectarea(7,5) == 135
E         assert 35 == 135
E         + where 35 = rectarea(7, 5)
(以下略)
```

1
2
3
4
5
6
7
8
9

テスト駆動開発という開発スタイルでは、失敗するテストを積極的に利用して開発を勧める (付

録 3.A を参照).

3.1.1 テストケースの作成

何をテストすれば良いかは関数の複雑さに依存する. 関数 `rectarea` を乗算だけで作成していれば, 1, 2 のケースでほぼ十分であろう. (ただしテストケースを欺く, 「仕様を満たしていないのに, テストケースでは検出できない実装」が存在することは認識しておく必要がある.)

Python3

```
def area_fake(a,b): # こんなコードは来ないことを前提としている 1
    if a == 5: 2
        return 35 3
    else: 4
        return 200 5
```

さっそくテストケースを自分で作ってみよう.

例題 issueCC-テストケース実行

以下の (先週扱った) 問題について `issueCC(3000000,0,10000000)` は, (借金返済額/年収が 30%以上であるから) `False` を返すことが期待される.

```
assert issueCC(3000000,0,10000000) == False 1
```

が実行されるテストを書いて, テストをパスすることを確認せよ. もし失敗した場合は, 関数 `issueCC` の定義を修正し, 再度テストを実行し, テストをパスすることを確認せよ.

問題 信用 (再掲)

(年収に対する) 借金返済額 `dt`, 返済遅延回数 `d1`, 年収の額 `ic` に対して, 以下の決定木に従って, クレジットカードを発行するかどうかを `true` か `false` で返す関数 `issueCC(dt,d1,ic)` を定義せよ.

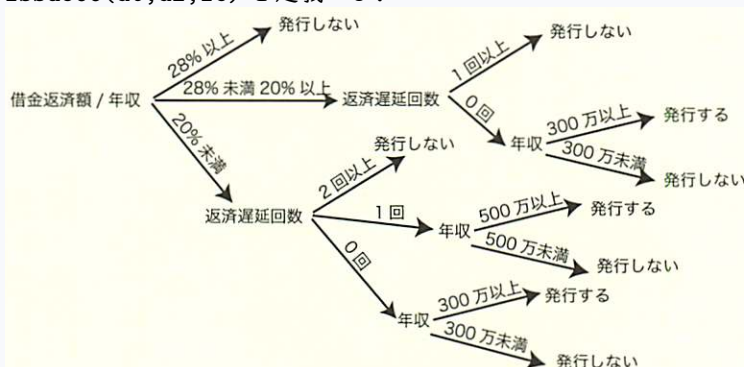


図 4.26 決定木

例題 Range-テストケース作成 (test_range)

以下の例題と回答例に対して、(1) もし正しく実装していれば成功するが、回答例のプログラムは失敗する、というテストケースを作成せよ。(2) 実際にテストを実行し、失敗することを確認せよ。(3) 関数 range の実装例を修正し、テストが成功することを確認せよ。

例題 Range (再掲)

3つの整数 a, b, c に対して、それらが $a < b < c$ の条件を満たすならば True を、満たさないならば False を返す関数 in_range(a,b,c) を作成せよ

回答例:

```
def in_range(a,b,c):  
    return a < b or b < c # 間違い
```

関数 range については、a,b,c の大小関係や等号の有無に関して、複数のケースを確認する方がより安心である。

問題 Circle in a Rectangle-テストケース作成 (test_in_rectangle)

先週作成した Circle in a Rectangle に対する適切なテストケースを作成せよ。

ヒント: この場合は、(i) (ii)
, (iii)-(vi) , の6通りをテストすること (一部は白文字で書いてある)。

今週以降の課題提出では、ほとんどの問題に対して、テストケースを添えることが条件となる。関数と同時にテストケースを作っておくこと。

3.2 配列と関数

3.2.1 関数内の変数 (前半は再掲)

関数の中でも、値に名前をつける目的で変数を使うことができる。

Python3

```
def 名前(仮引数 1, 仮引数 2...):  
    変数 1 = 式 1  
    変数 2 = 式 2  
    ...  
    return 式 # 仮引数に加えて変数 1, 2... を利用可能
```


このように、変数は式に別名を与えるもので、その値は関数内で一意に定まるという使い方を基本とする。



NameError

定義していない名前の変数を用いると (e.g., 名前を間違えると), `NameError` が発生する

```
>>> Pi = 3.14
>>> Pi
3.14
>>> Pi2
Traceback (most recent call last):
NameError: name 'Pi2' is not defined
...
```

例題 三角形の面積 (triangle_area)

3 辺の長さ a, b, c に対応する三角形の面積を計算する関数 `triangle_area(a, b, c)` を作成せよ。
面積は数式 $\sqrt{s(s-a)(s-b)(s-c)}$ で与えられる。ただし $s = (a + b + c)/2$ である。

回答例

Python3

```
def triangle_area(a, b, c):
    s = (a+b+c)/2.0
    return # ここに s, a, b, c を使った面積の式を書く
```

計算の様子を知るために `print` や `assert` を用いることもできる。先に書いた文から実行される。

Python3

```
def 名前 (仮引数 1, 仮引数 2...):
    変数 1 = 式 1
    print(変数 1)
    変数 2 = 式 2
    print(変数 2)
    assert(式)
    ...
    return 式 # 仮引数に加えて変数 1, 2... を利用可能
```

3.2.2 配列

配列は、0 個以上の式をカンマでつなげ “[” と “]” でくくって構築したものである (例: `[3, 4, 5]`)。空の配列も存在する (例: `[]`)。

変数 `a` が配列だとして, `a[i]` は `a` の `i` 番目の要素を与える演算である (`i` は整数とする). また `len(a)` は要素の数を与える.

```

>>> a = [3, 15+53, 10000]
>>> a
[3, 68, 10000]
>>> a[0]
3
>>> a[1]
68
>>> a[2]
10000
>>> a[-1] # 負の数は逆順に列を参照
10000
>>> len([10,20,30])
3
>>>

```



IndexError

存在しない配列の要素にアクセスすると, `IndexError` が生じる

```

>>> a[3] # aの要素数は3なので、各要素に対応する添字は0,1,2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range

```

例題 Range 配列版

3つの整数を要素として持つ配列 `a` に対して, それらが $a_0 < a_1 < a_2$ の条件を満たすならば `True` を, 満たさないならば `False` を返す関数 `in_range_array(a)` を作成せよ. なお, 今回のように対応関係が自明の場合は, `a[i]` を a_i と断りなく表記する.

Python3

```

def in_range_array(a):
    return # ここを埋める

```

```

>>> in_range_array([3,4,5])
True
>>> in_range_array([3,-4,5])
False

```

3.3 変数の値の書き換えとループ不変条件

既に定義済の変数の値を書き換える操作を**代入**という。

Python3

```
変数 = 式
```

1

既に紹介している**初期化**との違いは、既に定義した変数の値を書き換える点である。代入前の値にはアクセスできなくなることから、破壊的代入と呼ばれることもある。初期化と(破壊的)代入は、呼び方だけではなく実質的な区別がある。多くの言語において、定数は、初期化は可能でも代入は許されない(文法エラーとなる)。C++では初期化にはコンストラクタ (constructor) が、代入では operator= がそれぞれ呼ばれ、ユーザ定義クラスでは動作を変えることもできる。



```
>>> s = 3 # 初期化
```

1

```
>>> s
```

2

```
3 # (これまで勧めていた使い方)
```

3

```
>>> s = 5 # 代入: この行を境に s の値は変わる
```

4

```
>>> s
```

5

```
5 # さっきと違う
```

6

以下は、初心者をしてしばしば混乱させる、しかし文法に則った文である。

Python3

```
a = a + 1
```

1

代入では右辺を先に評価し、その値を左辺に代入する。つまり

Python3

```
a = 2
```

1

```
a = a + 1
```

2

であれば、 $a = 2+1$ すなわち、 $a = 3$ として実行される。なお、このようなケースはしばしばあるので、次のような略記法が用意されている。

Python3

```
a += 1 # a = a+1 と同じ
```

1

代入は、理解に右辺値や左辺値など様々な概念を必要とする、難しい概念である。

本演習では、機会を限定して代入を用いることを勧める。その一つが、次に紹介するループと共に用いる場合である。また、この資料では当面、代入の左辺には変数単体、または $a[i]$ のような配列の要素のみを扱うとする。

Python3

```
def 名前(仮引数 1, 仮引数 2...):           1
    変数 = 初期値                         2
    for ...                             3
        変数 = 式 # 変数を新しい値に書き換え 4
    return 変数                          5
```

例題 連続する数の和 (sum_to_n)

整数 1 から n までの合計を、愚直に足して求める関数 `sum_to_n(n)` を作成せよ。ただし $1 < n$ とする。(n=10000 などにて検算する)

回答例:

Python3

```
def sum_to_n(n):                         1
    total = 0                            2
    for i in range(1, n+1):              3
        total = total + i                4
    return total                          5
```

このようなプログラムがどのように導かれるか追ってみよう。

仮に固定値である 3 までの合計であれば、シンプルに記述可能である。

Python3

```
def sum3():                             1
    return 1+2+3                         2
```

補助変数 s_i (0 から i までの和) を用いて、等価なプログラムに書き直すこともできる。

Python3

```
def sum3():                             1
    s0 = 0 # 空集合の和                  2
    s1 = s0 + 1 # 1 の和                  3
    s2 = s1 + 2 # 1 と 2 の和              4
    s3 = s2 + 3 # 1 と 2 の 3 の和          5
    return s3                             6
```

代入を用いると、補助変数 s_i を一つの変数 `total` で表現することもできる。

Python3

```
def sum3():                             1
    total = 0 # total は s0 相当           2
    total = total + 1 # 左辺の total は s1, 右辺の total は s0 相当 3
    total = total + 2                     4
    total = total + 3                     5
```

```
return total # total は s3 相当
```

6

これを for で書き換えたものが、冒頭の回答例である。

Python3

```
def sum_to_n(n):
    total = 0
    for i in range(1, n+1):
        # (a) この時点で total の値は 0 から i-1 までの和 =i(i-1)/2
        total = total + i # なお, total += i と書いても同じ
        # (b) この時点で total の値は 0 から i までの和 =i(i+1)/2
    return total
```

コメント (a)(b) に記述した total の値に関する条件が、ループのどの時点でも成り立つことを確認してほしい。このような条件を、**ループ不変条件**という。これらの total の値に関する条件を用いて、関数が n までの和を計算することを証明することができる。

次の 3.4 節の課題では、提出する全てのソースコードに**ループ不変条件を明確にしてコメント**で記述すること。なお、説明のために上記の例では (a), (b) 二つの条件を書いたが、以下の演習の回答では、各ループ開始直後 (a) または終了直前 (b) のどちらかを記述すれば良い。



Notice: Loop 不変条件の表現

動作や変更を表す言葉は避ける。**静止した状態を表現**すること。

例: ジャガイモの皮を剥いて... v.s. カレーライス

例: 1 を足して 2 を足して...i まで足したもの v.s. 1 から i までの和/合計値。似ているが後者は計算手段は指定していない。

3.4 配列とループ

例題 要素の和 (mysum)

配列 a に格納されているデータの合計を求める関数 mysum(a) を作成せよ。

例題 最大値 (mymax)

配列 a に格納されているデータの最大値を求める関数 mymax(a) を作成せよ。

なお実際には、Python では、配列の最大値を返す max という関数が用意されている。(が、この例題ではそのことは忘れるとする)

例題 最大値の添字 (max_index)

配列 `a` に格納されているデータの最大値の添字を求める関数 `max_index(a)` を作成せよ。

例題 二番目に大きな値 (second_max)

配列 `a` に格納されているデータのなかで 2 番目に大きな値を求める関数 `second_max(a)` を作成せよ。

問題 最小と最大を除いた合計 (score)

審判の点数が配列 `a` で与えられる。それをオリンピック方式 (最大と最小の点数をつけた審判をそれぞれ一人ずつ除いた残りを使う) で合計する関数 `score(a)` を作成せよ。審判は 3 人以上と仮定して良い。

ループ不変条件の練習なので、`max`, `min`, `sort` などを使わずに書くこと。

問題 各桁の和 (digit_sum)

与えられた正の整数 `n` に対して、十進表現での各桁の和を求める関数 `digit_sum(n)` を作成せよ。

整数 `n` の桁数は `math.floor(math.log10(n))+1` で得ることができる (`import math` が必要)。また 10 の `n` 乗は `10**n` で得られる。

3.5 配列や文字列の変更

合計を表す変数に数を足す変更が役に立ったように、配列に要素を追加したい場合もある。Python では `append` というメソッドが用意されている。



```
>>> a = [1, 2, 3] 1
>>> a.append(4) 2
>>> a 3
[1, 2, 3, 4] # a の指す配列の内容は変更されている 4
>>> a = a + [5] 5
>>> a 6
[1, 2, 3, 4, 5] # 配列同士の連結 (上記は+=と略記可) 7
```

Python で文字列に対しても `+` 演算が用意されていることは既に紹介したが、`+=` という演算も用意されている。数に対する `+=` 同様に、これらも破壊的的操作である。

なお、C++ の配列は、C の配列に由来するもので、Python と比べて制限が大きい。代わりに標準ライブラリの `std::vector<int>` を用いると良い。

C++11

```

#include <vector>                                1
#include <iostream>                              2
using namespace std;                            3
int main() {                                    4
    vector<int> a = { 1,2,3 };                  5
    for (size_t i=0; i<a.size(); ++i)          6
        cout << a[i] << endl; // 1,2,3 が一行毎に表示される 7
                                            8
    cout << endl; // 下記と区別するための空行 9
    a.push_back(4); // a に 4 を追加            10
                                            11
    for (auto n: a) // a の要素を一巡する別の記法 12
        cout << n << endl; // 1,2,3,4 が一行毎に表示される 13
}                                              14

```

例題 1 から n までの配列 (iota)

1 から与えられた正の整数 n までを順に要素として持つ配列を返す関数 iota を作成せよ。



```

>>> iota(5)                                1
[1, 2, 3, 4, 5]                             2
>>> iota(7)                                3
[1, 2, 3, 4, 5, 6, 7]                       4

```

ヒント: a = [] から始めて, 順に a に要素を追加する。

回答例 (一部は白文字で書いてある):

Python3

```

def iota(n):                                1
    a = []                                  2
    for i in range(1,n+1):                  3
        a.append(i)                         4
        # ループ不変条件は..                5
    return a                                6

```

例題 倍数フィルタ (filter_multiple37)

整数を要素とする配列 a に対して, 3 または 7 で割り切れる全ての数を含む配列を関数 filter_multiple37(a) を作成せよ。



```

>>> filter_multiple37(iota(15))            1
[3, 6, 7, 9, 12, 14, 15]                  2

```

ヒント: if 文を用いて条件が成り立った時のみ, 答えの配列に要素を加える.

問題 約数列 (factor)

与えられた整数 $n (> 0)$ の約数を全て求め, 配列として返す関数 `factor(n)` を作成せよ. 効率は n 回の除算を行うもので良い.

問題 たきぬつきね (decrypt2)

二つの文字列 a, b に対して, 両者を先頭から2文字ずつ交互に混ぜた文字列を s とする. 例えば, $a="ABCD", b="EFGH"$ の時, $s="ABEFCDDGH"$ である. この s から逆に a, b を求めたい. 与えられた s に対応する, 2つの文字列を要素として持つ配列 $[a, b]$ を返す関数 `decrypt2(s)` を作成せよ. なお, 文字列 a, b の文字数は偶数で同じとする. すなわち文字列 s の長さは4の倍数と仮定して良い.



```
>>> decrypt2("takinutukine")
['tanuki', 'kitune']
>>> decrypt2('mutirahasayamebunorutukayumimoyoma' +
↳ 'modakihikanuzumatakitunotahaganiha') # 百人一首
```

ヒント:

Python3

```
def decrypt2(x):
    a = ""
    b = ""
    for i in range(len(x)):
        # i が何文字目かを考えて x[i] を a または b に加える
        # a, b のループ不変条件は...
    return [a, b]
```

C++11

```
#include <vector>
#include <string>
#include <iostream>
using namespace std;
vector<string> decrypt2(string x) {
    string a="", b="";
    for (size_t i=0; i<x.size(); ++i) {
        // i が何文字目かを考えて x[i] を a または b に加える
    }
    return { a, b };
}
int main() {
```



```

auto v = decrypt2("takinutukine");           13
cout << v[0] << endl;                         14
cout << v[1] << endl;                         15
}                                              16

```

問題 たぬき暗号 (erase_ta)

与えられた文字列 x から "ta" を取り除いた文字列に相当する文字列を作成して返す関数 `erase_ta(x)` を作成せよ。文字列 x は 2 文字以上とする。しかし, "ta" は, 偶数文字目奇数文字目いずれにも来るかもしれない。

Python で文字列 x の i 文字目から 2 文字を取り出すには, `x[i:i+2]` と記述する。

実は Python には `replace` という置換を行うメソッドが用意されているので, 実用にはそちらを使うべきである。この演習では, テストケースの作成に活用すると良い。



```

>>> "tatotauktayotauttao".replace("ta","")    1
'toukyouto'                                    2

```

testcase 作成の指針: 比較的複雑なので, 2 文字や 3 文字のケースをよくテストする。if 文の全ての分岐を通ることが望ましい。(例: t で終わるもの終わらないもの..)

Python3

```

assert erase_ta('fight') == 'fight'           1
assert erase_ta('aa') == 'aa'                 2
assert erase_ta('at') == 'at'                 3
assert erase_ta('bt') == 'bt'                 4
assert erase_ta('tb') == 'tb'                 5
assert erase_ta('ta') == ''                   6
assert erase_ta('xyz') == 'xyz'               7
assert erase_ta('ttaa') == 'ta'               8

```



ヒント

「"t"ぬき暗号」であれば下記のように簡単に作成できる。"ta"を除く場合も, if 文の条件式を変更するだけで対応できる (考えてみよう)。

```

def erase_t(x):
    s = ''
    for i in range(len(x)):
        if ...: # x[i] が 't' でなければ s に追加
            s += x[i]
        # ループ不変条件: s は、x の先頭から i 文字目までの、t 以外の
        ↪ 文字列に相当
    return s

```

3.6 発展: 関数のパラメータ化

関数を用いる際に、計算ごとに異なる値はパラメータとして呼び出し時に与える。

多くの言語では、関数自体をパラメータとして受け渡すことができる。大枠の計算手順の中で、小さな計算だけが違うような場合には、その部分を表す関数をパラメータとして渡すと良い。

Python の場合、関数を渡す場合に関数名を括弧を付けずに書き、受け取った関数を使う場合には通常の関数と同様に実行する。(後者は、2.4 節でも紹介した)

Python3

```

def f_sample(x):
    return 3.0*x**2 + 2.0*x + 1.0

def value_at_zero_and_ten(f): # f に応じて何かをする関数
    y0 = f(0) # f(0)
    y10 = f(10) # f(10)
    return [y0, y10]

```



```

>>> value_at_zero_and_ten(f_sample)
[1.0, 321.0]

```

C++11 の場合、double を引数に取り double を返す関数は、`function<double(double)>` という型で表す。

C++11

```

#include <functional>
#include <vector>
#include <iostream>
using namespace std;
double f_sample(double x) {
    return 3.0*x*x + 2.0*x + 1.0;
}

```

```

vector<double> value_at_zero_and_ten(function<double(double)> f) {
    double y0 = f(0);
    double y10 = f(10);
    return { y0, y10 };
}

int main() {
    vector<double> v = value_at_zero_and_ten(f_sample);
    cout << v[0] << ' ' << v[1] << endl;
}

```

問題 数値積分と関数パラメータ ★ (integral)

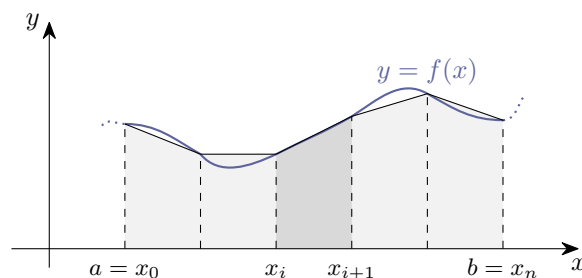
数値 x を引数に取り、対応する数値を返す関数を f とする。区間 $[a, b]$ ($a \leq b$) の定積分を、近似して求める関数を作成せよ。近似手法として、台形公式とシンプソン公式の2種類を使えるようにせよ。

(この課題ではテストケースの代わりに) 様々な関数 f と分割数 n に応じて、答えの精度がどのように変化するかを観察して、仮説を立てて議論せよ。実験結果と議論を、コメントにコピーペーストするなどしてソースコードと合わせて提出せよ。

精度については、解析的に原始関数 ($F(x)$) を求められる関数 (たとえば $f(x) = x^2$ と $F(x) = 1/3x^3$) を対象に用いて、 $F(b) - F(a)$ を真の値として差を議論する。

台形公式 近似のもっとも簡便な方法は、区間を細かく分けて長方形の面積の和で近似するものである。

小分けにした短冊部分について、長方形を台形に変更すると、近似の精度を高めることができる。すなわち、分割数を n 、分割した各区間の幅を $h = (b - a)/n$ 、 $x_i = a + ih$ として、 $\sum_{0 \leq i < n} h(f(x_i) + f(x_{i+1}))/2$ を答えとする。($\sum$ を整理すると効率が良くなるが、定義通りに計算しても良い)



シンプソン公式 (Simpson's rule) 直線の代わりに二次式で近似すると、さらに精度が改善する。

一般に、3点 $(a, v_a), (b, v_b), (c, v_c)$ を通る2次の多項式は次のように一意に定まる:

$$L(x) = \frac{(x-b)(x-c)}{(a-b)(a-c)} v_a + \frac{(x-a)(x-c)}{(b-a)(b-c)} v_b + \frac{(x-a)(x-b)}{(c-a)(c-b)} v_c.$$

$a = x_i$, $b = x_{i+1}$ として、区間内の三点 $(a, f(a)), (\frac{a+b}{2}, f(\frac{a+b}{2})), (b, f(b))$ を通る補間多項式を整理

すると、その区間の面積は以下のように近似できる:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

もし区間で $f(x)$ が二次以下の多項式であれば、計算誤差を無視すれば正確な値を得ることが出来る。

3.7 この章の提出課題

以下のいずれかで合格:

- 問題 (タイトル部分に★なし) を全て Python で解く (通常コース)
- ★つきの問題を 1 題解く (言語指定なし)
+ 通常の問題を 1 問以上解いてループ不変条件とユニットテストを添える

提出条件:

- 各関数にテストケースが付加されていること **New!**
- ループ不変条件が明確にコメントで記述されていること **New!** (本日 3.4 節では必須, 他も適宜推奨)
- 冗長なコードがないか確認し, 標準的な書き方に整えられていること (*) **New!**
- 指定した課題が, 指定した関数名で作成されていること
- 提出ファイルは, 2.A 節を参考にして, *jupyter から作成した* weekN-(学生証番号).ipynb とする
- 日本語 (または英語) でコメントが付記されていること: { 途中か完成品か, 授業中に既に OK を得ていればその教員名 } 学んだこと, (あれば) 他に参考にした資料や他の人の回答など, (途中で提出する場合は何がうまくゆかずに困っているなど) をコメントで盛り込む. (何も苦労がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること — 授業中にスタッフに OK を得た場合は簡潔で良い)

(*) の項目の一つを挙げる. コード中に

Python3

```
if a:
    return True
else:
    return False
```

という式があったら, if ブロックの中が評価されて値をとることを考えると, これは

Python3

```
return a
```

と書き換えても同じである. この書き換えは, 式 a の部分が, 複雑な式であっても成り立つ. したがって, この演習では後者の記法を推奨する.

3.A テスト駆動開発★

機能を拡張する場合は、(1) 先にテストケースを作る (2) すぐにテストを動かしてそのテストが働いている (i.e., 失敗する) ことを確認する (3) 関数を変更する (4) 先ほど失敗していたテストが今度は成功する (他のテストの結果は変化がない) ことを確認する、という手順を繰り返すと良いとされる。各ステップを省略すると、テスト自体が間違っていたり、変更により齟齬が生まれたりといったケースで、発見が遅れうるためである。

3.A.1 閏年の判定

ここでは閏年を判定する関数を開発するという架空のストーリーを通して、関数の実装とテストデータを並行して充実させることを経験する。このようにテストを作成しながら開発すると、テストを作成しない場合よりタイプする文字数は2倍以上になることが多いが、一般には十分に元が取れる。一般にバグが混入すると解決には開発時間の10倍以上の時間を要することが珍しくないが、ユニットテストを併用した開発は、バグの混入する機会を減少させるためである。

失敗するテストの作成

`is_leap_year(y)` という関数の雛形を作成する。この関数の目的は、引数 `y` で与えられた年が閏年かどうかを真偽値で返すことである。

Python3

```
def is_leap_year(y):
    return True # とりあえずなんでも閏年
```

この関数をテストするクラスを作製する。

Python3

```
%%run_pytest[clean] -qq
def test_leap(): # 名前を"test_"で始める
    assert is_leap_year(2015) # 2015 年は平年
```

動かすと以下のように、失敗 (F や failures に注目) が報告される。



```
F
===== FAILURES =====
----- test_leap -----
def test_leap(): # 名前を"test_"で始める
>     assert is_leap_year(2015) == False # 2015 年は平年
E         assert True == False
E         + where True = is_leap_year(2015)
```

中頃の line 6, in `test_leap` から、失敗したテストを含む関数と、行数を読み取る。(もっとこの

場合はテストが一つしか無いので自明である). このように、まず間違いようがない単純な段階でテストを意図通りに失敗させると良い。テストが実は動いていないとか、勘違いが二重に作用してテストの不備が見過ごされるなどを防ぐため。

実装とテストの拡充

では、関数 `is_leap_year(y)` を編集して、もう少しまともに閏年が4年に一度であることを判定するようにする。

Python3

```
def is_leap_year(y):
    if y % 4 == 0:
        return True
    else:
        return False
```

これに伴い、テストも拡充させる。関数の中で4で割った剰余を用いているので、連続する4年間をテストすること全ての剰余のケースを試す。

Python3

```
def test_leap():
    assert False == is_leap_year(2015)
    assert True == is_leap_year(2016)
    assert False == is_leap_year(2017)
    assert False == is_leap_year(2018)
```

これを実行して、テストが全て成功することを確認する。

実装の変更

ここで関数 `is_leap_year(y)` の実装が、冗長であることに気づいたとする。if 文を使う必要はないので、以下のように書き換えてみよう。

Python3

```
def is_leap_year(y):
    return y % 4 == 0
```

書き換えたら必ずテストを実施し、書き換えによって失敗することがないことを確認する。

バグの原因の一つは、プログラマが等価だと思って書き換えた変更が、実際には勘違いであるケースである。このような書き換えの前後には、テストを行って関数の挙動に変化がないことを確認すると良い。

仕様の変更

これまでは(ストーリー上)忘れていたが、実は「100で割り切れる年は閏年ではない」と上司の指示で(あるいはユーザからの指摘)で分かったとする。

このような場合には、関数を変更する*前に*テストを変更する。これまでのメソッド `test` に追加することもできるが、ここでは `test_leap100` という関数を新たに追加し、そこに新たに `assert` 文を用いてテストを記述する。このように意味のまとまり毎に関数を分けておくと、見通しが良い。ただし、メソッド名は `test` で始める必要がある(それらのもののみ実行される)。

Python3

```
def test_leap():
    ... # すでにあるものはそのまま

def test_leap100(): # 新たに作成
    assert True == is_leap_year(1896)
    assert False == is_leap_year(1900) # メインのテスト
    assert True == is_leap_year(1904) # 念のため前後の年も
```

実際にテストが 1900 年で失敗することを、実行して確認する。これにより新たに作成した `test_leap100` が正常に働いていることが分かる。



```
.F [100%]
===== FAILURES =====
----- test_leap100 -----
def test_leap100(): # 新たに作成
    assert True == is_leap_year(1896)
>    assert False == is_leap_year(1900) # メインのテスト
E    assert False == True
E    + where True = is_leap_year(1900)
```

この準備を経て、いよいよ関数を修正する。

Python3

```
def is_leap_year(y):
    return not (y % 100 == 0) and y % 4 == 0
```

実際に、失敗していたテストが成功するようになったことを確認する。

例題 より正確な閏年

現実世界では 400 で割り切れる年は 100 で割り切れる年であっても閏年である。適切なテストを追加したうえで、そのような条件を含めた `is_leap_year(y)` を作成せよ。

3.B C++でのテスト

C,C++では `assert` という標準ライブラリを用いて、同様のテストを実現可能である。演習の範囲では、`assert` で十分であろう。`assert` の中には、`true` になるはずの式を記述する。

C++

```
#include ... // 自分のファイルを読み込み      1
#include <cassert> // 追加                        2
int main() {                                    3
    assert(! is_leap_year(2014));                4
    assert(is_leap_year(2015)); // 失敗するテスト 5
}                                                  6
```

卒業研究やプロジェクト実習などで中規模以上のプログラムを開発する場合は、あるいは実用的には、`boost::test` や `gtest` などの外部ライブラリを用いると、圧倒的に便利である。詳しくは各ライブラリのドキュメントを参照のこと。

第 4 章 参照と二次元配列

References and two-dimensional arrays

2026-04-29 Wed

概要

先週に引き続き、関数内でデータを加工して返すタイプの関数の作り方を演習する。

```
def 名前 (仮引数 1, 仮引数 2...):
    変数 = 初期値 # 配列や二次元の配列を扱う。
    for ...
        変数を加工
    return 変数
```

1
2
3
4
5

なお、先週の補足は 4.B に。

4.1 配列の要素の書き換え

先週導入した、代入文を拡張し配列の一部分のみを変更する文法を導入する。なお、Python では文字列をこのように変更することはできない。

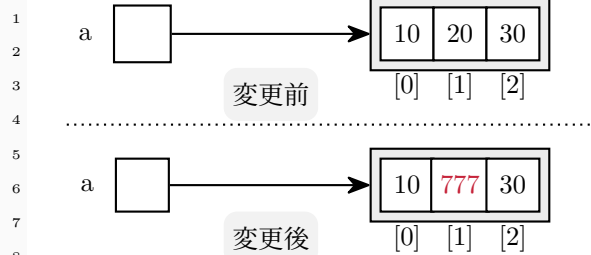
Python3

{配列を表す式}[{整数を表す式}] = 式

1



```
>>> a = [10,20,30]
>>> a
[10, 20, 30]
.....
>>> a[1] = 777
777
>>> a
[10, 777, 30]
```




```

lucky_numbers = [False for _ in range(30)]          4
lucky_numbers[7] = True                             5
lucky_numbers[14] = True                            6
lucky_numbers[21] = True                            7
lucky_numbers[28] = True                            8
lucky_numbers[16] = True                            9
lucky_numbers[25] = True                           10
assert lucky_numbers == lucky_array(30)            11

```

例題 選抜

真偽値を要素とする配列 a が与えられる。 $0 \leq i < \text{len}(a)$ である整数 i について、 $a[i]$ が True であるような i を昇順に並べた配列を返す関数 `select_index(a)` を作成せよ。



```

>>> select_index([False, False, True, False, True])  1
[2, 4]                                              2

```

問題 エラトステネスの篩 (`prime_array`)

正の整数 n 未満の素数を配列として返す関数 `prime_array(n)` を作成せよ。



```

>>> prime_array(20)  1
[2, 3, 5, 7, 11, 13, 17, 19]  2

```

回答方針: まずは整数 $1 < i < n$ に対して、 i が素数なら $a[i]$ が True, i が合成数なら $a[i]$ が False となるような配列を作成する

- 初めに全ての要素を True とする
- 0,1 を False とする
- 2を残し, 4以上の2の倍数を全て False とする
- 3を残し, 6以上の3の倍数を全て False とする
- 4は既に false になっている (=合成数). また4の倍数も全て既に False になっているので何もしない
- (一般化) 「ある整数 i が True の場合, i を残し, i より大きく n 未満の i の倍数を全て False とする」という手順を $i=2$ から, $i=\text{math.floor}(\text{math.sqrt}(n))$ まで繰り返す.

4.2 二次元配列

Python では配列の要素に配列を配置することで、**二次元配列**に相当するデータを表現することができる。

```

>>> a = [[8,3,4], [1,5,9], [6,7,2]]
[[8, 3, 4], [1, 5, 9], [6, 7, 2]]
>>> a[0]
[8, 3, 4]
>>> a[1]
[1, 5, 9]
>>> a[2]
[6, 7, 2]
>>> a[1][2]
9

```

1
2
3
4
5
6
7
8
9
10

なお、縦横の概念は模式的なものであり、逆に解釈することもできる。

例題 横の和

整数を要素とする二次元配列 a と行番号 r が与えられる。 $0 \leq c < \text{len}(a[r])$ の範囲の列番号 c について、各要素 $a[r][c]$ の和を求める関数 $\text{rsum}(a, r)$ を定義せよ。ただし行番号 r は 0 以上、 $\text{len}(a)$ 未満の整数とする。

```

>>> a = [[1,2,3], [10,20,30]]
[[1, 2, 3], [10, 20, 30]]
>>> rsum(a,0)
6
>>> rsum(a,1)
60

```

1
2
3
4
5
6

例題 縦の和

整数を要素とする二次元配列 a と列番号 c が与えられる。 $0 \leq r < \text{len}(a)$ の範囲で、各要素 $a[r][c]$ の和を求める関数 $\text{vsum}(a, c)$ を定義せよ。ただし a の列数は全ての行で等しく、列番号 c は 0 以上かつ列数未満の整数とする。

```

>>> vsum(a,0)
11
>>> vsum(a,1)
22
>>> vsum(a,2)
33

```

1
2
3
4
5
6

二次元配列 (height 行, width 列, 初期値 initial) を作成するには、さしあたり以下の関数 `make2d` を用いる。

Python3

```
def make2d(height,width):
    a = [[0 for _ in range(width)] for _ in range(height)]
    return a
```

問題 問 (発展):

make2d に代えて, `[[0]*width]*height` を用いたとする. どのような差異 (不具合) があるかを考察せよ. (考え方と回答は 4.A 節で)

C++で2次元配列を扱う場合は, たとえば `vector` の入れ子を用いる:

C++11

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
    vector<vector<int>> a = {{8,3,4}, {1,5,9}, {6,7,2}};
    for (auto& row: a) {
        for (auto e: row)
            cout << e << ' ';
        cout << endl;
    }
}
/* 実行結果
8 3 4
1 5 9
6 7 2
*/
```

例題 2 倍

縦横の行数列数が揃った整数を要素とする二次元配列 `a` が与えられるので, 同じ行数列数を持ち, 各要素が2倍になっている行列を返す関数 `matrix_double(a)` を作成せよ.



```
>>> matrix_double(a)
[[2, 4, 6], [20, 40, 60]]
```

回答例

Python3

```
def matrix_double(a):
    b = make2d(len(a), len(a[0])) # 答えとなる配列
    for r # 適切な範囲の r 全てについて
        for c # 適切な範囲の c 全てについて
```

```

        b[r][c] = a[r][c]*2
    return b

```

問題 表計算 (extend_sum)

二次元配列が与えられるので、縦方向と横方向に両方にその和を加えた二次元配列を (新しく作成して) 返す関数 `extend_sum(a)` を作成せよ。(元の配列は変更しないとする)

入力の二次元配列は、長方形型 (どの列の行数もどの行の列数も同じ) とする。

図解 http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ITP1_7_C

(リンク先では表示するよう指示があるが、この演習では配列を返す関数を作成すること)



```

>>> a
[[1, 2, 3], [10, 20, 30]]
>>> extend_sum(a)
[[1, 2, 3, 6], [10, 20, 30, 60], [11, 22, 33, 66]]

```

4.3 成長スライドパズル



Notice:

アルゴリズム入門未修など「有限ライフゲーム」を作ったことがなければ、この節を飛ばして、次節の有限ライフゲーム に取り組むことをお勧めする。

2048 は 4x4 のグリッドでタイルを動かす、1 人ゲームである。ここでは 3x3 の縮小版を考える。

3x3 の 9 カ所の各セルは、空白であるか、正の整数のタイルが配置されている。いまプレイヤーが「左」を選んだとすると、各行は次のように変化する

1. タイルは左に詰められる。つまり、空白はすべてのタイルより右のみに存在する。
2. 左右に並ぶ同じ数のタイル 2 つは融合し、その合計の数をもつタイル 1 つに変化する。融合は左から順に処理され、1 度融合したタイルが別の融合に参加することはない。融合後の空白も、規則 1 に準じて詰められる。(実行例も参照)

4.3.1 1 行の移動

例題 slide3

ある行の実行結果をシミュレートする関数 `slide3(row)` を作成せよ。row は整数を要素とする長さ 3 の配列で、0 は空白を、他の数はタイルを表すいろいろな実装方法があるが、まず日本語で条件を整理しよう



```
>>> slide3([0, 2, 0])      1
[2, 0, 0]                  2
>>> slide3([2, 0, 0])      3
[2, 0, 0]                  4
>>> assert slide3([2, 0, 2]) 5
[4, 0, 0]                  6
```



```
>>> assert slide3([0, 0, 2]) == [2, 0, 0]      1
>>> assert slide3([2, 0, 2]) == [4, 0, 0]      2
>>> assert slide3([4, 4, 2]) == [8, 2, 0]      3
>>> assert slide3([4, 2, 4]) == [4, 2, 4]      4
>>> assert slide3([0, 2, 4]) == [2, 4, 0]      5
>>> assert slide3([0, 4, 4]) == [8, 0, 0]      6
>>> assert slide3([4, 4, 4]) == [8, 4, 0]      7
>>> assert slide3([2, 2, 4]) == [4, 4, 0]      8 # [8, 0, 0] は失敗
>>> assert slide3([2, 4, 4]) == [2, 8, 0]      9 # 0 なしでも変化は起こり得る
```

問題 1 行のスライド (slide_n)

非負整数の配列 `seq` が与えられる。スライドさせた配列を返す関数 `slide_n(seq)` を作成せよ。以下の 3 つの機能を別々に作って、それぞれ機能をテストした後で、組み合わせると作りやすい

- `seq` から 0 を除いたリスト `non_zeros` を作成する (`filter3` や `erase_t` と同じ構造)
- 隣合う要素が等しければ合体させる `merge_pairs` を作成する (`is_increasing`, `erase_ta` と近い構造)
- 要素がすべて 0 で、指定長さ `n` の配列を作る関数 `make_zeros(n)`

テストケースは、さきほどの `slide3` のものをまず試そう。



```
>>> assert slide_n([2, 2, 2, 2]) == [4, 4, 0, 0]      1
>>> assert slide_n([4, 2, 0, 2]) == [4, 4, 0, 0]      2
>>> assert slide_n([2, 2, 0, 4]) == [4, 4, 0, 0]      3
>>> assert slide_n([2, 0, 2, 2]) == [4, 2, 0, 0]      4
>>> assert slide_n([2, 4, 8, 16]) == [2, 4, 8, 16]    5
```

```
>>> assert slide_n([0, 4, 0, 2, 0, 2, 0]) == [4, 4, 0, 0, 0, 0, 0] 6
```

4.3.2 正方形の盤面

これ以降、正方形の 4x4 の盤面を考える。たとえば、以下のような 2 次元配列となる。

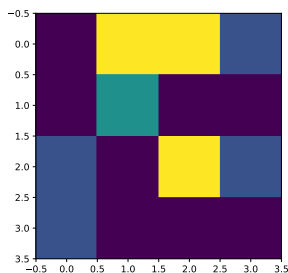
```
>>> board = [[2, 2, 0, 0], [0, 0, 4, 8], [0, 8, 0, 8], [0, 2, 0, 2]] 1
```

これを心の目で、[2, 2, 0, 0] が一番上の行のように、ゲーム盤と対応指せて読む。

読みにくい場合は、以下のように画像として可視化しても良い。色と数の対応をとることは難しいが、どの増すが同じ色=数かは認識できる。

Python3

```
import matplotlib.pyplot as plt 1
plt.imshow([[0, 8, 8, 2], [0, 4, 0, 0], [2, 0, 8, 2], [2, 0, 0, 0]]) 2
plt.savefig('board.pdf') 3
```



例題 左寄せ (slide_left)

全部の行を左に寄せた新しい盤面を返す関数 `slide_left(board)` を実装しよう。 `slide_n` を使えば簡単に作成できる

```
def slide_left(board): 1
    N = len(board) 2
    assert len(board[0]) == N # 正方形であることを確認 3
    new_board = [] 4
    for i in range(N): 5
        new_board.append(...) 6
    return new_board 7
```




```
>>> slide_left(board)
[[4, 0, 0, 0], [4, 8, 0, 0], [16, 0, 0, 0], [4, 0, 0, 0]]
```

例題 回転 rotate90

盤面を反時計回りに 90 度回転した 2 次元配列を返す関数 `rotate90(board)` を作成しよう
 ヒント 以下のひな形は新しい 2 次元配列を `new_board` を作成後に、各要素 (r, c) に値を設定する。 $r, c, N - r - 1, N - c - 1$, などを検討すると良い

Python3

```
def rotate90(board):
    N = len(board)
    new_board = ita.array.make2d(N, N)
    for r in range(N):
        for c in range(N):
            new_board[r][c] = board[...][...]
    return new_board
```



```
>>> rotate90(board)
[[0, 8, 8, 2], [0, 4, 0, 0], [2, 0, 8, 2], [2, 0, 0, 0]]
```

例題 回転 rotate90n

$(90 \cdot n)$ 度回転を考える。90 度の回転を 2 回, 3 回行えば、それぞれ、180 度, 270 度回転となる。 n 回 (n は 1,2,3 のどれか) 回転させる関数 `textttrotate(board, n)` を `rotate90` を利用して、作成せよ



```
>>> rotate(board, 2)
[[2, 0, 2, 0], [8, 0, 8, 0], [8, 4, 0, 0], [0, 0, 2, 2]]
>>> rotate(board, 3)
[[0, 0, 0, 2], [2, 8, 0, 2], [0, 0, 4, 0], [2, 8, 8, 0]]
```

例題 右寄せ slide_right

`slide_left` の反対に右に寄せた盤面を返す関数 `slide_right(board)` を作成せよ。
 ヒント 180 度回転した盤面を `slide_left` を使って左寄せし、結果をさらに 180 度回転させると、元の右寄せ相当



```
>>> slide_right(board)
[[0, 0, 0, 4], [0, 0, 4, 8], [0, 0, 0, 16], [0, 0, 0, 4]]
```

例題 上寄せ slide_up

同様に上に寄せた盤面を返す `slide_up(board)` を作成せよ
 ヒント: 90 度回転, 左寄せ, -90 度 (=270 度) 回転



```
>>> slide_up(board) 1
[[2, 2, 4, 16], [0, 8, 0, 2], [0, 2, 0, 0], [0, 0, 0, 0]] 2
```

例題 下寄せ slide_down

同様に下に寄せた盤面を返す `slide_down(board)` を作成せよ



```
>>> slide_down(board) 1
[[0, 0, 0, 0], [0, 2, 0, 0], [0, 8, 0, 16], [2, 2, 4, 2]] 2
```

問題 文字列方向の移動 slide_dir

変数 `dir` に `udlr` のどれか一つの文字が与えられ, それぞれ上下左右の意味であるそちらにスライドさせた盤面を作成する関数 `slide_dir` を作成せよ. これまでの例題で作った関数を使うこと.

問題 GUI *

`ipywidget` を使って, GUI を作ってみよう. 当日サンプルコードを紹介する.

4.4 有限ライフゲーム

ライフゲーム (Conway's Game of Life) では無限に広がる二次元グリッドに配置された各マスについて, 生物が生きているか死んでいるかの 2 状態を扱う. この演習では有限のフィールドに限定し, 生死をそれぞれ `True`, `False` で表すことにする.

例題 初期状態作成

二次元配列の大きさ `height`, `width` と, 生きているマスの一覧を表す配列 `living` が与えられる. 各 $0 \leq i < \text{len}(\text{living})$ に対して, `living[i][0]` 行 `living[i][1]` 列目が生きているマスとする. 生きているマスが `True` そうでないマスが `False` を持つ二次元配列を返す関数 `make_field(height,width,living)` を作成せよ.



```
>>> make_field(2,3,[]) 1
[[False, False, False], [False, False, False]] 2
>>> make_field(1,3,[[0,0],[0,1]]) 3
[[True, True, False]] 4
```

ライフゲームの初期状態には、何ステップか経てやがて死滅するもの、やがて初期状態に戻り周期的に繰り返すもの、無限に増え続けるものなど様々なパターンがあることがわかっている。その中で移動しながら変形を繰り返す移動型の一つである、グライダーと呼ばれる初期状態を作成し、図示してみよう。

matplotlib というライブラリを用いる。



組み合わせ注意

かつてはブラウザ内で画像を表示するために、次のような magic command を用いていた。

```
%matplotlib inline 1
```

これらは使用中の Jupyter のバージョンに合わせて適切に使う必要がある。最近の Jupyter 7.x や Colab ではこれを書かなくても表示される模様である（むしろ書くとトラブルの要因になるかもしれない）。

インタラクティブに操作できるグラフを表示するために、%matplotlib notebook, %matplotlib ipympl もよく使われるが、前者は Jupyter version 7以降では動作しない（絵がでなくなる）。後者は別途インストール（pip install ipympl など）が必要。なお、前者を新しいバージョンで使う方法として、classic 環境を別途インストールして用いる（jupyter nbclassic で起動）という回避手段もあるが、授業では特に勧めない。

ある環境で絵が出ない場合は、`print(matplotlib.get_backend())`などを調査して、動いている環境の結果と比較すると良い。

<https://matplotlib.org/stable/users/explain/figure/backends.html>

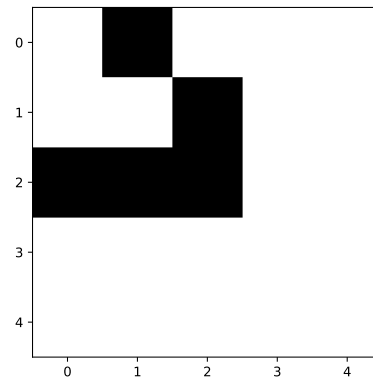
Python3

```
import matplotlib 1
import matplotlib.pyplot as plt 2
3
def plot(array2d): 4
    fig = plt.figure() 5
    ax = fig.add_subplot(1,1,1) 6
    im = ax.imshow(array2d, interpolation="nearest", 7
        cmap="Greys")
    fig.show() # fig.savefig("plot.pdf") 8
9
a = make_field(5, 5, [[0,1], [1,2], [2,0], [2,1], [2,2]]) 10
```

```
plot(a)
```

11

この `plot` 関数を、そのまま使って良い。¹⁾



各 `a[r][c]` について 0 が白に、1 が黒に対応し、位置は左上が原点で `r` が 1 増えると 1 行下の行が、`c` が 1 増えると 1 列右の列に対応する

ライフゲームでは次の時刻での各マスの生死は、現在のマスの状況と周囲 8 近傍の生きているマスの数に基づいて、次の規則により決定される。この規則に基づき次の時刻の状態を求める関数を順に作成しよう。

現在	8 近傍の生のマス			
	0,1	2	3	4-8
生	死	生	生	死
死	死	死	生	死



ModuleNotFoundError

必要に応じて `matplotlib` をインストールする。以下は、インストールが必要な場合のエラーの例

```
Traceback (most recent call last):
ModuleNotFoundError:
<ipython-input-1-6ea0e9114e84> in <module>
----> 1 get_ipython().run_line_magic('matplotlib', 'notebook')
      2 import matplotlib
      3 import matplotlib.pyplot as plt
      4
      5 def plot(array2d):
```

1) なお、全ての要素が `True` の配列を表示すると真っ白になるという“仕様”がある。`imshow` が値を正規化するため。

インストール方法は環境による e.g., `pip3 install matplotlib`.

例題 範囲内の座標

長方形形状のフィールドを表す二次元配列 `field` と、`r` 行 `c` 列目を表す整数 `r`, `c` が与えられる。座標がフィールド内部かどうかを真偽値で返す関数 `inside(field, r, c)` を作成せよ。整数 `r`, `c` は配列の外側であったり、負である場合もある。



```
>>> inside(make_field(1,2,[]), 0,0) 1
True 2
>>> inside(make_field(1,2,[]), 0,1) 3
True 4
>>> inside(make_field(1,2,[]), 0,2) 5
False 6
>>> inside(make_field(1,2,[]), 0,-1) 7
False 8
```

例題 座標のマス

指定されたマスの生死を整数で返す関数 `is_living_cell(field, r, c)` を作成せよ。生を 1, 死を 0 とする。ただし、指定されたマスがもしフィールドの外であれば、死と扱う。(引数の意味は前問と同じである)

例題 8 近傍の生きているマスの数

(引数の意味は前問と同じである) 指定されたマスの 8 近傍で生きているマスの数を数える関数 `count(field, r, c)` を作成せよ。ただし、8 近傍とは斜めを含めた隣のマスであり、 (r', c') が (r, c) の 8 近傍であれば (かつその時に限り), $|r' - r| \leq 1$, $|c' - c| \leq 1$, $(r', c') \neq (r, c)$ が成り立つ。

Python3

```
for i in [-1,0,1]: 1
    for j in [-1,0,1]: 2
        if (i != 0 or j != 0): 3
            ... # r+i, c+j が r,c の 8 近傍のどこか 4
```

例題 次の時刻の生死

ある特定のマスについて、現在の生死を表す整数 `living` と現在の 8 近傍で生きているマスの数を 0 以上 8 以下の整数で表す `neighbor_count` が与えられる。それらを元に、そのマスの次の時刻の生死を求める関数 `next_cell_living(living, neighbor_count)` を作成せよ。

問題 次の状態 (next_field)

真偽値 (True または False) の要素の長方形の領域を表す二次元配列 a を与えられて、ライフゲームのルールに従った次の状態を返す関数, next_field(a) を作成せよ。



```
>>> a
[[False, True, False, False, False], [False, False, True, False,
↪ False],
 [True, True, True, False, False], [False, False, False, False,
↪ False],
 [False, False, False, False, False]]
>>> next_field(a)
[[False, False, False, False, False], [True, False, True, False,
↪ False],
 [False, True, True, False, False], [False, True, False, False,
↪ False],
 [False, False, False, False, False]]
>>> plot(next_field(a))
```

Wikipedia http://en.wikipedia.org/wiki/Conway%27s_Game_of_Life#Examples_of_patterns や Wiki https://www.conwaylife.com/wiki/Main_Page などを参考に、固定パターンと、周期が 2 の小さなパターンを一つずつ試してみよう。

連続アニメーション これまでは 1 枚 1 枚の絵を show で別々に表示してきた。Python の場合, matplotlib でアニメーション作成機能がある。下記のサンプルコードでは, show_animation でアニメーションを表示している。

Python3

```
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.animation as animation

def show_animation(initial_field, step):
    a = []
    fig = plt.figure()
    field = initial_field
    for i in range(step):
        a.append([plt.imshow(field)])
        field = next_field(field)
    anim = animation.ArtistAnimation(fig, a, interval=500)
    plt.show()
    return anim
glider = [[0,1],[1,2],[2,0],[2,1],[2,2]]
show_animation(make_field(15,15,glider),50)
```

セルを実行すると、ノートブック上でアニメーションが再生される。



アニメーションだけ動かない

静止画像は表示されるがアニメーションができない場合は、アニメーションの処理方法を確認する。以下のように 'none' になっている場合は、'html5' もしくは 'jshtml' のどちらかの指定で動くかもしれない。

```
>>> plt.rcParams["animation.html"]
'none'
```

```
plt.rc('animation', html='html5')
```

```
plt.rc('animation', html='jshtml')
```

問題 移動型 (animation)

Glider や Lightweight spaceship など移動型を動かし、周期や移動幅を調査してみよう。各型ごとに、初期配置と自分の選んだ特徴的な時刻の合計二つの時刻のフィールドの状況に関するテストケースを付与すること。さらに、アニメーション (mp4 など) をファイルに保存して、合わせて提出せよ。

アニメーションの保存は、以下のように save を呼ぶと、jupyter を実行しているディレクトリにファイルが保存される。

Python3

```
anim = show_animation(make_field(15,15,glider),50)
anim.save("filename.mp4")
```

mp4 形式の保存が難しい場合でも、アニメーション gif 形式なら保存できる場合もある。

Python3

```
anim.save("filename.gif", writer='PillowWriter')
```

mp4 で保存するためには、事前準備が必要かもしれない。本教材では設定方法をサポートしないが、標準的にはまず ffmpeg をインストールする。たとえば mac で homebrew を使用中なら brew install ffmpeg し、インストールされた場所に合わせて

Python3

```
plt.rcParams['animation.ffmpeg_path'] = "path-to-ffmpeg"
```

などと指定する。最近の Mac であれば、"path-to-ffmpeg" は "/opt/homebrew/bin/ffmpeg" と

なるだろう。さらに、場合によっては、`Writer = animation.FFMpegWriter()` の指定も必要という報告もある。

例題 色つきの可視化

白黒を表す真偽値の代わりに、色を表す配列 $[r, g, b]$ (ただし $r \in [0, 1]$, $g \in [0, 1]$, $b \in [0, 1]$) を要素として与えると、カラー画像を表示できる。

```
a = [[0,0,0], [1,0,0], [0,1,0], [1,1,1], [0.5,0,0.5]]
plot(a)
```



問題 美しいライフゲーム (color_animation) *

彩色方法を工夫して、ライフゲームを題材に、美しいアニメーションを作成せよ。ソースコード等と合わせて、アニメーション (mp4 など) も提出せよ。(実装の訓練なので、matplotlib の `cmap` ではなく、直前の例題のように自分で色を指定してほしい)

4.5 この章の提出課題

以下のいずれかで合格:

- 問題 (タイトル部分に * なし) を全て Python で解く、ただし 4.3 成長スライドパズル と 4.4 節 有限ライフゲームはどちらかの選択で良い。(通常コース)
- * つきの問題を 1 題以上解く

提出条件:

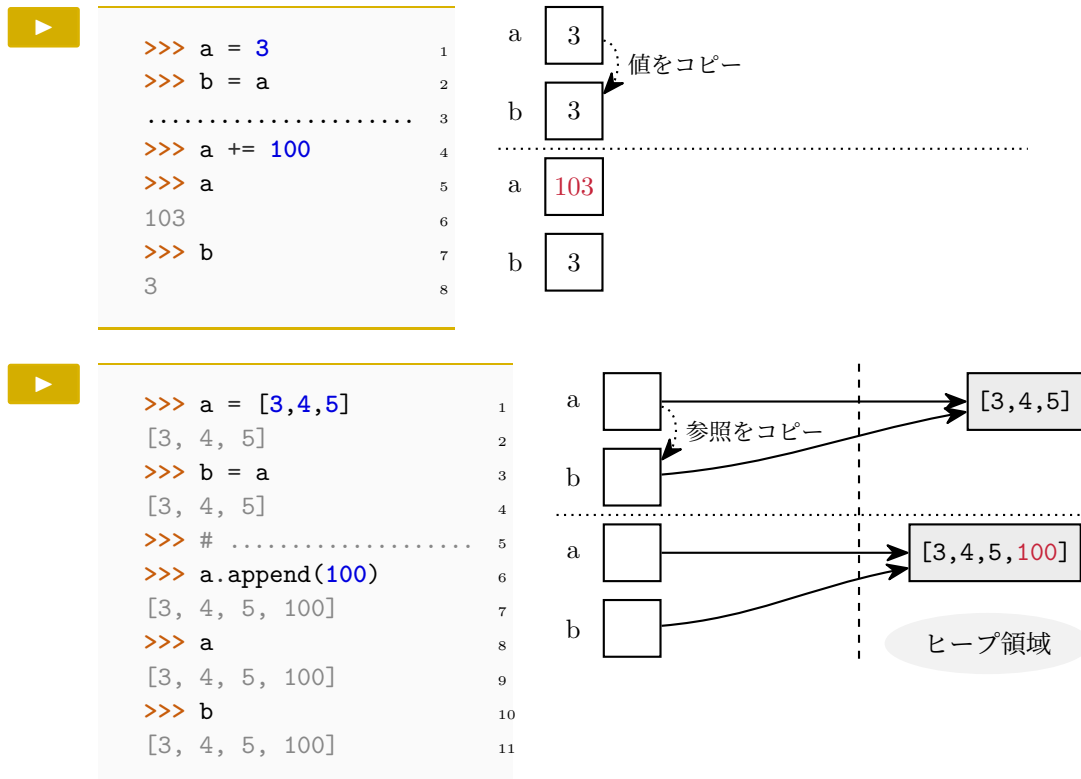
- 各関数に対応するテストケースが記述されていること
- ループ不変条件が明確にコメントで記述されていること (本日 4.1 節では必須, 他も適宜推奨)
- 冗長なコードがないか確認し, 標準的な書き方に整えられていること
- 指定した課題が, 指定した関数名で作成されていること
- 提出ファイルは, 2.A 節を参考にして, *jupyter から作成した* weekN-(学生証番号).ipynb とする
- 日本語 (または英語) でコメントが付記されていること: { 途中か完成品か, 授業中に既に OK を得ていればその教員名 } 学んだこと, (あれば) 他に参考にした資料や他の人の回答など, (途中で提出する場合は何がうまくゆかずに困っているなど) をコメントで盛り込む。(何も苦労がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること — 授業中に教員の

OK を得た場合は簡潔で良い)

4.A 実体と参照★

変数とは何か、最初の説明として「データの入れ物に変数である」などと説明されるが、入れ物の中に何が入っているか、複雑なプログラムを書く際は実体と参照の区別が必要である。多くのプログラミング言語で、**型**によって実体を扱うか参照を扱うかの区別がある。たとえば Java では primitive type (整数など) と reference type (String など) と明確に区別される²⁾。Python で変数の型は type 関数により知ることが出来る。

参照の複製 次の二つの例で数と配列の違いを見てみよう: 整数の場合は、変数には値そのものが格納されている (と理解できる)。したがって `b=a` とした後で、`a` の値を変更しても、`b` の値は変わらない。



一方、配列の場合は、配列の実体はヒープ領域と呼ばれる別の場所に確保され、矢印で図示した**参照**によって実体と変数と関連付けられる。矢印の先を参照先、元を参照元と呼ぶ。`b=a` とした際は参照のみがコピーされ、参照先の変更は、`a`、`b` どちらにも反映される。

2) <https://docs.oracle.com/javase/specs/jls/se7/html/jls-4.html>

```

1 >>> a = [3,4,5]
2 [3, 4, 5]

3 >>> import copy
4 >>> b = copy.deepcopy(a)

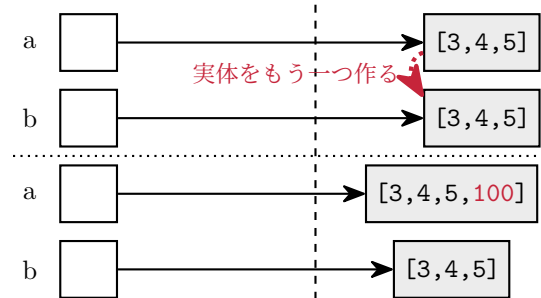
5 [3, 4, 5]
6 .....

7 >>> a.append(100)
8 >>> a

9 [3, 4, 5, 100]
10 >>> b

11 [3, 4, 5]

```



参照ではなく、実体を複製したい場合は、新しい配列をリスト内包表記や `[]` で構築して各要素を自分でコピーする、もしくは `copy.deepcopy` というメソッドを用いる。

関数呼び出しと参照 Python や C++ では関数呼び出しの際に引数に、値が複製される。これを**値渡し**と呼ぶ。

Python3

```

1 def add3(n):
2     n += 3
3     return n

```

```

1 >>> n = 5
2 5
3 >>> add3(n)
4 8
5 >>> n
6 5 # n の値は元のまま

```

すなわち関数内部で引数の値を変更しても、関数の呼び出し元の変数の値には影響がない。しかしながら、引数が参照である場合には、複製された参照は同じ参照先を指すために、参照先の変更は関数の呼び出し元からも可視である。

Python3

```

1 def add3_array(a):
2     a.append(3)

```

```
return a
```

3



```
>>> a = [1,2]
>>> add3_array(a)
[1, 2, 3]
>>> a
[1, 2, 3] # a の指す配列は変更されている
```

1

2

3

4

5



```
>>> a = 1
>>> type(a)
<class 'int'>
>>> b = 1.0
>>> type(b)
<class 'float'>
>>> c = [1, 2, 3]
>>> type(c)
<class 'list'>
```

1

2

3

4

5

6

7

8

9

二次元配列と参照 Python では配列の配列として、二次元配列を実現する。



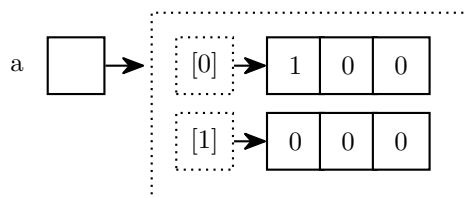
```
>>> a = make2d(2,3)
>>> a[0][0] = 1
>>> a
[[1, 0, 0], [0, 0, 0]]
```

1

2

3

4



失敗例:



```
>>> a = [[0]*3]*2
>>> a
[[0, 0, 0], [0, 0, 0]] # 一見, 二次元配列
>>> a[0][0] = 1
>>> a
[[1, 0, 0], [1, 0, 0]] # 2箇所変更されている
```

1

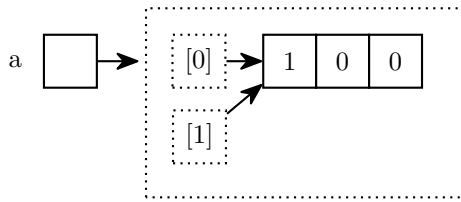
2

3

4

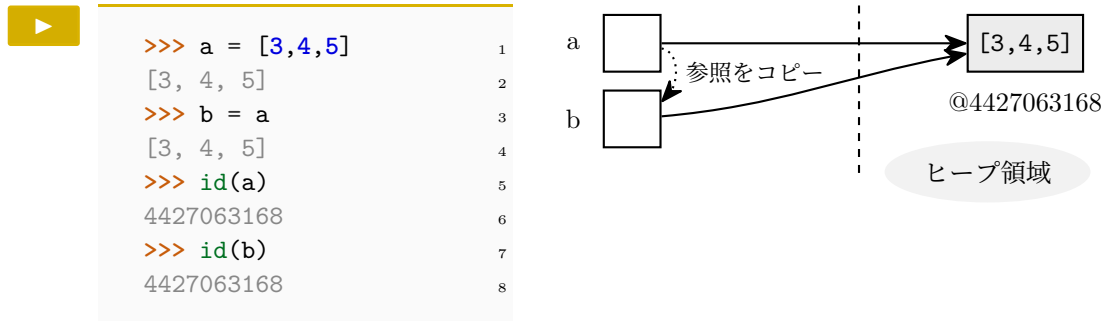
5

6

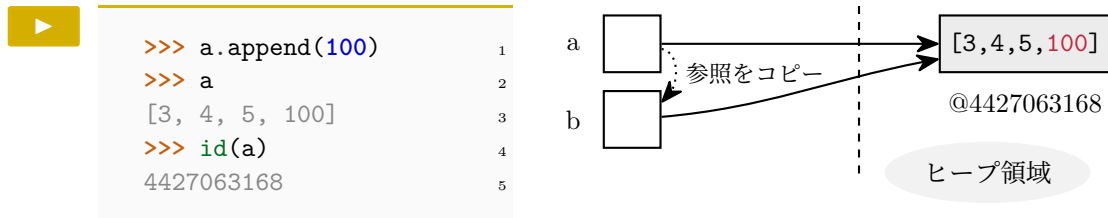


補足 — Python ではすべてが object** 厳密には Python は、整数も含めて、すべての型の変数が object への参照を持つ。つまり、ここまでの説明では、ある種の「方便」も含んでいる。しかしながら、整数など immutable (変更不可) な型 (特に int, float) には、以下のように特別な扱いが適用されるため、総合すると他の言語での実体と同等と考えて整合する。

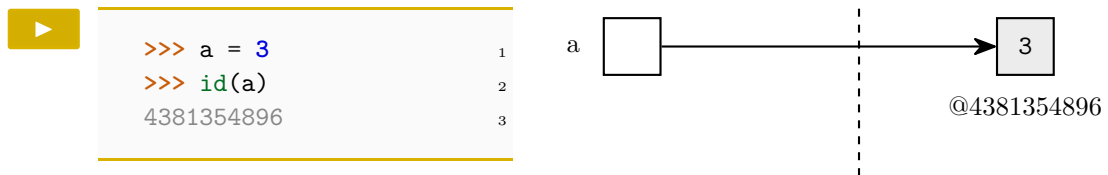
Python object の identity (≈ 参照先) は、組み込み関数 `id` により得られる。これは CPython ではメモリアドレスに対応する。



リスト (mutable な型) の identity は、`append` などに変更しても変わらない。



整数など immutable の object としての参照先も、`id` により確認できる



ここで、`+=` により `a` の値 3 から 4 に増やしてみる。すると `a` の指す参照先の `id` が変化している。


```

    string name = "Taro";                                20
    string& copy = name; // name を参照                    21
    copy[0] = 'J';                                         22
    copy[1] = 'i';                                         23
    cout << name << endl; // Jiro                          24
    cout << copy << endl; // Jiro                          25
}                                                         26
{                                                         27
    string name = "Taro";                                28
    decorate(name);                                       29
    cout << name << endl; // Taro                          30
}                                                         31
{                                                         32
    string name = "Taro";                                33
    decorate_ref(name);                                   34
    cout << name << endl; // Taro-san                      35
}                                                         36
}                                                         37

```

4.B 先週の補足

どのようなテストケースを作るべきか 作成したテストケースが十分かどうか心配というコメントがあった。現状ほぼ問題ないが、不足があれば個別に指摘する。

一般的な指針は以下のとおりである：

- 標準的な値とその周辺 (例: 閏年判定なら閏年とその ± 1 年)
- 条件判断 (if) があれば、条件が成り立つ場合と成り立たない場合
- 引数の仕様の範囲での極端な値 (整数であれば、最大値、最小値、0、文字列であれば空文字列 "", 空の配列など)
- ループ (for, while など) があれば、ループの最小回数や最大回数を与える入力 (分析可能な場合)

なお、この演習では手でテストケースを作成し、明らかな間違いに速やかに気づくことを目的としている。実用的には必要に応じて、自動的なテストも併用される。入力として与えられる種類が小さければ全ての場合について入出力を網羅できる場合もある。またそうでなくても、可能な範囲のデータで耐久テストをすることもある (例: 将棋の全ての盤面をテストすることはできないが、流通している棋譜に現れる局面では正常に動作する)。

異常な入力 (「整数 n に対して」と問題文に書いてあるのに文字列を与えるようなケース) に対しては、煩雑を避けるためにこの演習では (特に指示がなければ) 対処不要とする。

第 5 章 再帰

Recursion
2026-05-13 Wed

概要

再帰について演習する。線形の再帰 (for で等価な内容を書ける) から始めて、徐々に複雑なものを扱う。複雑な再帰は (簡単には) 反復ではかけないので、再帰で綺麗に書けるような処理の手順が存在することとその書き方について理解しよう。

5.1–5.3 の問題は、変数の破壊的代入を用いずに (i.e., 定数変数のみを用いて), 解くこと。

この章も、**テストケースを提出に添える**こと。

本章の再帰は次週以降のためにも重要で、また人によって理解しやすい説明が異なりうるので、10 分程度つまったら質問することを推奨。

5.1 帰納的定義と線形再帰

「何かを繰り返して計算したい」状況を考える。while や for のような反復・ループによって書く方法と、関数の中から自分自身を呼び出す**再帰** (recursion) によって計算する方法がある。ここでは後者について掘り下げる。

例として、 $1 + 2 + \dots + n$ に相当する関数を、**漸化式**で定義してみよう。この関数を $\text{sum}(n)$ と書くことにすると、次のように帰納的に定義できる。

$$\text{sum}(n) = \begin{cases} 1 & n = 1 \text{ のとき} \\ n + \text{sum}(n - 1) & \text{それ以外} \end{cases}$$

この定義の中では sum 自身を使っていることに注意。これは、ほとんどそのまま Python や C++ の定義に置き換えることができる:

Python3

```
def sum1(n):
    if n == 1:
        return 1
    else:
        return n + sum1(n-1)
```

例題 階乗 (再帰)

正の整数 n についての階乗 ($n! = 1 \times 2 \times \dots \times n$) の帰納的な定義と関数 `factorial(n)`.

例題 倍数の個数 (再帰)

「1 から正の整数 n までの中にある正の整数 m の倍数の個数」を、再帰で数える関数 `count_multiples(n, m)`. この関数を $f(n, m)$ とすると、帰納的には次のように定義できる.

$$f(n, m) = \begin{cases} 0 & n = 0 \text{ の場合} \\ 1 + f(n-1, m) & n \text{ が } m \text{ の倍数の場合} \\ f(n-1, m) & \text{それ以外} \end{cases}$$

末尾再帰 (tail recursion) 式を少し変形した、次の定義の $\text{sum}'(s, n)$ を考える.

$$\text{sum}'(s, n) = \begin{cases} s + 1 & n = 1 \text{ のとき} \\ \text{sum}'(s + n, n - 1) & \text{それ以外} \end{cases}$$

上記の定義中の s は、「これまでの和」に相当すると考えて、 $\text{sum}(n) = \text{sum}'(0, n)$ であることを確認せよ.

Python3

```
def sum2(s,n):
    if n == 1:
        return s+1
    else:
        return sum2(s+n, n-1)
```

返り値として自分自身を呼ぶ構造を**末尾再帰**と呼ぶ. このように書いておくと、コンパイラが最適化しやすいというメリットがある.

また、 s のような補助的な変数を導入すると再帰の見通しがよくなることがある.

5.2 範囲を絞る: 二分法・二分探索

例題 最大公約数

与えられた 2 つの数 a, b の最大公約数を求める関数 `gcd(a, b)` を作成せよ. ただし $a > b \geq 1$ とする.

$$\text{gcd}(a, b) = \begin{cases} b & b \text{ が } a \text{ を割り切るとき} \\ \text{gcd}(b, a \% b) & \text{それ以外} \end{cases}$$

例題 平方根

再帰を用いて、与えられた数 x の平方根である $\sqrt{x}$ を求める関数 `sqrt(x,a,b)` を作成せよ。

与えられた数 $x > 0$ の平方根である $\sqrt{x}$ を求める関数 `sqrt(x,a,b)` を考える。予め区間 $[a, b]$ 、すなわち $a < \sqrt{x} < b$ に解があることが分かっているとする。すると、 $\sqrt{x}$ の単調性から $m = \frac{a+b}{2}$ として

$$\text{sqrt}(x, a, b) = \begin{cases} m & |m^2 - x| < \epsilon \text{ のとき} \\ \text{sqrt}(x, a, m) & m^2 > x \text{ のとき} \\ \text{sqrt}(x, m, b) & \text{それ以外} \end{cases}$$

ただし、 ϵ は許容誤差を表す。たとえば 0.0001 を用いる。また正しい区間、すなわち $a^2 < x < b^2$ となるような a, b を与える必要がある。

実行例:



```
>>> sqrt(3, 0, 10)
1.732025146484375
```

1
2

実装例

Python3

```
Epsilon = 0.0001
def sqrt(x, a, b):
    m = (a+b)/2.0
    # print(x,a,b,m) # どのような引数で呼び出されているか観察できる
    if abs(x - m*m) < Epsilon:
        return m
    elif m*m > x:
        return sqrt(x, a, m)
    else:
        return sqrt(x, m, b)
```

1
2
3
4
5
6
7
8
9
10

浮動小数演算は誤差を含むので、自動テストの際は `pytest.approx` を用いて許容誤差を指定する¹⁾

Python3

```
Epsilon = 0.0001
assert 1.4142 == pytest.approx(sqrt(2, 0, 10), abs=Epsilon)
```

1
2

問題 シグモイド関数 (`sigmoid.inverse`)

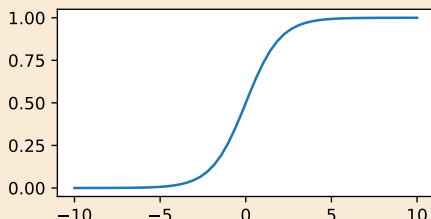
シグモイド関数 $f(x) = \frac{1}{1+e^{-x}}$ は、 $x \in [-\infty, \infty]$ を $[0, 1]$ にマップする。再帰を用いて、与えられた数 y と区間 $[a, b]$ に対して $f(x) = y$ となるような x を求める関数 `sigmoid.inverse(y,a,b)`

1) <https://docs.pytest.org/en/latest/reference/reference.html?highlight=approx#pytest.approx>

を作成せよ。テストケースの書き方は、直前の例題を参照。

なお e^x は `math.exp(x)` で計算できる (`import math` を一度行う)。

関数形の把握のための図示の例



実行例

```
import matplotlib.pyplot as plt
import numpy as np

fig=plt.figure(figsize=(4,2))
x = np.linspace(-10,10)
plt.plot(x, 1/(1+np.exp(-x)))
```



```
>>> sigmoid_inverse(0.5, -10, 10)
0.0
>>> sigmoid_inverse(0.2, -10, 10)
-1.3862943611
>>> sigmoid_inverse(0.1, -10, 10)
-2.1972245773
>>> sigmoid_inverse(0.9, -10, 10)
2.1972245773
```

なお、上記の実行例は、 x の区間を幅 `eps=1e-11` まで分割し、`round` 関数で 10 桁に丸めたものである。



無限ループ

実装を誤ると、計算が終わらず、どこかで制限を超えて `Maximum recursion depth exceeded` というエラーで停止する。仕組みは、次の囲み記事を参照。 `sigmoid_inverse(y,a,b)` を作っていてこのエラーが出た場合は、(1) y と区間 $[a,b]$ の対心が `float` のせいで外れていないか (正常な場合は $f(a) < y < f(b)$)、(2) (整数除算の失敗などで) $(a+b)/2 == a$ または $(a+b)/2 == b$ になっていないか、その場合には再帰を深めても $[a,b]$ が狭まらない などを確認する。確認には、`a,b,f(a),f(b),y` などを関数の入り口で表示すると良い。



Maximum recursion depth exceeded

再帰関数は、どこかで止まるように実装する必要がある。たとえば以下の関数は止まるところが定義されていない。

```
def inf():
    return 1+inf()
```

この関数を `inf()` と呼び出すと,

```
>>> inf()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in inf
  File "<stdin>", line 2, in inf
  File "<stdin>", line 2, in inf
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

というように, `RecursionError: maximum recursion depth exceeded` というエラーで処理を続行できなくなった旨表示される。

問題 二分探索 (binary_search)

配列 a の要素は整数で昇順に整列されているとする。添字の範囲 $[l, r)$ の a の要素に整数 x と等しいものが存在すればその添字, そうでなければ -1 を返す関数 `binary_search(a, x, l, r)` を, 再帰を用いて作成せよ。なお探す範囲は半开区間であり, $l \leq i < r$ で $a[i] == x$ となるような i を探す。また正しく計算するために $a[l] \leq x < a[r]$ かつ $0 \leq l < r \leq \text{len}(a)$ となるような l, r を与えるとする。配列 a の要素は互いに異なると仮定して良い。

なお, 変数名 l, r は `left` と `right` からとられている。

簡単に `binary_search` を `bs` と略記すると, 帰納的定義を配列の要素の単調性から次のように書ける:

$$\text{bs}(a, x, l, r) = \begin{cases} l & l+1 = r \text{ かつ } a[l] = x \text{ のとき} \\ -1 & l+1 = r \text{ かつ } a[l] \neq x \text{ のとき} \\ \text{bs}(a, x, l, m) & l+1 < r \text{ かつ } a[m] > x \text{ のとき, 但し } m = \lfloor (l+r)/2 \rfloor \\ \text{bs}(a, x, m, r) & \text{それ以外} \end{cases}$$

注: Python3 で $m = \lfloor (l+r)/2 \rfloor$ を計算するためには, 整数除算//演算子を用いると良い。



```
>>> a=[1,3,5,7]
[1, 3, 5, 7]
>>> binary_search(a, 1, 0, len(a))
0
```

```

>>> binary_search(a, 3, 0, len(a))           5
1                                              6
>>> binary_search(a, 5, 0, len(a))           7
2                                              8
>>> binary_search(a, 7, 0, len(a))           9
3                                              10
>>> binary_search(a, 2, 0, len(a))           11
-1                                             12
>>> b=[0,1,10,100,1000]                      13
>>> binary_search(b, 1, 0, len(a))           14
1                                              15

```

辞書や名簿のように順番に並んでいるデータ列にある要素が存在するかどうかを判定する問題も、同様の考え方で解くことができる。たとえば“Moore”という姓を、全体が 1024 ページの名簿から探す場合に、1024 ページの真ん中の 512 ページから始める。そのページが“M”より後なら、前を調べる。“M”以前なら、後を調べる。いずれの場合でも、探す範囲を半分に絞ることが出来るといった具合である。この探し方は、1 ページ目から順に 2,3 ページと一ページずつ最終ページまで名簿を探すより、速い。名簿のページ数を n とすると、調べるページ数は $O(n)$ と $O(\log_2(n))$ の差がある。

5.3 枝分かれを伴う再帰

ここまで見た再帰呼び出しでは、1 つの関数からその関数を 1 回だけ呼び出していた。今節以降は、複数の再帰を呼び出す場合を扱う。

例題 フィボナッチ数

フィボナッチ数列 (*Fibonacci numbers*) とは、

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

のように、「前 2 つの数の和」で作られる数の列である。 n 番目のフィボナッチ数 $fib(n)$ を帰納的に定義せよ。また、関数 $fib(n)$ を定義せよ。

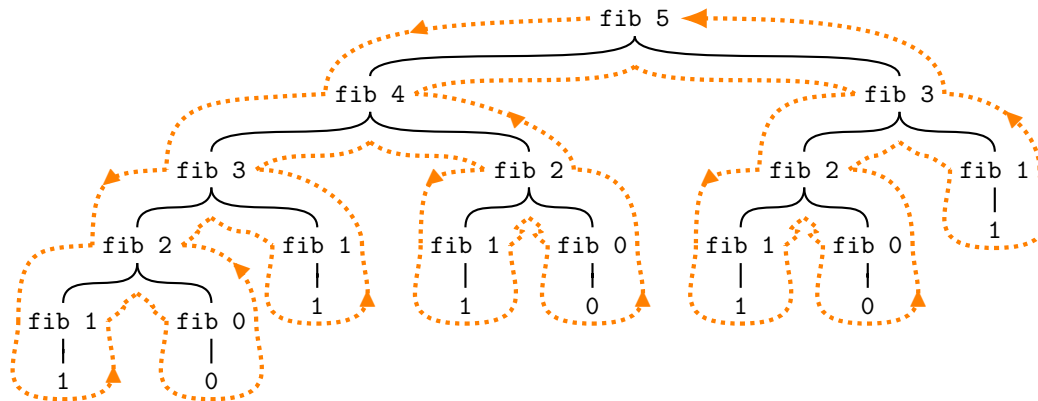
Python3

```

def fib(n):                                     1
    print("fib",n) # 関数呼び出しの際に引数を表示 2
    if n == 0:                                  3
        return 0                                4
    elif n == 1:                                5
        return 1                                6
    else:                                       7
        return fib(n-2)+fib(n-1)                8

```

再帰的に定義した関数で `fib(5)` を計算する様子は下図のようになる。



なお再帰は fibonacci 数を計算することに関しては最善の方法ではない。より効率良くフィボナッチ数を求める方法は別に扱う。

例題 m 進数

整数 m, n が与えられる ($1 \leq n \leq 8, 1 < m \leq 10$). n 桁の m 進数を小さい順に全て表示する関数 `mdigit(m, n)` を作成せよ。



```
>>> mdigit(3,2)
```

```
00
01
02
10
11
12
20
21
22
```

```
1
2
3
4
5
6
7
8
9
10
```

簡単のため $m = 2$ すなわち 2 進数を考える。正の整数 n に対して目的を達する手続きを、 $n - 1$ に対して同様の処理を行う手続きを元に組み立てたい。つまり、 $n - 1$ 桁の 2 進数を全部表示する関数を誰かが作ってくれていたとして (*1), それを元に n 桁の m 進数を全部表示する関数を作れるだろうか。

n 桁の 2 進数は左端の 1 文字 (0 または 1) と残り $n - 1$ 桁の 2 進数に分解できる。そこで、左端に 0 を追加して $n - 1$ 桁の 2 進数を全部表示する、左端に 1 を追加して $n - 1$ 桁の 2 進数を全部表示する、というようなことをしたい。

上記を踏まえて (*1) の関数を微調整して、左端に書いてほしい文字列 `prefix` を引数に追加し、引数 `prefix` と n に対して、「 n 桁の 2 進数を全て生成して、それぞれに `prefix` をつけて表示する」という関数を `B(prefix, n)` とする。

$$B(\text{prefix}, n) = \begin{cases} \text{prefix を表示} & (n = 0) \\ B(\text{prefix} + 0, n - 1) \text{ と } B(\text{prefix} + 1, n - 1) \text{ を実行} & (n > 0) \end{cases} \quad (5.1)$$

prefix	3	2,1	
10	0	00	← B(100,2) を通じて実現
10	0	01	
10	0	10	
10	0	11	
10	1	00	← B(101,2) を通じて実現
10	1	01	
10	1	10	
10	1	11	

B(10,3) の動作: B(100,2) と B(101,2)

$B(0,n)$ が, n 桁の 2 進数を全て表示することを確認せよ. 2 進数だけでなく m 進数に対応するには, 0 と 1 のみ分岐させていた部分を, for 文などで 0 から $m-1$ まで分岐させれば良い.

prefix は文字列 (str) を想定して説明したが, int で表現しても良い. その場合, たとえば, prefix の右に 1 を加える操作は, $\text{prefix} * 10 + 1$ となる.

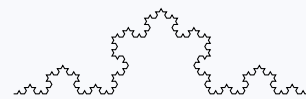
文法: 整数 a を 5 桁で, 桁が足りない際に 0 を補った文字列を得るには, 以下のように行う.

Python3

```
>>> a = 3
>>> f"{a:05d}"
00003
```

問題 Recursion / Divide and Conquer - Koch Curve (koch)

フラクタルの一つである**コッホ曲線** (koch curve) の頂点の列を計算して配列として返す関数 $\text{koch}(x_0, y_0, x_1, y_1, n)$ を作成せよ. ただし点 (x_0, y_0) と (x_1, y_1) を始点と終点とする. また再帰の深さを n とする. 頂点列を図示して確認もすること.



テスト用のデータ例: http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_5_C&lang=jp

ループを用いずに再帰だけで書くことができる. また引数の破壊的変更も不要である.

Python3

```
# (x0,y0) から (x1,y1) までのコッホ曲線の頂点列を返す
# ただし (x1,y1) は含まない
def koch(x0,y0, x1,y1, n):
    if n == 0:
```

```

        return [[x0,y0]] # (x0,y0)..(x1,y1) の直線
    else:
        # 変数の準備
        # 再帰 ... 部分は適当に埋める
        return koch(x0,y0, ..., n-1) + koch(..., n-1) \
            + koch(..., n-1) + koch(..., x1,y1, n-1)

```

ヒント: 基準線が (x_0, y_0) から $(x_0 + dx, y_0 + dy)$ に引かれるとき, 左手側にこぶを作るには, 中点 $(x_0 + dx/2, y_0 + dy/2)$ から, $(-dy, dx)$ 方向に適当な距離を (i.e.,) 伸ばせば良い.

matplotlib による描画 作成した関数の結果を描画してみよう. 関数 `koch` の返り値 `a` が `[[0,0],[100,0]]` のような点を表す配列の配列だとする.

Python3

```

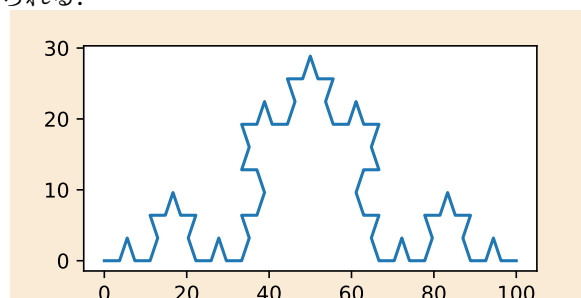
import matplotlib.pyplot as plt
import numpy as np
def plot_koch(a):
    fig = plt.figure()
    ax = fig.add_subplot(1,1,1)

    a_ = np.array(a)
    xlist = a_[:, 0]
    ylist = a_[:, 1]

    ax.plot(xlist, ylist)
    ax.set_title('koch curve')

```

上記の関数 `plot_koch` で, jupyter notebook 上に結果を描画することが出来る. なお, 関数 `koch` の返り値は終点を含まないのので, `a = koch(0, 0, 100, 0) + [[100, 0]]` のように明示的に終点を加えると期待する図を得られる.



💡 test cases

浮動小数点数のリストの (リストの) 比較を, 許容誤差を考慮してテストするには, `np.allclose` を用いると簡便.

```
import numpy as np
np.allclose([3, 1.4], [[3, 1.4001]], atol=1e-3) # True
```

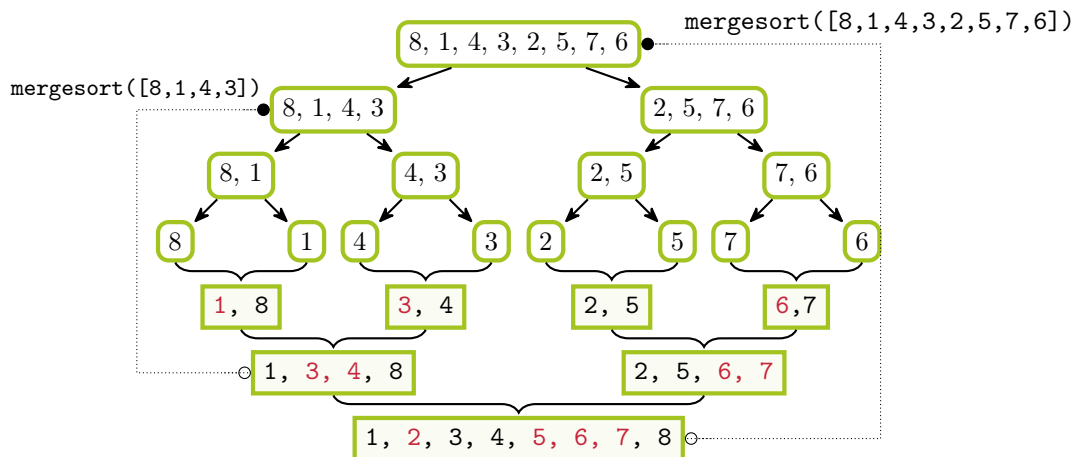
<https://numpy.org/doc/stable/reference/generated/numpy.allclose.html>

5.4 併合整列 (Merge Sort)

再帰的な考え方の応用の例として、*merge sort* という整列手法を取り上げる。この手法は、与えられた配列を半分に分割し、それぞれを整列する。そのうえで、整列された2つの配列を一つにマージ(併合)する。左右の半分の配列の整列も、同じ手順を適用する。整列対象の配列はどんどん小さく(要素数ほぼ半分に)なるので、最終的には、要素数1の配列になって、整列の必要がなくなる。

図は、8,1,4,3,2,5,7,6 という配列を整列する際の処理の流れを示している。上半分で分割し、下半分で整列が行われる。赤字は併合の際に右側から来た要素を示す。配列の要素数を N とすると、行が $O(\log N)$ あり、各行あたりで必要な計算が $O(N)$ なので、全体で $O(N \log N)$ という計算量が導かれる。

プログラムとしては、分割と併合の1段のみ書けば十分なことに注意せよ。冒頭の `sum(n)` の例題で、`sum(n-1)` の処理を任せて `n` の加算のみ書いたように、分割された部分をソートすることは再帰先に任せることができる。



例題 併合 (merge)

整数を要素とし昇順に整列された配列 a, b を与えられて、全ての要素を昇順に並べた新しい配列を返す関数 `merge(a, b)` を作成せよ。



```
>>> merge([1,3,5], [2,4])
[1, 2, 3, 4, 5]
```

1
2

この併合の準備のために、回数を定めず条件が成り立たなくなるまで繰り返す、`while` という構文を用意する。

Python3

```
while 条件:
    条件が成り立っている間繰り返し実行する文 またはブロック
```

1
2

Python3

```
def while_test(a):
    while a > 0:
        print(" ", a)
        a = a//2
    return a
```

1
2
3
4
5



```
>>> while_test(128)
128
64
32
16
8
4
2
1
0
```

1
2
3
4
5
6
7
8
9
10



slice

Python では、配列 `a` の区間 $[l, r)$ の部分の配列を取り出す記法として、`a[l:r]` という記法が用意されている。これを *slice* という。

Python3

```
def merge(a, b):
    c = []
    i, j = 0, 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            c.append(a[i])
            i += 1
        else:
            c.append(b[j])
            j += 1
```

1
2
3
4
5
6
7
8
9
10

```

    # ループ不変条件: a[0:i] と b[0:j] は昇順に c に詰まっている
    c += a[i:] # a[i] 以降の a の要素を全て c に詰める
    c += b[j:]
    return c

```

例題 分割

整数を要素とする配列 `a` を与えられて、前半の配列を返す関数 `first_half(a)` と後半の配列を返す関数 `second_half(a)` を作成せよ。奇数の場合は前半の要素が一つ多いものとする。



```

>>> a = [0,1,2,3,4]
[0, 1, 2, 3, 4]
>>> first_half(a)
[0, 1, 2]
>>> second_half(a)
[3, 4]
>>> b = [0,1]
[0, 1]
>>> first_half(b)
[0]
>>> second_half(b)
[1]

```

問題 併合整列 (mergesort)

整数を要素とする配列 `a` を与えられて、全ての要素を昇順に並べた新しい配列を返す関数 `mergesort(a)` を作成せよ。

注 関数 `merge` の動作テストを終えてから、`mergesort` の動作確認を行うこと。



```

>>> mergesort([8,3,4,1,5,9,6,7,2])
[1, 2, 3, 4, 5, 6, 7, 8, 9]

```

5.5 再帰と反復の応用

5.1 節の問題は `for` ループで、5.2 節の問題は `while` ループを用いることで、再帰を使わずに書くことができる。この節の問題は、再帰または反復の適切な方もしくは両方で書いて見よう。(この節の問題は、他が終わってから取り組むこと)

問題 Recursion / Divide and Conquer - The Number of Inversions*
(inversion_count)

配列内の要素のペアで, $a[i] > a[j]$ ($i < j$) のものを数えたい. 例えば配列 $[3\ 1\ 2]$ の中には $(3,1)$ と $(3,2)$ の二つのペアの大小関係が逆転している. 愚直に次のようなコードを書くと, 要素数 N の自乗に比例する時間がかかる ($O(N^2)$). Merge sort の応用で, 半分に分割しながら数えると, $O(N \log N)$ で求めることができる.

そのような関数 `inversion_count(a)` を作成せよ.

参考: http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_5_D

制約を満たす最小値を求める 配列から値を探す状況以外にも, 二分法の考え方をを用いると綺麗に解ける問題もある.

問題 Search - Allocation* (min_sufficient_load)

様々な重さの荷物を, 並べられた順にトラックに積むことを考える. 荷物の重さは並べられた順に配列 w に格納してあるとする. 全てのトラックの最大積載量は同じとする. k 台のトラックにすべての荷物を順に積む切するためには, 各トラックの最大積載量は最低どれだけ必要か. 「トラックの最大積載量」の最小値を求める関数 `min_sufficient_load(w, k)` を作成せよ.

参考: http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_4_D

ヒント: トラックの最大積載量 P として二分探索し, 全ての荷物を積める最小値を求める. 積みきれない最小値がほしいので, 「積みきれない最大値」が存在する範囲を $[l, h)$ で表現し, 上限 h を減らす条件を `ok` として表現する. 求める答えは h となる.

`ok` 関数は, たとえば以下のように作成できる:

- 現在のトラックに積んだ重さ (初期値 0) とトラック台数 (初期値 1) を表す変数を作成
- 各荷物について, 現在のトラックに積んで最大積載量を超えないならそのまま積む (現在のトラックに積んだ重さを増やす)/そうでなければ新しいトラックに積む (トラックの台数を増やして, 現在のトラックに積んだ重さをその荷物に設定)
- 最後に台数を K と比較

三分法 : 2 分法では区間を 2 つに分けて一つを除外したが, 似た考え方として区間を 3 つに分けて 1 つを除外する方法を紹介する.

問題 極値 * (extremum)

二次式 $ax^2 + bx + c$ は $a > 0$ とすると, 最小値を持つ. 最小値を含む区間 $[l, r]$ を与えられて, 再帰的に範囲を絞ることで最小値を求める関数 `extremum(a, b, c, l, r)` を作成せよ.

ヒント: $m_1 = (2l + r)/3, m_2 = (l + 2r)/3$ とすると, 最小値が存在する区間を, $[l, m_2]$ または $[m_1, r]$ に絞ることができる.

ミニマックス探索：オセロのような二人ゼロ和ゲームを考える．終端 (終局時の) 局面 p の各プレイヤーの得点を, (p の自分の石の数 $-p$ の相手の石の数) とする．プレイヤーを固定して, $\text{score}(p)$ = 「 p の黒石の数 $-p$ の白石の数」とすると, 黒は score を最大化する, 白は score を最小化するように打つことが目標となる．互いに最善を尽くすことを仮定すると, 手番 $t \in \{\text{黒}, \text{白}\}$ の (終局とは限らない) 局面 p の, 両者最善を尽くした結果 $f(p, t)$ は以下のように再帰的に定義される．ただし $\text{succ}(p, t)$ は, 局面 p の次の〈局面, 手番〉の組の集合とする:

$$f(p, t) = \begin{cases} \text{score}(p) & p \text{ が終端} \\ \max_{\langle s, t' \rangle \in \text{succ}(p, t)} f(s, t') & p \text{ が終端でない, かつ } t \text{ は黒番} \\ \min_{\langle s, t' \rangle \in \text{succ}(p, t)} f(s, t') & p \text{ が終端でない, かつ } t \text{ は白番} \end{cases}$$

問題 オセロ ** (othello)

4x4 のオセロについて, 黒と白が互いに最善を尽くした場合の結果を計算するプログラムを作成せよ.

プログラムを書く際に $\text{succ}(p)$ を陽に求めるのは効率が良くないので, 局面 p の空白に手番の石をおいてみて, 合法手であれば (石を返せたなら) それを $s \in \text{succ}(p)$ として計算し, そうでなければ $\max$ や $\min$ の計算から除外すれば良い．オセロのルールでは, パスは合法手が一手もないときのみ可能である．また石を返せない場所に置くことはできない．この演習では, それらのルールを多少変更しても良いが, 変更は明記すること．

ヒント: 局面を, 黒 1, 白 -1 , 空白 0 の二次元配列で表す．局面 p で手番 t (1 または -1) がマス x, y に打った後の局面を返す関数 $\text{place}(p, t, x, y)$ を作成し, 十分テストすると良い．局面 p でマス x, y に打てない場合は, 配列以外の目印となる値 (たとえば `False`) を返すとする．

5.6 この章の提出課題

以下のいずれかで合格:

- Section 5.4 までの問題 (5.4 を含む, タイトル部分に ★ なし) を全て Python で解く (通常コース)
5.1–5.3 の問題は, 変数の破壊的代入を用いずに (i.e., 定数変数のみ使用して) 解くこと.
なお提出は求めないが, 末尾再帰への書き換えと, 「 m 進数」も習得しておいてほしい
- “Koch Curve” と Section 5.5 の問題のなかから 2 題以上解く. (通常コースと発展コースの間)
“Koch Curve” は, 変数の破壊的代入を用いずに (i.e., 定数変数のみ使用して) 解くこと
- 「オセロ」を解く (発展コース)

提出条件:

- 各関数に対応するテストケースが記述されていること
- ループを含むコードは, ループ不変条件が明確にコメントで記述されていること
- 冗長なコードがないか確認し, 標準的な書き方に整えられていること

- 指定した課題が、指定した関数名で作成されていること
- 提出ファイルは、2.A 節を参考にして、*jupyter* から作成した `weekN-(学生証番号).ipynb` とする
- 日本語 (または英語) でコメントが付記されていること: { 授業中に既にフィードバックを得ていればそのスタッフの名前, 学んだこと, (あれば) 他に参考にした資料や他の人の回答など、(途中で提出する場合は何がうまくゆかずに困っているなど) } をコメントで盛り込む. 何も苦労がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること — 授業中にスタッフのコメントを得ている場合は簡潔で良い

5.A 関数と実行状態の管理

「文が上から下に実行されることと再帰呼び出しの関係」について, 補足する.

通常関数呼び出しの場合 多少作為的だが, 次のようなソースコードがあったとする. 関数 `helloworld` を実行すると, `hello` と `world` が一行ずつ表示される.

Python3

```

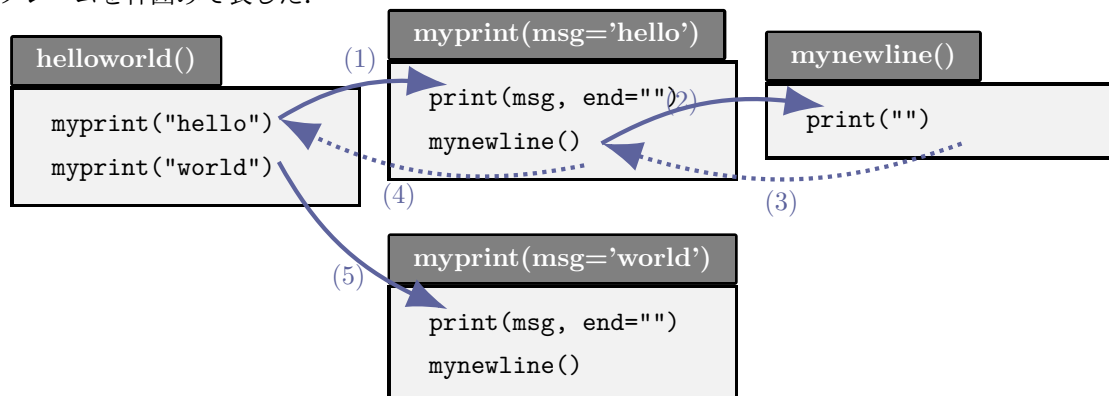
def mynewline():                                1
    print("")                                    2
                                                3

def myprint(msg): # メッセージを書いて改行する  4
    print(msg, end="")                          5
    mynewline()                                 6
                                                7

def helloworld():                               8
    myprint("hello")                            9
    myprint("world")                           10

```

この実行過程の一部を図で模式的に表すと以下ようになる. C++を含む多くのプログラミング言語では, 関数を呼ぶ (call) とその関数の実行状態や local 変数を管理する **フレーム** が作られる. 下図ではフレームを枠囲みで表した.



各フレーム内で見た時に, 文は書かれた順に実行される. たとえば, 一番左の `helloworld` のフレームにおいて `myprint("hello")` の実行が完了してから `myprint("world")` の実行が開始される.

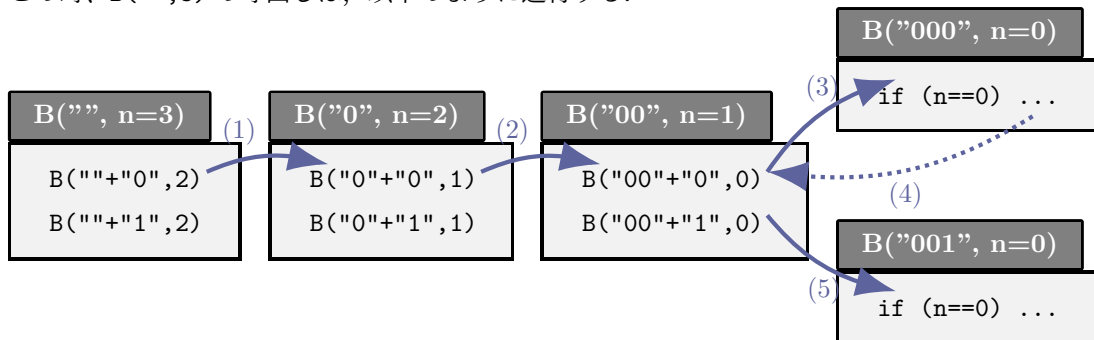
`myprint("hello")` の実行が完了するとは、`myprint` 関数の呼び出しが実行され (矢印 (1))、そのフレーム内で、全ての文の実行を終えて制御が戻る (矢印 (4)) ことである。その過程では `mynewline` 関数も実行される (矢印 (2),(3))。矢印 (3) や (4) のように、あるフレームで全ての文の実行を終えるか `return` が起こると、フレームは破棄され、呼び出し元の適切な位置に制御が戻される。

再帰の場合 再帰の場合も同様に、呼出し毎にフレームが作られる。5.3 節の `m` 進数の例題を以下のように実装したとする。

Python3

```
def B(prefix, n): 1
    if (n==0): 2
        ... 3
        return 4
    B(prefix+"0", n-1) 5
    B(prefix+"1", n-1) 6
```

この時、`B("",3)` の呼出しは、以下のように進行する。



ソースコード上は同じ関数であっても、呼出し毎に異なるフレームが作成されることに注意する。それぞれのフレームごとに、引数、ローカル変数やどこまで実行したか、などの情報を維持する。このような情報の管理のために、**スタック領域**が用いられる (データ構造のスタックとは異なる)。実行が完了したフレームは破棄されるが、完了していないフレームはメモリ上に維持する必要がある。そのため、何段も関数を呼び出すと (たとえば 100 万)、実行時エラーになる場合がある。たとえば標準演習環境では、8 メガバイトが限度である。



```
ssh0-01m:~ 0123456789$ ulimit -a 1
... 2
stack size (kbytes, -s) 8192 3
... 4
```

5.B ユークリッドの互除法と応用

例題 GCD and LCM

正の整数 a, b に対する最大公約数, 最小公倍数を求めるプログラムを作成せよ.

<http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=0005&lang=jp>

最大公約数は, **ユークリッドの互除法** (参考書 [5, pp. 107–]) で求めると良い.

正の整数 $a \geq b$ に対する最大公約数を $\gcd(a, b)$ と表記すると,

$$\gcd(a, b) = \begin{cases} a & (b = 0) \\ \gcd(b, a \% b) & (\text{otherwise}) \end{cases} \quad (5.2)$$

と定義できる.

問題 Extended Euclid Algorithm (extend_euclid)

与えられた2つの整数 a, b について $ax + by = \gcd(a, b)$ の**整数解** (x, y) を求めよ.

http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E&lang=jp

拡張ユークリッド互除法で求められる. ユークリッド互除法が行っている計算で付随して求められ情報を活用する.

問題 中国剰余定理 (chinese_remainder)

「3で割ると2余り、5で割ると3余り、7で割ると2余る数はいくつか」一般に互いに素な整数 a, b, c に対する剰余 a', b', c' に対して, 「 a で割ると a' 余り、 b で割ると b' 余り、 c で割ると c' 余る数」を一つ求める関数 `chinese_remainder` を作成せよ.

オンラインジャッジに適切な問題がなさそうなので, この問題は各自で入出力を作ってテストすること. テストしたデータをソースコードに添えること.

巨大な数を伴う計算を行う場合に, すべてを多倍長整数で計算すると計算コストが大きい. 代わりに, 複数の素数に対する剰余を計算して, 最後に復元すると計算の効率が良い. 囲碁 (19路盤) の合法局面の数²⁾は, このように求められた.

2) 208168199381979984699478633344862770286522453884530548425639456820927419612738015378525648451698519643907259916015628128546089888314427129715319317557736620397247064840935 (DOI:10.1007/978-3-319-50935-8_17)

第 6 章 木構造，複合データ (クラス)

Trees and classes

2026-05-20 Wed

概要

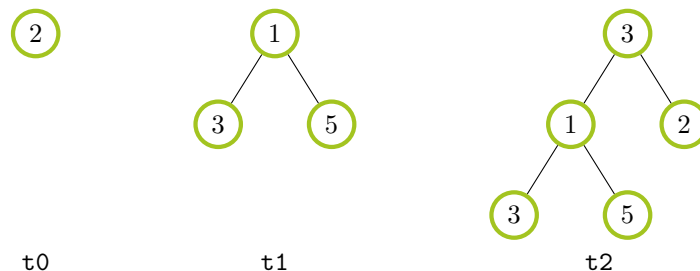
世の中のさまざまな概念は，階層的な構造を持っている．そのような概念をモデル化する技法の一つとして，木構造と，クラスを紹介する．演習時間の都合で同時に紹介するが，それぞれ独立の概念である．

6.1 階層的構造

データの中にデータがあり，その中にまた別のデータがある……というようなものを階層的なデータと言う．Python の配列は (C++や C と異なり) 異なる型の要素を持つことができるので，配列を要素とする配列という形で木構造を書いてみよう．

6.1.1 数値の二分木

例として，数値の**二分木**を考える．二分木は空の木であるか，**根**という**節点** (*node*) を持つ．各節点は数値を持ち，また左右の枝に二分木を持つ．二分木は空でも良いので，左右のどちらかあるいは両方の枝を持たない場合もある．



二分木に対する操作は，

- 構築 (`make_node`),
- 根の値の参照 (`value`),¹⁾
- 左右の部分木の参照 (`left`, `right`),
- 空の木かどうかの判定 (`is_empty`)

とする．空の木を `EmptyTree` と書く．例として `make_node(2, EmptyTree, EmptyTree)` は図の `t0` 相当

1) 注 次の項目の部分木の参照と組み合わせれば根以外の節点の値も参照できる

の木を作って返し、`left(t0)` は `EmptyTree` を返す。`make_node(1,make_leaf(3),make_leaf(5))` は図の `t1` 相当の木を作って返し、`left(t1)` は節点 3 を返す。

`make_node(3,make_node(1,make_leaf(3),make_leaf(5)),make_leaf(2))` は図の `t2` 相当の木を作って返し、`left(t2)` は `t1` 相当の木を返す。

ただし `make_leaf(x)` は、`make_node(x,EmptyTree,EmptyTree)` の略記である。

このような木を配列で表現するとすると、次のように定義できる:

Python3

```
def make_node(num, left_node, right_node): 1
    return [num, left_node, right_node] 2
def value(tree): 3
    return tree[0] 4
def left(tree): 5
    return tree[1] 6
def right(tree): 7
    return tree[2] 8

# また、特殊ケースとして空の節点を文字列 'EmptyTree' で表わすことにす 9
#   ↳   る 10
# None や [] など様々な候補がある、make_node の表現と重ならない方が良い 11
EmptyTree = "EmptyTree" 12
def is_empty(tree): 13
    return tree == EmptyTree 14

# 便利のために、子を持たない節点 (=葉) を作るための略記法を用意 15
def make_leaf(num): 16
    return make_node(num, EmptyTree, EmptyTree) 17
18
```

動かしてみよう



```
>>> t0 = make_leaf(2) 1
>>> t0 # 要素数 3 の配列: 要素はそれぞれ 2, なし, なし 2
[2, 'EmptyTree', 'EmptyTree'] 3
>>> t1 = make_node(1, make_leaf(3), make_leaf(5)) 4
>>> t1 5
[1, [3, 'EmptyTree', 'EmptyTree'], [5, 'EmptyTree', 'EmptyTree']] 6
>>> t2 = make_node(3, t1, t0) 7
>>> t2 # 要素数 3 の配列: 2 つの要素は、それ自身も配列 8
[3, [1, [3, 'EmptyTree', 'EmptyTree'], [5, 'EmptyTree', 9
↳ 'EmptyTree']], [2, 'EmptyTree', 'EmptyTree']]
>>> value(right(left(t2))) 10
5 11
```

6.1.2 木の頂点の訪問

このようにして作成された数値の二分木について、木の性質を調べる関数を作成しよう。

節点の数 木 t の節点の個数 $C(t)$ は、左右の枝の個数の合計を考えて、次のような式で表せる:

$$C(t) = \begin{cases} 0 & t \text{ が空のとき} \\ 1 + C(\text{left}(t)) + C(\text{right}(t)) & \text{それ以外} \end{cases}$$

例題 節点の個数

木 tree の節点の個数を数える関数 $\text{count_node}(\text{tree})$ を、再帰を用いて作成せよ。

実行例:

```
>>> count_node(t0) 1
1 2
>>> count_node(t1) 3
3 4
>>> count_node(t2) 5
5 6
```

回答例: これまでの再帰関数の実装と同様に、式に対応させたコードを書けば良い。

Python3

```
def count_node(t): 1
    if is_empty(t): 2
        return 0 3
    else: 4
        return 1 + count_node(left(t)) + count_node(right(t)) 5
```

例題 節点の数の合計

木 tree の各節点が表す数を合計する関数 $\text{sum_node}(\text{tree})$ を、再帰を用いて作成せよ。

木 t の節点の数の合計 $S(t)$ は次のような式で表せる:

$$S(t) = \begin{cases} 0 & t \text{ が空のとき} \\ \text{value}(t) + S(\text{left}(t)) + S(\text{right}(t)) & \text{それ以外} \end{cases}$$

実行例:

```
>>> sum_node(t0) 1
2 2
>>> sum_node(t1) 3
```

```

9
>>> sum_node(t2)
14

```

問題 節点の持つ値の最大値 (`max_value`)

木 `tree` の各節点が表示する数の最大値を関数 `max_value(tree)` を、再帰を用いて作成せよ。但し、`tree` の各節点が表示する数は正と仮定して良い。

ヒント: まずは式を作ってみよう

実行例:

```

▶
>>> max_value(t0)
2
>>> max_value(t1)
5
>>> max_value(t2)
5

```

例題 木の深さ

木 `tree` の深さ返す関数 `depth(tree)` を、再帰を用いて作成せよ。深さとは、根から各節点までの距離 (たどる辺の数) の最大値とする。

ヒント: まずは式を作ってみよう

```

▶
>>> depth(t0)
0
>>> depth(t1)
1
>>> depth(t2)
2

```

preorder, inorder, postorder 木を再帰的に訪問し節点を一度ずつ処理する順序として、*preorder* (自分, 左, 右), *inorder* (左, 自分, 右), *postorder* (左, 右, 自分) などが用いられる。

例題 行きがけ順

木 `tree` の各節点の数値を、*preorder* の順に並べた配列を返す関数、`preorder(tree)` を作成せよ。

preorder の順に並べた配列を $P(t)$ とすると、次のように定義される。

$$P(t) = \begin{cases} [] & t \text{ が空のとき} \\ [\text{value}(t)] + P(\text{left}(t)) + P(\text{right}(t)) & \text{それ以外} \end{cases}$$

ただし $+$ は配列の結合を表すとする。配列の結合方法については、3.5 節で紹介した。

実行例:

```
>>> preorder(t0) 1
[2] 2
>>> preorder(t1) 3
[1, 3, 5] 4
>>> preorder(t2) 5
[3, 1, 3, 5, 2] 6
```

問題 通りがけ順 (inorder)

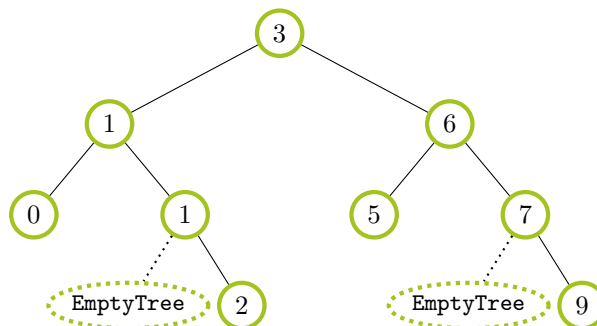
木 $tree$ の各節点の数値を、inorder の順に並べた配列を返す関数、 $\text{inorder}(tree)$ を作成せよ。

実行例:

```
>>> inorder(t0) 1
[2] 2
>>> inorder(t1) 3
[3, 1, 5] 4
>>> inorder(t2) 5
[3, 1, 5, 3, 2] 6
```

6.1.3 規則を持つ木: 二分探索木

数値の二分木の部分集合として以下の規則を満たす木を考える: 各節点の数値は、左側の枝に現われるどの数値よりも大きく、右側の枝に現われるどの数値よりも小さいまたは等しい。(これを **二分探索木条件** と呼ぶ)。



例題 節点の追加

二分探索木 `tree` と数値 `x` を与えられて、`x` の節点を `tree` に加えた新しい二分探索木を作成して返す関数 `add_node(tree, x)` を作成せよ。

注: 引数 `tree` として、二分探索木条件を満たすもののみを対象とする。また元の `tree` を変更するのではなく、`tree` の構成要素を利用して良いので、新たな木を作成せよ。ヒント: `is_node(t, x) is False`)

木 t に x を加えた二分探索木を $B(t, x)$ とすると、次のように定義可能である:

$$B(t, x) = \begin{cases} x \text{ を要素とする葉} & t \text{ が空のとき} \\ t \text{ の } \text{left}(t) \text{ を } B(\text{left}(t), x) \text{ で置き換えた木} & x < \text{value}(t) \\ t \text{ の } \text{right}(t) \text{ を } B(\text{right}(t), x) \text{ で置き換えた木} & \text{それ以外} \end{cases}$$

`make_leaf` や `make_node` を用いて、再帰的に定義せよ。

問題 二分探索木の作成 (`make_binary_search_tree`)

与えられた数値の配列 `a` の各要素を節点として持つ二分探索木を作成して返す関数 `make_binary_search_tree(a)` を作成せよ。

回答例: `EmptyTree` に順に要素を `add_node(tree, x)` する。

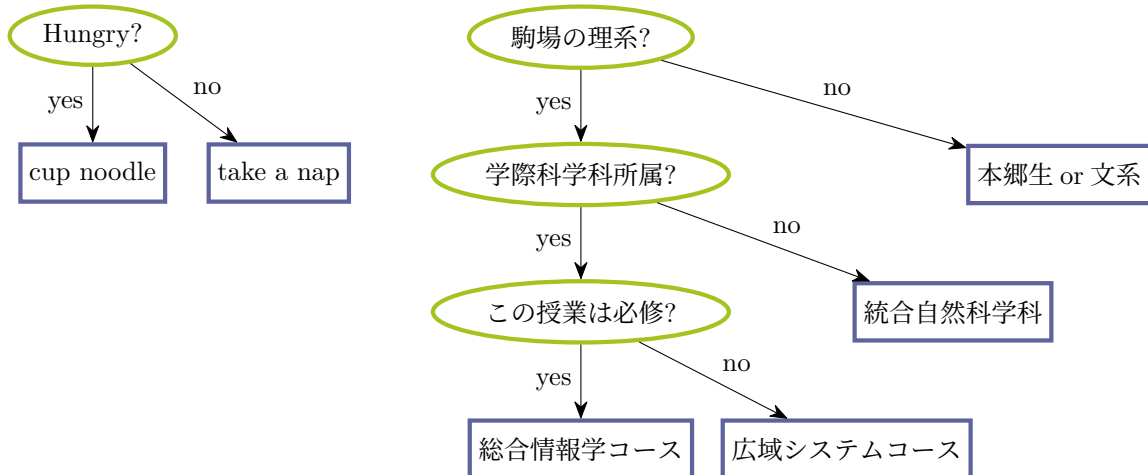
実行例

```
>>> t = make_binary_search_tree([3,1,6,1,7,9,2,0,5])      1
>>> t                                                       2
[3, [1, [0, 'EmptyTree', 'EmptyTree'], [1, 'EmptyTree', [2,   3
↪ 'EmptyTree', 'EmptyTree']]],
    [6, [5, 'EmptyTree', 'EmptyTree'], [7, 'EmptyTree', [9,   4
↪ 'EmptyTree', 'EmptyTree']]]]
>>> count_node(t)                                           5
9                                                            6
>>> inorder(t)                                              7
[0, 1, 1, 2, 3, 5, 6, 7, 9]                                  8
```

二分探索木の要素を通りがけ順 (`inorder`) に出力すると、昇順に並んでいることを確認する。

6.1.4 決定木

「はい」か「いいえ」で答えるような質問によって枝分かれをして、最終的に適当なメッセージを表示するシステムを作ろう。



システムの状態は (1) 質問中 (図の楕円) か (2) 答え (図の長方形) のどちらかである。各状態に木の節点 (node) を対応させる。質問中の状態は、(a) 質問文の文字列 (b) 「はい」と答えた場合の次の状態 (c) 「いいえ」と答えた場合の次の状態とする。答えは文字列だけを持つ。

以下のような関数があれば、状態を管理できるだろう：

- 質問中の状態に関して、ノードを作る関数 `make_question_node(question, yes_branch, no_branch)`・質問文または左右の分岐を取り出す関数 `question_text(qnode)`, `question_yes(qnode)`, `question_no(qnode)`
- 答えを作成する関数 `make_answer_node(answer)`・答えの文字列を参照する関数 `answer_text(anode)`
- 状態が (1) か (2) のどちらかを判定する `is_question_node(node)`

システムは現在の状態に応じて適切な表示を行う。(1) の場合は、問題文と「yes」「no」の質問を表示する。キーボード入力に応じて選ばれた状態の表示を行う。(2) の場合は、メッセージを表示する。これらは、次のように定義できる：

Python3

```

# 状態に応じて適切な表示を行う
def display(node):
    if is_question_node(node):
        # 質問の場合、質問文を表示して y/n をキーボードから読み込む
        print("Q: "+question_text(node)+ " ? [yn]")
        yn = input()
        if yn.lower()[0] == "y":
            display(question_yes(node))
        else:
            display(question_no(node))
    else:
        # 答えの場合、メッセージを表示する
        print("----")

```

```
print("Answer: " + answer_text(node))
```

14

問題 Q&A

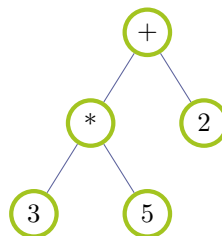
以下の必要な関数を定義し、適当な質問・答えを作り、displayに適用せよ。(関数群はテストケースを添える。適当な質問・答えの実行結果はコピーペーストする)

- 質問中の状態に関して、ノードを作る関数 `make_question_node(question, yes_branch, no_branch)`・質問文または左右の分岐を取り出す関数 `question_text(qnode)`, `question_yes(qnode)`, `question_no(qnode)`
- 答えは文字列だけを持つ。作成関数 `make_answer_node(answer)`・答えを参照する関数 `answer_text(anode)`
- 状態が (1) か (2) のどちらかを判定するための関数, `is_question_node(node)`

ヒント: 数値の二分木の定義を参考にせよ

6.1.5 四則演算の構文木

また別の種類の木として、内部節点に四則演算を表す文字、葉に整数を持つ二分木を考える。木は値を持ち、葉の値は所持する整数、内部節点の値は左右の木の値をその節点の演算で計算したものとする。なお、節点は子を持たないか (=葉), 2つの子を持つとする。以下の例で、根の値は $((3 \times 5) + 2) = 17$ である。



問題 四則演算 (calc)

四則演算を表す構文木 `tree` を与えられて、根の計算結果を返す関数 `calc(tree)` を作成せよ。



```
>>> t = make_node("+", make_node("*", make_leaf(3),
    make_leaf(5)), make_leaf(2))
>>> t
['+', ['*', [3, 'EmptyTree', 'EmptyTree'], [5, 'EmptyTree',
↳ 'EmptyTree']],
      [2, 'EmptyTree', 'EmptyTree']]
>>> calc(t)
17
```

1
2
3
4
5
6
7

ヒント: 節点 `t` が子を持つかどうかは, `is_empty(left(t))` などで判定できる.

6.1.6 内部表現の世界と使用する世界の分離

これまで木の内部表現として配列を用いてきた. 別案として, これを `dict`(連想配列) を用いて表現することもできる (詳しくは 9.2.5 節で取り扱う). 6.1.1 節で定義した `make_node`, `value`, `left`, `right` の 4 つの関数の実装を以下のように取り替えてみよう. 残りの関数はそのままでも, 二分探索木などで作成したプログラムをそのまま動かすことができる.

Python3

```
def make_node(num, left, right):
    return {"number": num, "left": left, "right": right}

def value(tree):
    return tree["number"]

def left(tree):
    return tree["left"]

def right(tree):
    return tree["right"]
```

ここでは連想配列のキーは, 文字列で表現した. 他には整数やタプルでも表現できる.

多くの場合, 表現したい内容に対して (今回は二分木), それを表現する方法は複数通りある. 内部表現次第で性能に影響する場合もあるので, 注意深く選ぶ必要がある (今回の配列と `dict` では大差ない). また, 内部表現をあとから取り替える場合は, 書き換え箇所が少なければ少ないほど望ましい. 今回の `make_node`, `value`, `left`, `right` の 4 つの関数のように, 適切に抽象化を行うことは, 書き換え箇所を抑えることにつながる. なお, Q&A のプログラムで, 内部状態を変更するには同様に, `make_question_node`, `question_text`, `question_yes`, `question_no`, `make_answer_node`, `answer_text`, `is_question_node` を書き換える.

6.2 クラスの利用の初歩

関連の深いデータをまとめる方法として, **クラス** (の機能の一部) を導入する. 保健センターなどの業務で, 複数人の身長と体重に興味があるとする. 身長も体重も数であるので, それぞれは Python で表現可能であるが, それらをまとめたデータを作りたい. 天下り式の説明であるが以下のようにクラスを用いると 2 つの変数 `height` と `weight` がセットになったデータ型を作ることができる.

Python3

```
class Person:
    def __init__(self, height, weight):
        self.height = height
        self.weight = weight
```

1 行目 `class` から 4 行目までのブロックが, `Person` と名付けられたクラスの定義である. 2 行目か

ら4行目のような、関数と似た記述は**メソッド**と呼ぶ特に`__init__`という名を持つ特別なメソッドは、初期化のために用いられる。この意味も追って説明する。

使いかたは次のようになる。



```
>>> taro = Person(180,90) 1
>>> taro 2
<__main__.Person object at 0x10542b9e8> 3
>>> taro.height 4
180 5
>>> taro.weight 6
90 7
>>> jiro = Person(160,55) 8
>>> jiro 9
<__main__.Person object at 0x10542ba90> 10
>>> jiro.height 11
160 12
```

Python で決められた構文である `Person(height, weight)` を呼ぶたびに、新しいデータの組が作られる (**インスタンス**と呼ぶ)。それらは `height` と `weight` で参照可能なデータの組を持つ。名称が同じ `height` であっても、インスタンスが異なれば、すなわち `taro` と `jiro` では異なる値を持つ。それぞれが持つ値は `Person(180, 90)` のように `Person` に与えた引数に対応し、`__init__` という特別な名前を持つメソッドにより内部変数 `height` と `weight` にコピーされる。

クラスに、メソッドを追加することで様々な機能を実現できる。たとえば `bmi` を計算する機能を付与してみよう。

Python3

```
class Person: 1
    # 以下を追加 2
    def bmi(self): 3
        return self.weight / (self.height/100.0)**2 4
```

実行例:



```
>>> taro.bmi() 1
27.777777777777775 2
>>> jiro.bmi() 3
21.484374999999996 4
```

ここまでの資料では二分木を、内部表現として配列あるいは `dict` を使って実現した。この内部表現にクラスを用いて実現してみよう。

問題 クラスを用いた木 (Node)

木の節点を表すクラス Node を適切に定義せよ。

動作確認として、6.1.1 節で定義した木の機能を次のように変更しても、二分探索木または四則演算が Node を対象に動作することを確認せよ。

```
def make_node(num, left, right): 1
    return Node(num, left, right) 2
3
def value(tree): 4
    return tree.value 5
6
def left(tree): 7
    return tree.left 8
9
def right(tree): 10
    return tree.right 11
```

注: EmptyTree や is_empty の定義は変更不要である。また上記は (メソッドではなく) 通常の間数である。

6.3 発展問題

問題 Reconstruction of a Tree 改題★ (recover)

ある二分木について異なる頂点には異なるラベルがついているとする。preorder で出力した頂点のリストと, inorder で出力したリストを与えられて、元の木を復元して返す関数 recover(preorder, inorder) を作成せよ。

(★付き問題にもテストケースも添えること。)

原題: http://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_7_D

ヒント: 部分木の頂点を求めて分割統治しよう

問題 構文解析★ (parse)

四則演算を表す文字列 s に対応する木を返す関数 parse(s) を作成せよ。

簡単のため文字列 s は、1 以上 100 未満の整数と四則演算と括弧のみを用いた演算を表すとする。また演算子を用いる際には必ず括弧が用いられるとする。すなわち "1+2+3" は与えられない。"((1+2)+3)" または "(1+(2+3))" が与えられる。

問題 二分探索木 (親の管理)* (successor)

「クラスを用いた木」について、Node の親を管理するフィールド `self.parent` を追加し、親を得る関数 `parent(tree)` を定義せよ。また `make_node` を適切に変更せよ。そのうえで、ある二分探索木 `tree` の任意の節点 `node` について、「次」の節点を返す `successor(node)` を定義せよ。「次」とは、二分探索木の節点を通りがけ順 (inorder) に並べた際の順序で考える。次の節点が存在しない場合は `EmptyTree` を返すとする。

ヒント: 右の節点を持つかどうかで場合分けして考えよう。

(オプション: 二分探索木 `tree` の任意の節点 `node` について、節点 `node` を削除した木を作る関数 `delete(tree,node)` を作成せよ。削除後の木も二分探索木条件を満たすようにすること)

問題 Q&A-C++*

「Q&A」または「二分探索木 (親の管理)」を C++ (C++11) で作成せよ。

節点を表す Node クラスを作成し、左右の枝としての c++11 の `shared_ptr<Node>` (Node 型のスマートポインタ) を持つものとする。`shared_ptr<Node>` については、8 章 (8.5.1 節) も参照。

6.4 この章の提出課題

以下のいずれかで合格:

- 問題 (タイトル部分に * なし) を全て Python で解く (通常コース)
ただし、「節点の持つ値の最大値」は提出不要
- 星付きの問題 2 題以上解く (発展コース)
ただし、一題は「クラスを用いた木」でも良い

提出条件:

- 各関数に対応するテストケースが記述されていること
- ループを含むコードは、ループ不変条件が明確にコメントで記述されていること
- 冗長なコードがないか確認し、標準的な書き方に整えられていること
- 指定した課題が、指定した関数名で作成されていること
- 提出ファイルは、2.A 節を参考にして、*jupyter から作成した* weekN-(学生証番号).ipynb とする
- 日本語 (または英語) でコメントが付記されていること: { 授業中に既にフィードバックを得ていればそのスタッフの名前, 学んだこと, (あれば) 他に参考にした資料や他の人の回答など、(途中で提出する場合は何がうまくゆかずに困っているなど) } をコメントで盛り込む。何も苦労がなかった場合を除き、自作したことが分かるだけの十分なエピソードを記述すること — 授業中にスタッフのコメントを得ている場合は簡潔で良い

第7章 状態管理，抽象インターフェースと継承

internal state, abstract interface and inheritance

2026-05-27 Wed

概要

先週は，value, left と right のような組となるデータをまとめて扱うための道具としてクラスとオブジェクトの概念を導入した．今週は，オブジェクトの内部状態の変化も含めて，再度クラスとオブジェクトについて演習する．インターフェースを限定することで，複雑なプログラムを管理しやすくする点が，先週から続く主題である．

身に付ける目標は，クラスとオブジェクト (インスタンス)，継承と合成，関数にオブジェクトを渡す際のインターフェースの考え方などである．

7.1 内部状態を持つオブジェクト

7.1.1 内部状態

オブジェクトの集合によって実世界をモデル化する場合には，個々のオブジェクトは *identity* を保ちながら，**内部状態**が変化してゆくと考える．例えば，銀行口座をオブジェクトとする場合，同じ口座という *identity* を保ったまま残高が変化してゆく．

例題 銀行口座

銀行口座を表すクラス `BankAccount` を作成せよ．口座作成時の金額 (たとえば1万円) を与えて `BankAccount(10000)` と口座に相当するオブジェクトを作成する．そのように作成したオブジェクトに対してメソッド `withdraw(x)` を呼ぶと口座から x 円引き出して，残高を返すものとする．もし残高よりも沢山引き出そうとした場合は “insufficient funds” というメッセージを返す．

はじめにプロトタイプとして，以下の実装例の `BankAccount0` について考えてみる．(あとで，改良した `BankAccount` を導入する)

Python3

```
class BankAccount0:
    def __init__(self, initial_balance):
        self.balance = initial_balance
```

```

def withdraw(self, amount):
    if self.balance >= amount:
        self.balance -= amount
        return self.balance
    else:
        return "Insufficient funds"

```

1 行目 class から 10 行目までのブロックが、BankAccount と名付けられたクラスの定義である。2 行目から 3 行目、5 行目から 10 行目では、それぞれ__init__と withdraw という二つのメソッドを定義する。

6.2 節で紹介したように、インスタンス作成時に、BankAccount0 で与えた変数で、__init__が呼ばれ、ここでは変数 self.balance が初期設定される。

このクラスを用いて、BankAccount(初期残高) が実行される度に新しく口座ができる。BankAccount で新しいインスタンス (オブジェクト) が作成され、その度に、__init__が一度呼ばれる。その引数は、BankAccount に与えられた引数 (上記の例では 1 万) である。実行例 3 行目の withdraw のように、メソッドを実行する際には、インスタンス (この例では a や b) にドットで続けて記述する。メソッド内で使われている self.balance はインスタンス変数と呼ばれる特殊な変数で、BankAccount(初期残高) して作られたインスタンス毎に (この例では a や b では) 異なる値を持つ。引数 self が、操作しているインスタンス a や b を指す。

使用例:

```

>>> a = BankAccount0(10000) # a さんの口座を一つつくる
>>> b = BankAccount0(10000) # b さんの口座を一つつくる
>>> a.withdraw(5000)
5000
>>> b.withdraw(7000)
3000
>>> b.withdraw(4000)
"Insufficient funds"
>>> a.withdraw(4000)
1000

```

この BankAccount0 クラスについて、withdraw メソッドが意図通りに動作することを確認する。

7.1.2 カプセル化

銀行口座の例の balance のように、インスタンス変数の一部は、非公開であることが望ましい場合がある。非公開とは、対応するメソッド (withdraw など) のみから変数を操作可能であること、つまり (特に裏口を作らなければ) 外部から直接操作することはできないことを指す。内部状態を外部から触れなくすることを一般に「カプセル化 (encapsulation)」と言う。



```
>>> a.balance += 1000000000 # 違法ハッキング(?) で財産拡大
(成功してしまった)
```

1
2

現状の BankAccount0 は、このようなコードも警告なしに実行してしまう。すなわちカプセル化は行われていない。Python では、プログラムの設計者が、カプセル化の意図を使用者に伝える方法として、2つのアンダースコア `__` をつける方法がある。¹⁾

Python3

```
class BankAccount:
    def __init__(self, initial_balance):
        self.__balance = initial_balance

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
            return self.__balance
        else:
            return "Insufficient funds"
```

1
2
3
4
5
6
7
8
9
10

このようにすると、先程の操作がエラーになることを確認せよ。



```
>>> a.__balance += 1000000000
(エラー)
```

1
2

これは Python が 2つのアンダースコア `__` を別の名前に付け替えるためである。この文法は厳密なアクセス制限とは異なり、実際には `a.BankAccount__balance += 10000` とすると成功する。しかし、意図しない変数の破壊を防ぐという意味では十分であるため、本演習では、今後カプセル化する変数は、名前に 2つのアンダースコア `__` をつける方針を採用する。なお、アンダースコア 1つのみを用いる方法を推奨する主張もある。²⁾

引き出ししかできない口座だと、残高は減る一方なので、預金もできるようにする。そのためには以下のように `deposit(amount)` メソッドを定義する。

Python3

```
# class BankAccount:
# 先ほどのセル内のクラス定義に以下の行を追加
def deposit(self, amount):
    self.__balance += amount
```

1
2
3
4

使用例

1) (double underscores を略して *dunder* と呼ばれる)

2) <https://www.python.org/dev/peps/pep-0008/>



```

>>> a = BankAccount(10000) 1
<__main__.BankAccount object at 0x10542be80> 2
>>> a.withdraw(5000) 3
5000 4
>>> a.withdraw(6000) 5
"Insufficient funds" 6
>>> a.deposit(4000) 7
9000 8
>>> a.withdraw(6000) 9
3000 10

```

例題 暗証番号付き口座

クラス `BankAccount` に、暗証番号を付け加えてみよ。

- `initialize` において、残高の初期値に加えて、暗証番号に相当する文字列を受け取って新しい口座を作る
- さらに、`withdraw`, `deposit` の手続きには、引き出し/預金額に加えて暗証番号を受け取り、口座を作ったときと同じ暗証番号であった場合のみ操作ができるようにする
- 暗証番号が間違っていた場合は、“Incorrect password” というメッセージを返すものとする

実行例:



```

>>> acc = BankAccount2(10000, "secret") 1
<__main__.BankAccount2 object at 0x10542bef0> 2
>>> acc.withdraw(4000, "secret") 3
6000 4
>>> acc.deposit(5000, "some-other-password") 5
"Incorrect password" 6

```

例題 カウンター

次のような数を数える機能を持つクラス `Counter` を作成せよ。`Counter` によって作成される。初期値は 0 とする。メソッド `increment` で数を 1 増やす。メソッド `counter` で現在の数を返す。

実行例



```

>>> c = Counter() 1
>>> c.increment() 2
1 3

```

```

>>> c.count()
1
>>> c.increment()
2
>>> c.count()
2

```

実装例:

Python3

```

class Counter:
    def __init__(self):
        self.__counter = 0

    def increment(self):
        self.__counter += 1

    def count(self):
        return self.__counter

```

まず、数を数える変数をカプセル化する、と判断したとする。つまりカウンターの値を変更するには、`increment` メソッドを呼ぶ必要がある、という設計を採用した。そこで変数の名前を、現在の数を返すメソッド `count` に合わせて、`__counter` とする。new の際に `__init__` が呼ばれるというルールを思い出して、そこで初期値に初期化する。続いて、メソッド `increment` と `count` を定義する。

問題 時計 (ClockTimer)

時計の時を刻むクラス `ClockTimer` を定義せよ。このクラスは、次のようなインターフェイスを持っている:

- `ClockTimer(hour, minute, second)` によって、hour 時 minute 分 second 秒を表わす時計が作られる。
- インスタンス変数 (あるいはプロパティ) `hour`, `minute`, `second` によって、その時計が表わしている時刻が得られる。
- メソッド `tick` によって 1 秒時刻が進む。



```

>>> timer = ClockTimer(22, 54, 14)
>>> timer.hour
22
>>> timer.minute
54
>>> timer.second
14

```



```

>>> timer.tick()                                     8
15                                                    9
>>> timer.hour                                       10
22                                                    11
>>> timer.minute                                     12
54                                                    13
>>> timer.second                                     14
15                                                    15
>>> for _ in range(45): # 45 回実行                16
...     timer.tick()                                17
...                                                  18
>>> timer.hour                                       19
22                                                    20
>>> timer.minute                                     21
55                                                    22
>>> timer.second                                     23
0                                                    24

```

今回は、時刻を表す状態変数 `hour`, `minute`, `second` について、カプセル化しないと判断したとする。すなわち、時刻を変更する方法としては、メソッド `tick` を呼ぶことと (時が 1 秒進んだ状況に相当)、変数に直接代入する (時計の針を無理やり動かした状況に相当) 方法の、両方を想定する。

7.1.3 メソッドの再帰呼出し

クラスでモデル化するものは、必ずしも実世界のモノに対応する必要はない。ここでは、(恣意的だが) フィボナッチ数を計算する計算主体というモデルを考えてみよう。

Python3

```

class Fib:                                           1
    def fib(self, n):                                2
        if n == 0:                                   3
            return 0                                  4
        elif n == 1:                                 5
            return 1                                  6
        else:                                         7
            return self.fib(n-1)+self.fib(n-2)        8

```



```

>>> f = Fib()                                       1
>>> f.fib(3)                                        2
2                                                    3
>>> f.fib(4)                                        4
3                                                    5
>>> f.fib(10)                                       6
55                                                    7
>>> f.fib(20)                                       8

```

6765

9

例題 呼び出し回数測定

上記の Fib クラスを拡張して、メソッド fib が何回呼ばれたかを測定する機能をもたせよ。またメソッド count によってその回数分かるようにせよ。

道具として Counter クラスを組み合わせると簡単に実現することができる。このようにあるクラスの部品として別のクラスを用いることを *composition* と言う。

Python3

```

class Fib:
    def __init__(self):
        self.__counter = Counter()

    def fib(self, n):
        self.__counter.increment()
        if n == 0:
            return 0
        elif n == 1:
            return 1
        else:
            return self.fib(n-1)+self.fib(n-2)

    def count(self):
        return self.__counter.count()

```

使用例:



```

>>> g = Fib()
>>> g.fib(3)
2
>>> g.count()
5
>>> g.fib(10)
55
>>> g.count()
182
>>> h = Fib()
>>> h.count()
0

```

問題 メモ化 (MemoFib)

これまでのフィボナッチ数の計算はメソッド `fib(n)` が同じ `n` に対して何度も呼ばれるため、計算効率が悪い。改善のための一つの方法は、配列 `memo` を用意し、`fib(n)` を計算するたびに `memo[n]` に答えを記憶し、次に `fib(n)` を計算する際に `memo[n]` に答えが既にあれば (あらためて `fib(n)` を計算する代わりに) `memo[n]` の値を使うという方法である。そのように計算を行うクラス `MemoFib` を作成せよ。なお簡単のため `n` は 1000 未満と仮定して良い。

このような工夫を一般に **メモ化** (*memoization*) と呼ぶ。

実行例 (memo の参照はカウントしないとする):

```
>>> m.fib(10) 1
55 2
>>> m.count() 3
11 4
>>> m.fib(20) 5
6765 6
>>> m.count() 7
21 8
>>> m.fib(20) 9
6765 10
>>> m.count() 11
21 12
```

実装例 (主要部分):

```
Python3
def fib(self, n): 1
    if ... # self.__memo[n] に答えが入っていなかったら 2
        self.counter.increment() 3
        ... # 答えを計算して self.__memo[n] に代入 4
    return self.__memo[n] 5
```

`self.memo[n]` は、`self.fib(n)` の計算結果と区別できる値で初期化しておく。

7.1.4 状態と複製

クラスのオブジェクト、たとえば銀行口座を表す変数は「参照」である (何回か前の演習の 4.A 節などを参照のこと)。したがって、`b=a` のように複製しても口座自体が複製されるわけではなく、口座にアクセスするカードや通帳に相当する概念がコピーされるにすぎない。したがって以下のように、複製された `b` を通じて引き落とせば、元の `a` にも影響が及ぶ。



```

>>> a = BankAccount(10000) 1
>>> a 2
<__main__.BankAccount object at 0x103ec8588> 3
>>> a.withdraw(3000) 4
7000 5
>>> b = a 6
>>> b 7
<__main__.BankAccount object at 0x103ec8588> 8
>>> b.withdraw(7000) 9
0 10
>>> a.withdraw(1) 11
'Insufficient funds' 12

```

例題 Copy

銀行口座の例について、口座ごと複製するメソッド `clone` を実装せよ。残高 5000 円の口座を `clone` すると新たに残高 5000 円の口座ができて、それぞれから 5000 円づつ下ろせるものとする。

使用例:



```

>>> a = BankAccount(10000) 1
>>> a 2
<__main__.BankAccount object at 0x103ec8588> 3
>>> b = a.clone() 4
>>> b 5
<__main__.BankAccount object at 0x103ec85c0> 6
>>> b.withdraw(3000) 7
7000 8
>>> a.withdraw(9000) 9
1000 10

```

実装例:

Python3

```

class BankAccount: 1
    def clone(self): 2
        return BankAccount(self.__balance) 3

```

自分と同じ状態 (`self.__balance`) を持ったオブジェクトを新しく生成すれば良い。

問題 変更可能な二分探索木 (BSTree) (optional)

先週作成した二分木のクラスを参考に、変更可能な二分探索木のクラス BSTree を用意せよ。さらに、これらを用いて、自分自身が二分探索木であるときに値を追加するメソッド add(x) を作成せよ。

空の木である EmptyTree や is_empty の定義は適当に変更して良い。

オプション: 二分探索木 tree の任意の節点 node について、節点 node を削除した木を作る関数 tree.delete(node) を作成せよ。削除後の木も二分探索木条件を満たすようにすること。node の子供の数が 0 または 1 の時は、node の代わりにその子を上につなげれば良い。node が子を 2 つ持つ時は、node の successor を元の木から取り除いて、node と差し替える。successor は性質上、子をたかだか一つ持つ。なお、successor とは、前の週の発展の節にある問題を参照のこと。

注: 先週作成した add_node(tree, x) は元の木 tree を一切変更せずに新しい木を作成した。今回のような変更を行う場合は影響範囲を慎重に把握する必要がある。次の問題も参照。

使用例: 以下の例では self.value=None, self.left=None, self.right=None であるような節点を EmptyTree と定義しなおしている。

```
>>> a = BSTree(None, None, None) 1
>>> a.add(3) 2
>>> for x in [1, 6, 1, 7, 9, 2, 0, 5]: 3
...     a.add(x) 4
... 5
>>> a.inorder() 6
[0, 1, 1, 2, 3, 5, 6, 7, 9] 7
```

問題 Deep Copy★

前問で作成した木のクラスに、木全体のコピーを行うメソッド clone を (copy.deepcopy を使わずに) 用意せよ。

clone 元の木にどのような変更 (左や右の部分木に対する変更も含む) を行っても、clone 後の木が変化しないように作成すること。

(★がついているようにそれほど簡単ではない。テストの方法も含めて検討せよ)

7.1.5 クラスの図表現

ソフトウェアを作る際に、設計を議論する段階では、具体的なソースコードで表現するよりも抽象化した図を用いる方が見通しが良いことが多い。クラスの図表現として広く使われている記法では、四角でクラスを表す。そして、3 つに区切った領域にそれぞれ、クラス名、そのクラスのオブジェクトが持つ属性 (インスタンス変数)、そのクラスのオブジェクトで行える操作 (メソッド) を記入する。

BankAccount
balance password
withdraw(amount, password) deposit(amount, password)

詳しくは *UML* や **クラス図** などのキーワードで調査してみよう。

7.2 インターフェース

オセロやすごろくなどのゲームをプレイするプレイヤを考える。複数のプレイヤにはそれぞれ個性があり、同じ状況でも異なる選択肢を選択するが、同じゲームをプレイするとする。このような場合にはゲームをプレイするのに必要なプレイヤの機能 (の外界とのやりとりの部分) を **インターフェース** として抽出し定義すると、見通し良く設計することができる。

7.2.1 囚人のジレンマゲーム

ここでは囚人のジレンマゲームを例として考える。このゲームには二人のプレイヤが登場し、各プレイヤは **協調** か **裏切り** のどちらかを選択し、ゲームマスターに提出する。ゲームマスターはその二人の選択の組み合わせに従って決まる **報酬** をそれぞれに与える。

	協調	裏切り
協調	2, 2	0, 3
裏切り	3, 0	1, 1

両者が協調すると、両者とも報酬があり両者の報酬の和が最大となる。しかし片方が裏切ると、裏切った方はすべての可能性の中で最大の報酬を得るが、裏切られた方の報酬はない。両者が裏切った場合は、一方的に裏切られるよりは良かったが、両者の報酬の和は 4 通りの組み合わせの中で最低となる。

7.2.2 ゲームとプレイヤのモデル化

まず、プレイヤの行動の選択肢と報酬を定義する。プレイヤの行動は協調か裏切りの 2 種類なので、ここでは整数 0 と 1 で表すとして、Cooperate と Defect と名づけた定数を設ける。それらに対応するように、二人のとった選択肢から報酬を返す `Reward[自分の選択][相手の選択]` という二次元配列を定義する。また、ゲームのルールに則った行動かどうかを検証する `valid_action` という関数を用意しておく。

Python3

```

Cooperate = 0
Defect = 1
def valid_action(act):
    return act == Cooperate or act == Defect

```

```
Reward = [
    [ 2, 0 ], # 自分が協調した
    [ 3, 1 ], # 自分が裏切った
]
```

プレイヤーにはどのような機能があれば良いだろうか？ まずは、協調か裏切りかを返すメソッド `play` が必須である。さらに結果を集計する便宜を考えて、名前を返すメソッド `name` を持たせる。

例として次の3種類のプレイヤーを考える：必ず協調する `CooperatePlayer`、必ず裏切る `DefectPlayer`、ランダムに決める `RandomPlayer` である。どのクラスも `name` と `play` というメソッドを持っていることに注意する。

CooperatePlayer	DefectPlayer	RandomPlayer
name() play()	name() play()	name() play()

Python3

```
import random
class CooperatePlayer:
    def name(self):
        return "CooperatePlayer"

    def play(self):
        return Cooperate

class DefectPlayer:
    def name(self):
        return "DefectPlayer"

    def play(self):
        return Defect

class RandomPlayer:
    def name(self):
        return "RandomPlayer"

    def play(self):
        return Cooperate if (random.randrange(2) == 0) else Defect
```

続いてこれらのプレイヤー同士を戦わせる関数 `play_one_game` を定義する。引数の `player_a` と `player_b` は、どちらも、先に定義した `CooperatePlayer`、`DefectPlayer`、`RandomPlayer` のどれかであると仮定する。

Python3

```

def play_one_game(player_a, player_b):
    act_a = player_a.play()
    act_b = player_b.play()
    if not valid_action(act_a): # 行動がルールに従っているかを確認
        raise ValueError
    if not valid_action(act_b): # 行動がルールに従っているかを確認
        raise ValueError
    reward_a = Reward[act_a][act_b]
    reward_b = Reward[act_b][act_a]

    print(player_a.name(), " v.s. ", player_b.name()) # 結果を表示
    print(" actions: ", act_a, act_b)
    print(" rewards: ", reward_a, reward_b)

    return act_a, reward_a, act_b, reward_b

```

2,3 行目で両プレイヤーの行動を受け取り, 6,7 行目でそれぞれの報酬に変換し, 9-11 行目で結果を表示し, 13 行目の報酬のペアを関数の結果とする. ここで 6,7 行目は例外的なケースの確認を行うもので, プレイヤがルール通りでない行動をした場合 (つまり協調でも裏切りでもない行動を選んだ) に, 「例外」として実行を停止する³⁾. `raise` は Python で例外を送出するための言語機能である.

実行例は以下のようになる. `CooperatePlayer` と `DefectPlayer` は毎回同じ行動をとるが, `RandomPlayer` の行動は異なる場合がある.



```

>>> pc = CooperatePlayer()
>>> pd = DefectPlayer()
>>> pr = RandomPlayer()

>>> play_one_game(pc, pd)
CooperatePlayer v.s. DefectPlayer
actions: 0 1
rewards: 0 3
(0, 0, 1, 3)
>>> play_one_game(pc, pr)
CooperatePlayer v.s. RandomPlayer
actions: 0 0
rewards: 2 2
(0, 2, 0, 2)
>>> play_one_game(pd, pr)
DefectPlayer v.s. RandomPlayer
actions: 1 1
rewards: 1 1
(1, 1, 1, 1)

```

3) 複数の作者のプレイヤーが競うような状況では, もしかするとバグを期待してルールの穴をつくプレイヤーがいるかもしれない? 演習では, 何かを大きく間違えた時に早く気づくことができるというメリットがある.

関数 `play_one_game` に 3 種類のどのプレイヤーを渡しても動くことに注目されたい。仮にもし、対戦するプレイヤーの組み合わせごとに `play_one_game_random_versus_cooperate` のように別々の関数を作る必要があるとしたら、耐えられない手間となってしまう。この例では、3 種類のプレイヤーがどれも共通に `name` と `play` という一字一句同じ名前のメソッドを持っている点が重要である。外から見て(使う立場で)オブジェクトがどのような振る舞いをするか(≈ どの名前のメソッドを呼べるか)を、インターフェースと呼ぶ。3 種類のプレイヤーは共通のインターフェースを持っている。特定のインターフェースをクラスに持たせる方法は、プログラミング言語によって異なる。

7.2.3 繰り返し囚人のジレンマゲーム

ゲームを少し拡張して繰り返し囚人のジレンマゲームを作ってみよう。このゲームでは、同じプレイヤー同士で、囚人のジレンマゲームを繰り返して行う。もし相手が常に裏切るプレイヤーならば協調するメリットはないが、協調も視野にいれた相手であればこちらも協調を選択肢に含めるほうが (1,1) よりも (2,2) の報酬をそれぞれ得られて得になる場合がある。もちろん相手が協調すると分かっていたら自分は裏切ったほうが得である。ゲームを繰り返すことで相手の出方を測る要素が生まれる。何回繰り返すかは固定とせず、各ターン終了後に、予め決められた確率 p でゲームは終了する (確率 $1 - p$ で次のターンに続く) とする。

7.2.4 インターフェースの拡張と継承

ゲームを繰り返しながら相手の戦略に対応するためには、各回で相手が選んだ選択肢を知っておく必要がある。そこでインターフェースを拡張して、各プレイヤーは `update(my_action, op_action)` というメソッドの呼び出しを通じて、結果を受け取るとする。自分の選択が `my_action` で相手の選択が `op_action` である。(自分の選択は知っているはずなので厳密には不要であるが、対称性のためこのようにしておく。なお自分の行動が確率的に決まるようなゲームでは、自分の結果を知ることが必須となる。)

これに合わせて、`play_one_game` も次のように変更する。

Python3

```
def play_one_game(player_a, player_b):
    act_a = player_a.play()
    act_b = player_b.play()
    if not valid_action(act_a): # 行動がルールに従っているかを確認
        raise ValueError
    if not valid_action(act_b): # 行動がルールに従っているかを確認
        raise ValueError
    reward_a = Reward[act_a][act_b]
    reward_b = Reward[act_b][act_a]

    player_a.update(act_a, act_b)
    player_b.update(act_b, act_a) # 2行で引数の順序が異なることに注意
```

```
# print による表示は、ゲームを繰り返す場合は冗長なので除しておく 15
16
return act_a, reward_a, act_b, reward_b 17
```

今までのプレイヤーは相手が誰でも気にしない (行動を変えない) ので, `update(my_action, op_action)` の動作は空で良い. しかしメソッド自体は定義しないと, 実行時に上記 9 行目や 10 行目でエラーとなってしまふ.

3つのクラス全てにメソッドを定義して回ることもできるが, 別の手段として**継承** (*inheritance*) を紹介する. まず, プレイヤの雛形として `Player` クラスを作る. このクラスは `name` や `play` が適切な値を返さないの, 実際にゲームをプレイすることはできない.

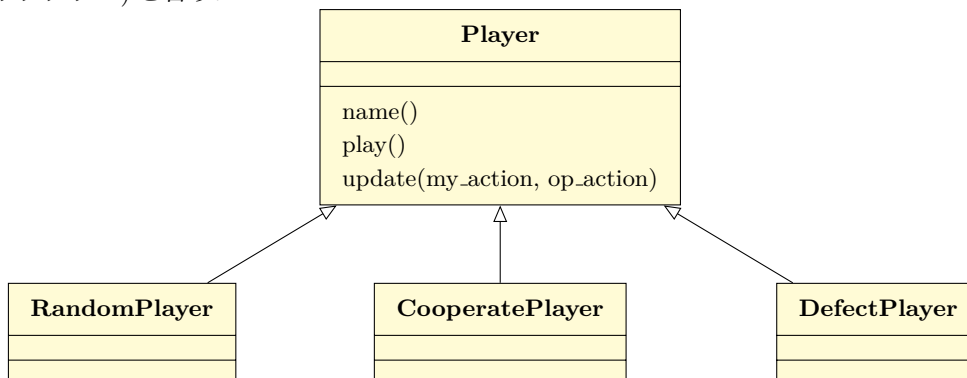
Python3

```
class Player: 1
    def name(self): 2
        pass 3

    def play(self): 4
        pass 5

    def update(self, my_action, op_action): 6
        pass 7
8
9
```

つづいて, 既存の三つのプレイヤークラスが `Player` クラスを継承するようにする. これにより, `Player` 内で定義したメソッド `update` を各クラスで継承して, 使えるようにする. クラス図で, 継承は上下の矢印で描く. 継承元のクラスを親クラス (またはベースクラス), 継承先のクラスを子クラス (サブクラス) と言う.



Python で親クラスを指定する方法は, クラス定義のクラス名に括弧を続けて親クラス名を指定する. 赤字部分のあたりが変更点である. ここでは `CooperatePlayer` の定義を示すが, `DefectPlayer` も `RandomPlayer` も同様である.

Python3

```

class CooperatePlayer(Player):
    def name(self):
        return "CooperatePlayer"

    def play(self):
        return Cooperate

```

親クラスのメソッドは継承されるので update は各クラスで、親クラスのものが使われる。一方、name と play は子クラスで定義されているので、そちらの新しい内容が使われる。これを **上書き (overriding)** という。

以上を総合して、繰り返しゲームを行う関数 play_games は次のように作ることができる。

Python3

```

P = 1.0/8
def play_games(player_a, player_b):
    sum = [0, 0] # a と b の得点
    history = ["", ""] # a と b の選択を文字列にして保存
    while True:
        act_a, reward_a, act_b, reward_b = play_one_game(player_a,
            ↪ player_b)
        sum[0] += reward_a
        sum[1] += reward_b
        history[0] += str(act_a)
        history[1] += str(act_b)
        if random.random() < P: # 終了判定
            break

    print(sum, history)

```

実行例: ランダムプレイヤはたまに裏切るのでずっと協調するプレイヤよりも高得点となる。何回繰り返すかは確率的に決まるので、実行によって異なる。

Python3

```

>>> play_games(pc, pr)
[10, 31], ['000000000000', '110111000011']

```

例題 必ず前回と異なる選択をする

必ず前回と異なる選択をするプレイヤ AlternatePlayer 作成せよ。

ヒント: last_action のようなインスタンス変数を設けて、update で渡される自分の指し手を記録する。それを play 時に参考にする。

問題 tit-for-tat (TitForTatPlayer)

繰り返し囚人のジレンマゲームでは、tit-for-tat が有力な戦略とされている。この戦略では、基本は協調するが、相手が裏切った場合は次のターンで自分も裏切る。

この戦略を用いる TitForTatPlayer を作成し、今までに作ったプレイヤー 4 種類に対して、それぞれどのように振る舞うかを観察せよ。

なおこの問題の主旨は、クラスの理解も兼ねているので、資料の流れに沿って実装すること。

テスト部分は概ね次のようになるだろう。

Python3

```
def test_tftplayer():
    player = TitForTatPlayer()
    act = player.play()
    assert Cooperate == act # 初めは協調

    player.update(Cooperate, Cooperate)
    act = player.play()
    assert Cooperate == act # 協調同士なら協調

    player.update(Cooperate, Defect) # 裏切られたら
    act = player.play()
    assert Defect == act # 次に裏切る
    ...
    # 裏切り同士など、色々な組み合わせを試す
    ...
```

なお、play_games にはインスタンスを生成した直後の TitForTatPlayer を与えると良い。同じインスタンスで何度も play_games を行う場合は、前の対戦の履歴が残っているため、最初が協調にならないことがある。

7.3 この章の提出課題

以下のいずれかで合格:

- 問題 (タイトル部分に ★ や optional の表記なし) を全て Python で解く
- tit-for-tat と問題 (タイトル部分に ★ や optional を含む) を 1 題以上解く (発展コース)

提出条件:

- 各関数に対応するテストケースが記述されていること
- ループを含むコードは、ループ不変条件が明確にコメントで記述されていること
- 冗長なコードがないか確認し、標準的な書き方に整えられていること
- 指定した課題が、指定した関数名で作成されていること

- 提出ファイルは、2.A 節を参考にして、*jupyter* から作成した `weekN-(学生証番号).ipynb` とする
- 日本語 (または英語) でコメントが付記されていること: { 授業中に既にフィードバックを得ていればそのスタッフの名前, 学んだこと, (あれば) 他に参考にした資料や他の人の回答など、(途中で提出する場合は何がうまくゆかずに困っているなど)} をコメントで盛り込む. 何も苦労がなかった場合を除き, 自作したことが分かるだけの十分なエピソードを記述すること — 授業中にスタッフのコメントを得ている場合は簡潔で良い

第8章 クラスライブラリ・デザインパターン

2026-06-10 Wed

(6月10日は栃木実習のため自由演習) 通常授業

概要

クラスの継承関係を利用したクラスライブラリの理解方法，自分でクラス群を設計する際に指針となるデザインパターン (の一つ) を紹介する．さらに，C++でのクラスの扱いについて触れる．今週の資料はページ数が長いが，C++の問題はオプションとしたため，必修の問題は3題のみである．また，今週からテストケースの作成を簡略化し，pytest 形式の代わりに後述する doctest 形式を用いる．

8.1 クラスとオブジェクト

Python3

```

class BankAccount:
    def __init__(self, initial_balance):
        self.__balance = initial_balance

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
            return self.__balance
        else:
            return "Insufficient funds"

    def deposit(self, amount):
        # ...

```

BankAccount
__balance
withdraw(amount) deposit(amount)

クラスとは属性と操作を同時に定義した設計図であり，それを元にオブジェクト (インスタンス) が作られる．銀行口座の例では，銀行口座のクラスを `BankAccount` と定義して，個々の口座を `BankAccount(initial_balance)` することで作成した．

文法 Python のクラス定義で用いられる文法をいくつかまとめる．

- `class` から始まるブロックでクラスを定義する
- クラス定義内で `self.` で修飾した変数 (たとえば `self.__balance` はインスタンス変数 (C++ではメンバ変数) である．それらは，通常の変数に似ているが，オブジェクトごとに別の変数と扱われる (口座ごとに `self.__balance` の値は異なる)．またオブジェクトと同じ寿命を持つ (通常の変数

は、一度関数を出ると消える)。

- クラス定義内では、通常の間数定義と同様の文法で `def` からのブロックでメソッド (C++ではメンバ関数) を定義する。メソッド内では、インスタンス変数を共通に利用可能である。メソッド呼び出しは関数呼び出しと同様だが、`a.deposit(100)` のように、(レシーバ) オブジェクトにドットを続けて実行する点異なる。これにより、どの銀行口座を操作するかを指定する。
- `__init__` という特別なメソッドは、クラスと同名のメソッドを呼んでオブジェクトを生成した時に (たとえば `BankAccount(1000)`) 呼ばれる。通常はここでインスタンス変数を全て初期化する。C++でもクラスと同名のメソッドが同じ役割を果たし、`constructor`(コンストラクタ) と呼ばれる。

性質 この演習では、(Python と C++の共通部分をとって) 以下を前提として説明する。

- 同じクラスから作られたオブジェクトは、同じ名前のインスタンス変数 (`self._balance` など) を持つ。ただし、値は個々に異なる。
- 同じクラスから作られたオブジェクトは、同じ名前のメソッドを持ち、振る舞いも同じである。

8.1.1 C++のクラス

例題 銀行口座 (C++)

銀行口座を C++で定義せよ。ただし、`withdraw` の引数 `amount` は `m_balance` 以下であることを仮定して良い。

C++

```
class BankAccount {
    int m_balance;
public:
    BankAccount(int initial_balance) : m_balance(initial_balance) {
    }
    // BankAccount(int initial_balance) {
    //     m_balance = initial_balance; // 書いても概ね同じ
    // }
    int withdraw(int amount) {
        ...
    }
    int deposit(int amount) {
        ...
    }
};
```

実行確認用

C++

```

#include <iostream>                                1
using namespace std;                                2
// この辺にクラス定義を書く                        3
int main() {                                         4
    BankAccount a(10000);                            5
                                                    6
    cout << a.withdraw(5000) << endl; // 5000        7
    cout << a.withdraw(2000) << endl; // 3000        8
    cout << a.deposit(100) << endl; // 3100          9
}                                                    10

```

コンパイルと実行 (復習)



```

$ g++ -Wall bankaccount.cc                            1
$ ./a.out                                              2

```

C++でも Python 同様に、`class` というキーワードを用いてクラスを定義する。クラスの中には、`balance` のような **メンバ変数** (インスタンス変数) と `withdraw` のような **メンバ関数** (メソッド) を定義する。メンバ関数の中でクラスと同名のものを **コンストラクタ** と呼び、クラスの構築の際に呼ばれる (Python の `__init__` と同様の役割である)。3 行目の、コロン: から中括弧開くまでの並びは、メンバ変数をカッコ内の値で初期化する **メンバ初期化子** という構文である。メンバ変数が整数などの基本型の場合は、7 行目のように代入文で初期化しても差はない。メンバ変数がコンストラクタを持つ場合に、コンストラクタに引数を渡すにはメンバ初期化子を用いる必要がある。なおこの資料では、メンバ変数には `m_` で始まる名前を持たせた。¹⁾ これは Python のインスタンス変数を、`__` で始まることに対応させたものであるが、Python と異なり文法上の意味はない。また Python でインスタンス変数にアクセスする場合には毎回 `self.` で修飾した。C++ で同じ修飾をするには、自信を表す予約後 `this` を用いて、`this->` と書くが、通常は省略可能である。

8.1.2 例外処理

十分な残高がない場合には Python の例題では文字列を返す設計としたが、C++ では **例外** を用いて実装しよう。²⁾

C++ の例外処理機構では、`throw` により例外処理状態に入る。これを例外の **送出** と呼ぶ。また `throw` に続けて書いたオブジェクトによりどのような例外かを区別する。ここでは `<stdexcept>` で実行時例外として定義されている `runtime_error("文字列")` という例外を用いる。一旦例外が送出されると、`try-catch` ブロックに出会うまで、ブロックを抜け関数の呼び出し元に再帰的に戻る。もし対応する `try-catch` ブロックが存在しなければプログラムが終了する。以下の例で、`subtract(a,b)`

1) この命名規則は開発者の自由である。機械的に変数名 (の一部) を決めること自体にも賛否があるが、ここでは深くは立ち入らない。

2) C++ では関数は特定の型を返す必要があるため、“整数もしくは文字列” という型を扱おうとすると煩雑になるためである。なお、先週紹介した `raise` を用いて Python で例外処理を書くことも可能である。

は $b-a$ を返す関数だが $a > b$ の時は例外を送出する。例外発生後に、制御は `subtract` の呼び出し元に戻り、そこが `try` ブロック内であったので対応する `catch` 節に移動してそこから制御を再開する。

C++

```

#include <iostream>                                1
#include <stdexcept>                                2
using namespace std;                               3
int subtract(int a, int b) {                        4
    if (a > b)                                       5
        throw runtime_error("a is greater than b"); // 例外を送出 6
    return b - a;                                   7
}                                                    8
int main() {                                        9
    try {                                          10
        cout << subtract(3,100) << endl; // 97      11
        cout << subtract(11,10) << endl; // ここで例外が発生 12
        cout << subtract(1,10000) << endl; // この行以降は実行されない 13
        cout << subtract(1,10000) << endl;         14
    }                                              15
    catch (exception& e) {                        16
        // try の中で何かが失敗した場合の処理    17
        cout << "failed: " << e.what() << endl;    18
    }                                              19
}                                                  20

```

例外は関数の呼び出しが1段ではなく、何段あっても `try-catch` ブロックまで一気に制御を移す。このため大域脱出のために用いられることもある。

問題 残高確認つき銀行口座の C++ での実装 (`bankaccount.cc`) (option)

銀行口座を C++ で定義せよ。ただし、`withdraw` の引数 `amount` が `m.balance` より大きい場合は `std::runtime_error("Insufficient funds")` を送せよ。

実行確認コード:

C++

```

int main() {                                       1
    BankAccount a(10000);                         2
                                                    3
    cout << a.withdraw(4000) << endl; // 6000      4
    cout << a.withdraw(4000) << endl; // 2000      5
    cout << a.withdraw(4000) << endl; // ここで例外が発生する 6
    cout << a.withdraw(4000) << endl; // これらの行は実行されない 7
    cout << a.withdraw(4000) << endl; //           8
}                                                  9

```



```
% ./a.out 1
6000 2
2000 3
terminate called after throwing an instance of 'std::runtime_error' 4
what(): Insufficient funds 5
```

8.1.3 複数の方を持ちうる型

(オプション) C++17 以降では `std::variant` という型が標準ライブラリに用意されている。以下の例のように、多少の手間を許容すれば、「整数と文字列のどちらか」のような型として使うことができる。

C++17

```
#include <iostream> 1
#include <variant> 2
#include <string> 3
#include <cassert> 4
using namespace std; 5
variant<int,string> subtract(int a, int b) { 6
    if (a > b) 7
        return "error!"; 8
    return b - a; 9
} 10
void show_int_or_string(const variant<int,string>& value) { 11
    if (holds_alternative<string>(value)) { 12
        cout << get<string>(value) << '\n'; 13
    } 14
    else { 15
        assert(holds_alternative<int>(value)); 16
        cout << "success, the result is " << get<int>(value) << '\n'; 17
    } 18
} 19
int main() { 20
    auto a = subtract(20, 100); // should be 80 21
    auto b = subtract(999, 100); // should be "error!" 22
    show_int_or_string(a); 23
    show_int_or_string(b); 24
} 25
```

8.2 Python の help と docstring

Python では `type` という関数で、そのオブジェクトが何かを調べることができる。たとえば、整数 5 の型を調べると、`int` というクラスであることがわかる。さらに、`help` により説明を表示することが出来る。



```
>>> type(5)
<class 'int'>
>>> help(int)
```

1
2
3

関数 `sum` の説明も、同様に `help` で調べることも出来る。



```
>>> help(sum)
```

1

例題 説明付き長方形の面積

演習の初めの方で作成した `rectarea` という関数に、説明を付与し、`help(rectarea)` で説明が表示されるようにせよ。

```
def rectarea(a,b):
    return a*b
```

1
2

文法: docstring Python で関数を説明するために、関数定義の `def` の行の直後に `"""` で囲んだ文字列 (複数行可) を配置する。これを *docstring* という。³⁾

Python3

```
def rectarea(a,b):
    """returns the area of ...
    """
    return a*b
```

1
2
3
4

`help` により確認せよ。

文法: doctest 関数の docstring には、実行例を付与すると分かりやすい。一方で、(何らかのミスやコードの変更で) 実行例が実際の関数と異なると、読者に有害である。そこで、docstring 中の実行例を自動でテストする、*doctest* という機構が備わっている。それには、以下のように `>>>` という記号で実行例を表記する。なお、docstring に複数行を書く場合は、先頭を要約行にして、次を空行にすることが慣例である。

```
def rectarea(a,b):
    """returns the area of a rectangle specified by width a and height b

    >>> rectarea(3,5)
    15
    >>> rectarea(20,10)
    200
    """
    return a*b
```

1
2
3
4
5
6
7
8
9

3) <https://www.python.org/dev/peps/pep-0257/>

jupyter 上で doctest を実行するには、以下のように `doctest.testmod()` により行う。

Python3

```
import doctest 1
doctest.testmod(verbose=True) 2
```

引数の `verbose=True` を消すと出力が簡潔になる。この使い方も `help(doctest.testmod)` で調べることが出来る。



`doctest.testmod()` は、定義済みの関数すべてのテストを一度に行う

問題 説明付き銀行口座

`BankAccount` クラスに説明を、メソッド `withdraw` に説明と doctest 用の実行例を付与せよ。つまり、`help(BankAccount)` や `help(BankAccount.withdraw)` で説明が表示され、実行例が `doctest.testmod(verbose=True)` で確認できるようにする。



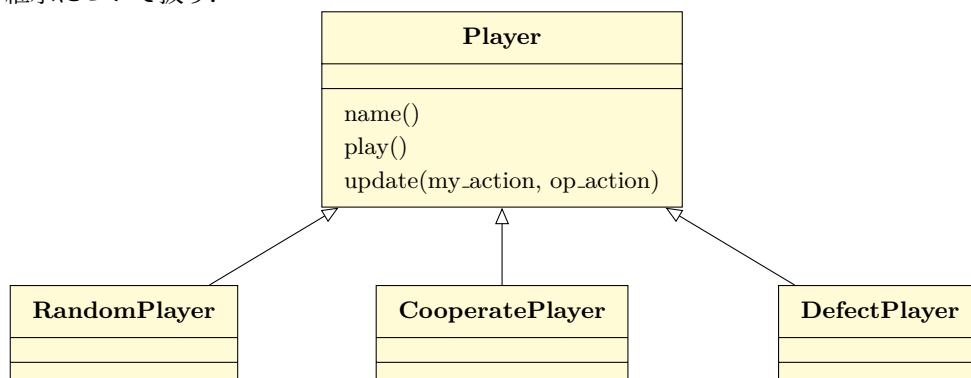
doctest と pytest の使い分け

doctest も pytest も、関数やメソッドの自動テストの機能を持つ。使い分けとしては、doctest には、使い方の説明として適切な (help で表示したい) 代表例を書き、pytest には網羅的なテストや複雑な状況でのテストを書くといい。

8.3 クラスとインターフェースの継承

クラスを定義する際に、親クラス (あるいは super class) を指定することができ、これを継承という。継承により親クラスのインスタンス変数やメソッド定義が子クラスにも受け継がれる。

ここでは、継承の機能の一部に焦点をあてて、メソッド呼び出し規約の定義の (インターフェースとしての) 継承について扱う。



まず親のクラスで、「子クラスで必ず定義してほしいメソッド」を、**抽象メソッド**として定義する。抽象メソッドには、メソッド本体は書かず (docstring のみ書く)、代わりに直前に`@abc.abstractmethod`というデコレータを添える。この例では、`name` と `play` は、子クラスごとに異なる振る舞いをしてほしいので、そのように指定する。

Python3

```
import abc 1
class Player(abc.ABC): 2
    """Player of repeated prisoners' dilemma game""" 3
    4
    @abc.abstractmethod 5
    def name(self): 6
        """Return name of the player""" 7
        pass # この pass は docstring があれば不要、無い場合は必要 8
    @abc.abstractmethod 9
    def play(self): 10
        """Play Cooperate or Defect""" 11
        pass 12
    def update(self, my_action, op_action): 13
        pass 14
```

一方、`update` は、抽象メソッドではなく何もしない `pass` を本体とするメソッドとして定義した。この場合は、子クラスでそのクラス用の `update` を定義しなければ、親クラスの定義 (すなわち `pass`) が使われる。

続いて子クラスを定義する。ここでは `CooperatePlayer` の定義を示すが、`DefectPlayer` も `RandomPlayer` も同様である。Python で親クラスを指定する方法は、クラス定義のクラス名に括弧を続けて親クラス名を指定する。

Python3

```
class CooperatePlayer(Player): 1
    def __init__(self): 2
        super().__init__() 3
    def name(self): 4
        return "CooperatePlayer" 5
    def play(self): 6
        return Cooperate 7
```

文法

- ・継承元 (親): クラス名に続く括弧 () 内は継承元の、親クラスを示す。
- ・親クラスのコンストラクタ: `__init__` 内で `super` という特別な関数により親クラスの `__init__` を呼び出すことができる。
通常は、子クラスの `__init__` の冒頭で `super().__init__()` を呼び出すことが適切である。
- ・抽象メソッド: 子クラスは、親クラスの抽象メソッドを全て実装する。一部でも実装しない場合はインスタンス化できない。以下にエラーの例を示す。 `CooperatePlayer` で実装するべき `play`

を (誤って) `pray` として実装した場合などに発生する。



```
>>> c = CooperatePlayer() 1
TypeError: Can't instantiate abstract class CooperatePlayer with 2
↳ abstract methods play
```

継承の活用 継承を用いることで、異なるクラス (`CooperatePlayer`, `RandomPlayer` など) に共通のインターフェース (`Player` で定義されたメソッド名と呼び出し方) を与えることができる。

- 同じクラスから作られたオブジェクトは、異なる状態 (インスタンス変数の値) を持つことはできるが、振る舞い (メソッド定義) は共通である
- メソッドが呼ばれた時の動作 (たとえば `play()` が何を返すか) を変えるには、異なるクラスを作る必要がある。異なるクラス間の共通性 (たとえば `play()` は何らかの手を返す) を管理するために継承が有用である。

これを用いて、`play_one_game` は、引数 `player_a` などが `CooperatePlayer` であっても `RandomPlayer` であっても動作するように作成することができる。

Python3

```
def play_one_game(player_a, player_b): 1
    act_a = player_a.play() 2
    act_b = player_b.play() 3
    # .. 省略 4
```

8.3.1 C++とインターフェースの継承

C++でも同様に各種のプレイヤーのインターフェースを定めた `Player` クラスを定義しよう。

C++

```
class Player { 1
public: 2
    virtual ~Player(); // destructor 3
    virtual string name()=0; // =0 で子クラスでの定義を強制 4
    virtual int play()=0; // Cooperate or Defect を選択 5
    virtual void update(int, int) { 6
    } 7
}; 8
Player::~Player() { 9
} 10
```

キーワード `virtual` が付与されたメソッド (メンバ関数) を**仮想関数**と呼ぶ。詳細には立ち入らないが、Python 同様に同一名称のメソッドをオブジェクト毎に振る舞いを変えたい場合には、C++では仮想関数として定義する必要がある。仮想関数は、サブクラスで定義が上書き (override) された場合に、そちらを優先して使用するための機構である。また C++では、継承される先祖となるクラスには

仮想デストラクタを定義する必要がある⁴⁾、3行目と9,10行目でデストラクタ (~Player) の宣言と定義を行っている。

さらに、=0; で定義が終わる仮想関数を**純粋仮想関数**と呼ぶ。それらは、メソッドの定義がこのクラス内ではなく、継承したクラスで定義されることを示す (定義なしに使用するとコンパイルエラーになる)。この Player クラスでは、name と play が純粋仮想関数である (4,5 行目)、一方、update は (純粋でない) 仮想関数である。

C++

```
class CooperatePlayer : public Player {           1
public:                                           2
    string name() { return "CooperatePlayer"; }   3
    int play() {                                   4
        return Cooperate;                         5
    }                                             6
};                                              7
```

文法 CooperatePlayer : public Player の部分は、CooperatePlayer が public に Player を継承することを示す。C++ では public でない継承もあるが、この演習では扱わない。

C++11

```
#include <vector>                                1
// Cooperate や Reward の定義は省略する          2
vector<int> play_one_game(Player& a, Player& b) {  3
    int act_a = a.play();                         4
    int act_b = b.play();                         5
    assert(valid_action(act_a));                  6
    assert(valid_action(act_b));                  7
    int reward_a = Reward[act_a][act_b];          8
    int reward_b = Reward[act_b][act_a];          9
    a.update(act_a, act_b);                       10
    b.update(act_b, act_a);                       11
                                                12
    cerr << a.name() << " v.s. " << b.name() << endl; 13
    cerr << " actions: " << act_a << ' ' << act_b << endl; 14
    cerr << " rewards: " << reward_a << ' ' << reward_b << endl; 15
                                                16
    return vector<int>({act_a, reward_a, act_b, reward_b}); 17
}                                                  18
```

関数 play_one_game の引数は、Player& a と Player クラスの参照を取るようにする。親クラスの参照 (この場合 Player クラス) は、それを継承したクラス (たとえば CooperatePlayer) のインスタンスを渡すことができる点である。

文法 : 親クラスの参照またはポインタは、派生クラスの参照またはポインタを代入可能。

関数 play_one_game の戻り値は、Python では配列を用いたが、C++ では (配列が扱いにくい) ため

4) 厳密には必要ないケースも稀に存在するが、数々の注意点が生ずるため必要と理解しておくことを勧める。

標準ライブラリの `vector` を用いた。17 行目の中括弧を用いた記法は C++11 の文法で、中括弧内の 4 つの要素を持つ `vector` をその場で生成する。

C++11

```
int main() {
    CooperatePlayer pc;
    DefectPlayer pd;
    RandomPlayer pr;
    play_one_game(pc, pd);
    play_one_game(pc, pr);
    play_one_game(pd, pr);
}
```

繰り返しゲームを行う場合の実装例を以下に示す。配列の代わりに `vector` を使っている他は、ほぼ Python のコードと対応している。

C++11

```
#include <random>
double random_value() {
    static default_random_engine engine;
    static uniform_real_distribution<double> distribution(0.0,1.0);
    return distribution(engine);
}
void play_many_games(Player& a, Player& b) {
    const double P = 1.0/20;
    int sum[2] = {0, 0};
    string history[2];
    int games = 0;
    while (true) {
        games += 1;
        vector<int> result = play_one_game(a, b);
        sum[0] += result[1];
        sum[1] += result[3];
        history[0] += to_string(result[0]);
        history[1] += to_string(result[2]);
        if (random_value() < P) break;
    }
    cerr << a.name() << ' ' << sum[0] << ' ' << history[0] << endl;
    cerr << b.name() << ' ' << sum[1] << ' ' << history[1] << endl;
}
```

1-5 行目の関数 `random_value` は C++11 の標準ライブラリで、区間 $[0.0, 1.0)$ の一様乱数を発生させる標準的な書き方。16, 17 行目の `to_string` は整数を `string` に変換する C++11 の記法。

問題 C++版 tit-for-tat (option) (tft.cc)

C++でTitForTatPlayerを作成し、動作を確認せよ。

8.4 デザイン・パターン

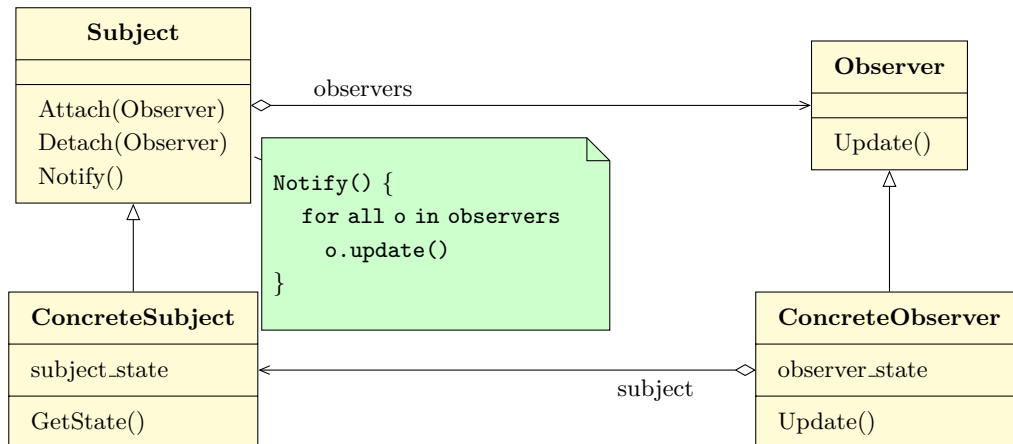
オブジェクト指向 (言語) でプログラムを書く利点の一つは、プログラムの再利用性を高くすることや、プログラムを組み合わせることを用意にすることである。しかし、オブジェクト指向 (言語) を学んだだけでは、そのような利点を活用する方法は自明ではなく、コツを学ぶ必要がある。**デザイン・パターン** (*design patterns*)[GHJV95] は、オブジェクト指向 (言語) をもちいて見通しの良いプログラムを作るための一つの解である:

- 「このような場合には、このように設計すると良い」という経験的なものである (絶対的なルールではなく、また理論的な裏付けがあるわけではない)
- プログラム中の数個のクラスの関係についての指針である (大きなシステムの全体の設計に関する話ではないし、1つのクラスの設計方法でもない)
- カタログである (人間が見て「ここにはこれを使おう」と思うもの)

8.4.1 パターンの例: Observer パターンと時計プログラム

刻々と変化するデータを、様々な主体が観察しそれぞれ適切に行動する、というモデル化について考える。たとえば株式市場のモデル化では、株価が刻々と変化し、様々なプログラムが情報を受け取って、ユーザの画面に表示したり、自動取引を行ったりする。この時に「株価を受け取って行動するプログラム」を簡単に追加したり切り離したりできると便利である。Observer パターンは、データを保持する Subject (株式市場) と、それを監視して変化に追随する Observer (表示や取引プログラム) と役割をモデル化するものである。

株式市場は複雑なので、ここでは Subject(変化するデータ) として時計の時刻、Observer として時刻の表示などを考えてみよう。ここで、時計は全体として「時を刻む」機能と「時刻を表示する」機能を持つが、これらを切り離すことができると双方の再利用性が高まる。例えば、ユーザの好みで時刻の表示方法を取り替えたり、複数の場所に表示するような機能を、時を刻む部分に影響を与えずに実装することができる。前者を subject 側、後者を observer 側として実際に作って見よう。Subject には、以前に 7.1.2 章で作った時計クラスを流用する。



Observer パターンを説明する上記の図には 4 つのクラスが登場する。Subject と Observer は、両者の機能を抽象化したクラスである。ConcreteSubject は具体的な機能を持つ Subject で、この例では時計に相当し、時刻などのメンバー変数を保持する。ConcreteObserver は、この場合は時計の描画部分に相当し、デジタル時計やアナログ時計の機能を持つクラスが該当する。

8.4.2 Subject と Observer

まず、Observer パターン部分のクラス Observer と Subject を定義する：

Python3

```

import abc
class Observer(abc.ABC):
    """abstract observer class"""
    @abc.abstractmethod
    def update(self):
        """update status"""
        pass

class Subject:
    """abstract subject class"""
    def __init__(self):
        self.observers = []
    def attach(self, an_observer):
        """add observer to the list"""
        self.observers.append(an_observer)
    def detach(self, an_observer):
        """remove observer from the list"""
        self.observers.remove(an_observer)
    def notify(self):
        """call update() for all observers"""
        for o in self.observers:
            o.update()

```

Observer は update というメソッドを持つものである。具体的に update が何をするかは、

Observer を継承したサブクラスの中で定義する。実は、Python のような言語ではこのようなクラスを実際に定義しなくても、update という名前のメソッドを持ったクラスであれば何でも Observer になり得る。

Subject の機能は、すべての observer に状態変更を通知する notify と、通知先の Observer を登録/登録解除する attach, detach である。ここでは self.observers というメンバ変数で管理することにする。

8.4.3 時を刻む部分の変更

先に 7.1.2 章で定義した ClockTimer は、tick() という時を進める機能と、hour などの時刻取得機能を持っていた。そのクラスを<Subject> の子クラスとなるよう、定義を少し変更する。そして、時刻が変化した場合は登録された observers の update メソッドが呼び出すために notify メソッドを呼ぶようにする。

Python3

```
class ClockTimer(Subject):
    """a class manages time (self.hour, self.minute, and self.second)"""
    def __init__(self, h, m, s):
        super().__init__() # 追加
        # 元の__init__の内容がここに書かれている
    def tick(self):
        # 元の tick の内容がここに書かれている
        self.notify()
```

このメソッドは内部状態を変化させるので最後に notify を呼ぶ。また継承の際に親クラスがカバーする部分を適切に初期化するために super().__init__() を追加する (8.3)。

ここでは1秒ごとにメソッド tick が呼ばれるものとする。

8.4.4 時刻を表示する部分

時刻を表示する部分として、ここでは単純にコンソールに文字列を書くクラス ConsoleClock を定義しよう。

- このクラスは、update が呼ばれた際に、最新の時刻を得る必要があるため、ClockTimer クラスのオブジェクトへの参照を持っている必要がある。そこで、インスタンス変数 self.subject を用意する。
- メソッド update は self.subject に入っているオブジェクトに最新の時刻を問い合わせ、表示する。表示は時分秒をコロンでつないでコンソールに書くだけとする。
- このクラスのオブジェクトが機能するためには、ClockTimer のオブジェクトに attach される必要がある。ここでは、オブジェクトが ConsoleClock() によって作られた際に必ず呼ばれるメソッド __init__ 内で実現した。

Python3

```

class ConsoleClock(Observer):
    """display time in console"""
    def __init__(self, subject):
        """create ConsoleClock and attach to subject"""
        super().__init__()
        self.subject = subject
        self.subject.attach(self)

    def clock_to_string(self, clock):
        return
        → f"{clock.hour:02d}:{clock.minute:02d}:{clock.second:02d}"

    def update(self):
        print(self.clock_to_string(self.subject))

```

実行例: まず, ClockTimer を作成する. ここで timer を tick すると, 内部で 1 秒進むが, 表示は何も起こらない.



```

>>> timer = ClockTimer(22, 54, 12)
>>> timer.tick()

```

次に, ConsoleClock を作成して timer に登録する. この状態で timer を tick() すると, 連動して登録された cc.update() が呼ばれて, 時刻が表示される.



```

>>> cc = ConsoleClock(timer)
>>> timer.tick()
22:54:14
>>> timer.tick()
22:54:15

```

その後, ConsoleClock を timer から切り離れたとする. この状態で timer を tick() しても, 表示は起こらない.



```

>>> timer.detach(cc)
>>> timer.tick()

```

問題 複数の observer (many-observers)

1 つの ClockTimer オブジェクトに対して複数の observer オブジェクトを作った場合でも, 全ての表示が正しく行われることを確認せよ. 確認後に, 適切な doctest として記述すること.

問題 砂時計 (countdown)

残り時間を測り、残り時間が無くなったらアラームを鳴らす時計 CountdownClock を作成せよ。

実装方法としては、Observer を継承して作成するとする。ConsoleClock とほとんど同じだが、Subject の tick に連動して時間を減らし、残り時間が 0 になった時にアラームを鳴らすようにすればよい。残りの時刻は、CountdownClock 側で管理し、*ClockTimer* は変更しないこと。適切な doctest を付与すること。

使用例:

```
>>> cd = CountdownClock(timer, 3) 1
>>> timer.tick() 2
3 3
>>> timer.tick() 4
2 5
>>> timer.tick() 6
1 7
>>> timer.tick() 8
!!! 9
```

8.5 C++で class を扱う注意点

8.5.1 複製と参照，スマートポインタ

オブジェクトを複製した場合に何が起ころうか？ Python の場合は、1 つの口座に対応する銀行カードを複製したような動作になる。

```
>>> a = BankAccount(10000) 1
>>> a 2
<__main__.BankAccount object at 0x102d8f5f8> 3
>>> a.withdraw(3000) 4
7000 5
>>> b = a # a の複製として b を作成 6
>>> b 7
<__main__.BankAccount object at 0x102d8f5f8> 8
>>> b.withdraw(3000) # b を引き出すと 9
4000 10
>>> a.withdraw(3000) 11
1000 # a も減っている 12
```

C++の場合はどうだろうか

C++

```

BankAccount a(10000);                                1
cout << a.withdraw(3000) << endl; // 7000              2

BankAccount b = a;                                     3
cout << b.withdraw(3000) << endl; // 4000              4
cout << a.withdraw(3000) << endl; // ???               5

```

C++では、上記の4行目のように書くと、銀行のカードではなく、同じ残高を持つ銀行口座が新しく作成される(これが望ましい動作かどうかは状況に依存する)。

C++

```

BankAccount a(10000);                                1
cout << a.withdraw(3000) << endl; // 7000              2

BankAccount &b = a; // アンパサンド記号に注意          3
cout << b.withdraw(3000) << endl; // 4000              4
cout << a.withdraw(3000) << endl; // 動作は変わるか???? 5

```

C++で口座に対する別名を実現するには、参照(上記の4行目)やポインタを用いる必要がある。しかし、オブジェクトの寿命に対する正確な理解が必要となり、ソースコードの管理コストが増える。本演習では、C++11で標準ライブラリで利用可能な参照カウンタ付きスマートポインタ `shared_ptr` を勧める。

C++11

```

#include <memory>                                       1
int main() {                                           2
    shared_ptr<BankAccount> a(new BankAccount(10000)); // 作成 3
    cout << a->withdraw(3000) << endl; // 7000          4

    shared_ptr<BankAccount> b = a; // b は a の別名      5
    cout << b->withdraw(3000) << endl; // 4000          6
    cout << a->withdraw(3000) << endl; // 1000          7

    shared_ptr<BankAccount> c; // 作成時に指定しなければ、対応口座なし 10
    // c->withdraw(3000) -- segmentation fault          11
    c = a; // あとから指す先を変更                     12
    c.reset(new BankAccount(10000)); // 新しく作った口座を指すように変 13
    ↪ 更
}
```

3行目の作成の読み方は、`shared_ptr<BankAccount>`が型を、`a`が変数名を、`new BankAccount(10000)`が`a`が表す実体のオブジェクトを表す。4行目以降の用に、メンバ関数(メソッド)を呼ぶときには、ドット(`.`)ではなくアロー演算子(`->`)を用いる。

**Notice: ポインタとメモリ確保/解放機構を使う前に知っておくこと**

文法をひと通り学ぶと `malloc/free` や `new/delete` を扱うので、つい (?) 使いたくなるかもしれない。しかし、使う前には誤った使用のリスクを知っておく必要がある。以下の規則に違反すると、プログラムの意図通りの実行は保証されない (i.e., 暴走する)。

1. アクセスするアドレスは正しいものであること (基本的には `new` したアドレスで、まだ `delete` されていないこと)。
2. 確保した方法と対応する方法で解放しなければならない (`malloc` と `free`, `new` と `delete`, `new[]` と `delete[]`)
3. 確保したアドレス以外は解放してはならない (例外として `nullptr` は `delete` に渡しても無害)。二重に解放してはならない。解放したあとはアクセスしてはならない

また、次の原則は、メモリが不足を避けるために必要なことが多い。

4. 確保したメモリ領域は、不要になったら解放する。

必要かどうかの検討には、`new` する領域の合計と `delete` 可能な領域の合計のバランスを見積もる。

問題 メモリ管理 (option) (memory.txt)

以下の `Node` class の定義では、`object` を作成時に呼ばれるコンストラクタに、左右の子ノードを `new` して作成たものを渡し、`object` の寿命を迎えた時点で呼ばれる destructor で左右の子ノードの領域を `delete` することを意図して書かれている。しかし、このクラスの定義にはいくつかの点で不備がある。クラス定義に続く `main` 関数での使用例を参考に、具体的に指摘せよ (3箇所上記の違反がある)。指摘を記述したテキストファイルを提出する。

```

class Node {
    Node *m_left, *m_right;
public:
    Node() {}
    Node(Node *l, Node *r) { m_left = l; m_right = r; }
    ~Node() { delete m_left; delete m_right; }
};

int main () {
    Node a;
    Node b(new Node(nullptr, nullptr),
           new Node(nullptr, nullptr));
    Node c(b);

    Node x(new Node(nullptr, nullptr),
           new Node(nullptr, nullptr));
    x = Node(nullptr, nullptr);
}

```

問題 C++版二分探索木 (option) (bst.cc)

前回までに Python を用いて二分探索木を作成した。これを C++でも作成せよ。

C++11

```

class Node {
    int m_number;
    shared_ptr<Node> m_left_node, m_right_node;
public:
    Node(int initial_number,
         shared_ptr<Node> left, shared_ptr<Node> right)
        : number(initial_number),
          m_left_node(left),
          m_right_node(right) {}
    int value() const { return m_number; }
    shared_ptr<Node> left() const { return m_left_node; }
    shared_ptr<Node> right() const { return m_right_node; }
};

```

実行例:

C++11

```

int main() {
    int A[] = {3, 1, 4, 1, 5, 9, 2};
    node_t t;
}

```



```

    for (int a: A)                                4
        t = add_value(t, a);                        5
    list_values(t);                                6
    cout << endl;                                  7
}                                                    8

```

問題 C++版 Observer パターン (opiton) (observer.cc)

Observer パターンを C++ で実装せよ.

C++11

```

int main() {                                       1
    ClockTimer timer(22, 54, 13);                 2
    ConsoleClock cc(&timer);                      3

    timer.tick();                                4
    timer.tick();                                5
    timer.tick();                                6
}                                                    7
                                                    8

```

実装例は以下の通り.

C++11

```

class Observer {                                  1
public:                                           2
    virtual ~Observer();                         3
    virtual void update() = 0;                  4
};                                                5
Observer::~Observer() {                          6
}                                                7
class Subject {                                  8
    vector<Observer*> m_observers;                9
public:                                         10
    virtual ~Subject();                       11
    void attach(Observer *observer) {          12
        m_observers.push_back(observer);      13
    }                                          14
    void detach(Observer *observer) {          15
        m_observers.erase(find(m_observers.begin(), m_observers.end(), 16
                                observer));    17
    }                                          18
    void notify() {                            19
        for (Observer *o: m_observers) {      20
            o->update();                       21
        }                                     22
    }                                          23
}

```

```
};
Subject::~Subject() {
}
```

C++の場合も、Pythonの場合と基本的に同じことを記述する。相違点として、まず `update` のようにサブクラスでは動作を変更したいメソッドには、`virtual` というキーワードを指定する。また後で継承して用いる先祖のクラスには、`virtual` 指定をしたデストラクタ (`~Observer` や `~Subject`) を定義する。Observer の一覧は、標準ライブラリである `vector` に `Observer` のポインタを入れて管理する。メソッド `detach` 内では、指定された `observer` のポインタを `m_observers` から削除する。find は指定範囲から要素を探す標準関数で位置を `iterator` で返すものである。メソッド `notify` 内の `for` 文は、C++11 で導入された文法で、`m_observers` すべてに対してブロックを実行する書式である。

C++11

```
#include <cstdio>
class ClockTimer : public Subject {
    int m_hour, m_minute, m_second;
public:
    ClockTimer(int hour, int minute, int second)
        : m_hour(hour), m_minute(minute), m_second(second) {
    }
    void tick() {
        int next_second = m_second+1;
        int next_minute = m_minute + next_second/60;
        int next_hour = m_hour + next_minute/60;

        m_hour = next_hour % 24;
        m_minute = next_minute % 60;
        m_second = next_second % 60;

        notify();
    }
    int hour() const { return m_hour; }
    int minute() const { return m_minute; }
    int second() const { return m_second; }
};
class ConsoleClock : public Observer {
    ClockTimer *m_subject;
public:
    ConsoleClock(ClockTimer *subject) : m_subject(subject) {
        m_subject->attach(this);
    }
    ~ConsoleClock() {
        m_subject->detach(this);
    }
    void update() {
```

```

        printf("%02d:%02d:%02d\n", m_subject->hour(),           33
               m_subject->minute(), m_subject->second());         34
    }                                                            35
};                                                                36

```

29-31 行目の ConsoleClock のデストラクタでは、subject から detach している。このデストラクタが呼ばれる状況以降では、Subject クラスの m_observers 内に格納されたポインタは無効となるためである（もし detach しないと次の notify で不正なメモリアクセスとなる）。なお、例にあげたコードは ConsoleClock のコピーを作ると、observer のインスタンスは必ず attach されているというこれまでの原則にあわない状態となる。実用品を作る際はコピーを禁止する（コピーコンストラクタを呼び出せないようにする）などの対処が必要である。

8.6 この章の提出課題

以下のいずれかで合格:

- 問題 (タイトル部分に option の表記なし) を全て Python で作成する
- 「複数の observer」と「C++ 版 Observer パターン (opiton)」を作成したうえで、(option) も含めた問題になるべく多く取り組む。

提出条件:

- 各関数に対応するテストケースが doctest で記述されていること
- 指定した課題が、指定した関数名やクラスとメソッド名で作成されていること
- 提出ファイルは、2.A 節を参考にして、*jupyter* から作成した weekN-(学生証番号).ipynb とする
- 日本語 (または英語) でコメントが付記されていること: { 途中か完成品か, 授業中に既に OK を得ていればその教員名 } 学んだこと, (あれば) 他に参考にした資料や他の人の回答など、(途中で提出する場合は何がうまくゆかずに困っているなど) をコメントで盛り込む。(何も苦労がなかった場合を除き、自作したことが分かるだけの十分なエピソードを記述すること — 授業中に教員の OK を得た場合は簡潔で良い)

第 9 章 状態の保存と通信

2026-06-17 Wed

概要

複数のプログラムが協調して動作する場合や、中断した計算を再開するためには、実行の状態を適当な表現に変換してやりとりする必要がある。テキスト形式のファイルの読み書きと、少し複雑なデータを json で表現することを紹介する。また、最終課題の準備の一部として、パッケージ管理の導入や、jupyter を離れてコマンドラインから Python を実行する方法を紹介する。

9.1 モジュールとパッケージ

一般にソフトウェアはさまざまな component から構成される。研究等でプログラムを作成する場合でも、自分で書く部分を正しく書くだけでなく、既存のソースコードを正しく理解してうまく組み合わせることも肝要である。

Python の module は、ひとまとまりの機能のまとまりで、import で利用可能となる。通常 import x とするとモジュール x を利用可能になる。これまでも、import math として math.sqrt を使うことを紹介した。また、1 つ以上の module を管理する単位を package と呼ぶ。Package は複数の module を含むうる。¹⁾

必要な機能を探す際は、おおむね以下の順が良い

種別	使い方	例
標準関数	Built-in functions, constants, types など built-in の項目に掲載されているもの。import なしで使える。 https://docs.python.org/3/library/index.html	sum, str
標準ライブラリ	上記の URL で、built-in でないもの。import 後に使える。	math, sys, os, urllib.
PyPI パッケージ	PyPI https://pypi.org/ に登録されているパッケージで、pip install するとダウンロード、インストールされる。その後に import して使える。	matplotlib, numpy
その他パッケージ	インストール方法がさまざま	潜水艦ゲーム (後述)

インストールされているパッケージの一覧は pip list により得られる (jupyter のセルから実行す

1) <https://docs.python.org/3/glossary.html>

る場合は `!pip list`). また, `import` 時にパッケージを探すフォルダのリストは, `sys.path` という変数に格納されていて, `import sys` 後にアクセスできる.

```
>>> import sys
>>> sys.path
['/usr/lib/python310.zip', ...,
↪ '/usr/local/lib/python3.10/site-package']
```

(実際の出力は各環境により異なる)

パッケージに対応する module について, `import` 後に `__path__` が対応するソースファイルの位置を示す.

```
>>> import numpy as np
>>> np.__path__
['/usr/local/lib/python3.10/site-package/numpy']
```

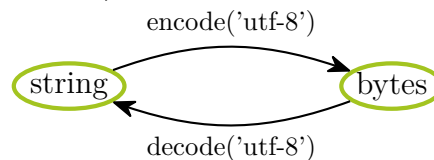
9.2 データの扱い

ファイル保存と読み込みや, ネットワークで送受信する際には, 対象となるデータの表現を適切に選ぶ必要がある.

通信相手	種別	用途
人	テキスト	log など
外部	csv	表
	<i>json</i> , yaml	階層データ, 木構造, dict
内部	<i>npz</i>	<code>np.array</code> (ベクトル, 行列)
	pickle	Python object (お勧めしない)

9.2.1 文字列とバイト列

Python や Java, Rust など現代的な言語では, 文字列 (`string`) と **バイト列** `bytes` を区別して扱う. 通常は文字列を扱えば良いが, ネットワーク通信などバイト列が必要になる場合もある. 文字列のバイト列表現を得たり, 逆の変換の際には, **符号化方式**を指定する必要がある.



以下の実行例で `\xe3` は 2 桁の 16 進数 `e3` を表す. 他も同様.



```
>>> "プログラミング".encode('utf-8') 1
b'\xe3\x83\x97\xe3\x83\xad\xe3\x82\xb0\xe3\x83\xa9\xe3\x83\x9f\xe3\x83\xb3\xe3\x82\xb0'
>>> "プログラミング".encode('euc-jp') 3
b'\xa5\xd7\xa5\xed\xa5\xb0\xa5\xe9\xa5\xdf\xa5\xfb\xa5\xb0'
>>> b = "プログラミング".encode('utf-8') 5
>>> b.decode('utf-8') 6
'プログラミング' 7
```

符号化方式は、日本では通常 *utf-8* を用いる。過去の日本では、ISO-2022-JP, EUC-JP, Shift_JIS など使われた。同じ文字でも、符号化方式が異なると、異なった byte 列に符号化される。

byte 列は、文字で表現できるものとも限らない。任意の byte 列を ASCII で印字可能な限られた文字で表現する手法の一つが *base64* である。電子メールに画像などを添付する場合や、colab に png をコピーペースト²⁾ した場合など、目にする機会も多い。

問題 Base64 (option)

文字列 *s* の 'utf-8' のバイト列に対する base64 表現を得る、*myencode(s)* を実装せよ。また、同関数で変換した base64 表現から、元の文字列を得る *mydecode(s)* を実装せよ。自分の姓名(持っている人は漢字)を Base64 でエンコードして表示し、それを復号して元に戻ることを確認する doctest を付与すること。

ヒント: Python3 では *base64* というモジュールが用意されている。import *base64* して使用せよ。モジュールの使い方は *help(base64)* により調査せよ。ヒント: バイト列を base64 に変換する関数は *base64.b64encode()* である。



```
>>> myencode("プログラミング") 1
'440X440t44Kw440p440f440z44Kw' 2
>>> mydecode('440X440t44Kw440p440f440z44Kw') 3
'プログラミング' 4
```

9.2.2 テキストファイルの読み書き

まず、テキストファイルを行単位で読み書きするには、*open* という関数を用いる。

Python3

```
with open("file-test.txt", "w") as file: 1
    file.write("hello\n") 2
    file.write("world\n") 3
```

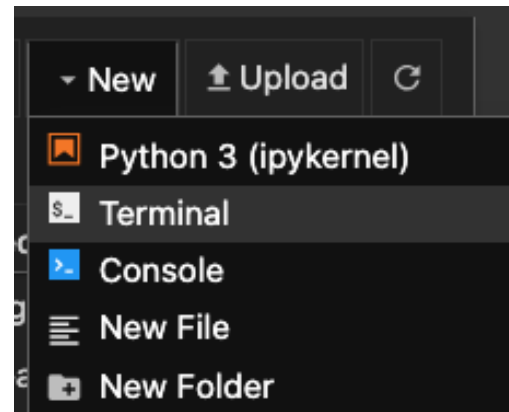
ここで with で始まるブロックにおいて、*file* は、open したファイルを指す。ブロックを抜けると、自動で close される。文中の *file.write* は、指定したファイルを対象に、引数で指定された文字列の

2) https://en.wikipedia.org/wiki/Data_URI_scheme

書き込みを行う。

ファイルが作成されたことを「ターミナル」を開いてコマンドで確認しよう。これ以降、ターミナルを多用する。macOS のターミナルは、 というアイコンでドックに登録されている。Microsoft Windows で同等の環境を用意する方法は 9.A 節 を参照。実行例で、“\$” で始まる行は、ユーザ (あなた) が入力するべき行の目印である。ただし、“\$” 自身はプロンプトの意図なので入力せず、その右から入力し行末でエンターキーをタイプする。使い方は、はいばーワークブックなどで確認すること。³⁾

最近の jupyter はターミナル機能を持つので、そちらを使っても良い。



`cat` はファイルの中身を表示するコマンドである。プログラムの動作確認のために、対応するコマンドを覚えておくと良い。なお、文字 “#” とそれより右は、コメントの意図なので、入力の必要はない。

```

$ cat file-test.txt # テスト
hello
world

```

改行文字や、アスキーコードも含めて確認するためには、`od -xc` コマンドを使うことができる。

```

$ od -xc file-test.txt
0000000  6568  6c6c  0a6f  6f77  6c72  0a64
          h  e  l  l  o  \n  w  o  r  l  d  \n
0000014

```

ファイルを読み込んで表示する Python のコードは以下のようになる。

```

Python3
with open("file-test.txt", "r") as file:
    for line in file:
        line = line.strip() # 改行文字を除去
        print(line)

```

3) <https://hwb.ecc.u-tokyo.ac.jp/hwb2023/information/cui/terminal/>

ターミナルの一部の操作は, jupyter のセルから ! 記号を用いて行うこともできる.

```

▶ >>> !cat file-test.txt
hello
world

```

9.2.3 path

macOS も含めて多くの OS で ~ はホームディレクトリの表記に用いられる. 一方で, (慣習から外れて) ~ そのものを名前とするファイルやフォルダも扱うことができるので注意が必要である. ホームディレクトリとしての表記を, 実際のパスに変換するためには, `os.path.expanduser` 関数を用いる. たとえば, ダウンロードフォルダ内の `filename.txt` を `open` する際は, 次のように変換してから用いる.

```

Python3
import os.path
os.path.expanduser('~Downloads/filename.txt')

```

`os.path` モジュールには, ファイルの存在確認などファイルパスを扱う関数が多数提供されている.
<https://docs.python.org/3/library/os.path.html>

9.2.4 CSV

表形式のデータをテキストで表現するフォーマットとして CSV が長く使われている. 1 行 1 レコード, 列の区切りにカンマを使うというシンプルな文法で, 計算機の可読性が高い. 内容が数だけの場合は, `'1,2,3'.split(',')` など要素を取り出すこともできるが, 文字列も含む場合はダブルクォートでくくるなど様々な文法があるため, 標準ライブラリ `csv` を使う方が良い.

例として, 神戸市のオープンデータ⁴⁾の観光・文化から, 「ビューポイント一覧」というファイルをダウンロードし, `~/Downloads/viewpoint.csv` に保存したとする. 以下はこのデータを読んで先頭 3 行を表示するコード片である. なお, 文字コードが Shift JIS なので `open` の際に `encoding` を指定している.

```

▶ >>> import csv
>>> import os.path
>>> with open(os.path.expanduser('~Downloads/viewpointmap.csv'),
↳ encoding='sjis') as file:
...     reader = csv.reader(file)
...     rows = [_ for _ in reader]
...     for i in range(3):
...         print(rows[i])

```

4) <https://data.city.kobe.lg.jp/>


```
['名前', '経度', '緯度', '説明'] 8
['01_保久良神社（10選）', '135.278118', '34.734791', '金鳥山の中腹にあ 9
↳ る...
```

CSV モジュールの詳細は、標準文書 <https://docs.python.org/3/library/csv.html> を参照されたい。なお、大きな表データを扱う際には、pandas や polar などの外部ライブラリを使うと、メモリ効率などの点でメリットがある。

9.2.5 階層データと dict

Python の *dict* (辞書, マッピング, **連想配列**) は、データの組 (key, value) の組を保持し、key から value を容易に検索することができる。別の言い方をすると、配列の添字は整数だが、連想配列では整数以外にも添字のように使える。(以前二分木のところで簡単に紹介した)。他の言語でも C++ の `unordered_map` や Ruby の **ハッシュ** などと同等の機能が提供されている。

連想配列は、`{}` で構築する (配列を `[]` で構築したことと似ている)。キーに対応するデータの登録や参照は、配列と同様の文法を用いる。キーが登録済みかどうかは、キー `in` 連想配列という構文で調べることができる。

```
>>> dic = {} # 連想配列の構築 1
>>> dic['hello'] = 'kon-nichi-ha' # データの登録 2
>>> dic 3
{'hello': 'kon-nichi-ha'} 4
>>> 'hello' in dic # key "hello"は今登録したばかり 5
True 6
>>> 'good morning' in dic 7
False 8
>>> dic['hello'] # データの参照 9
'kon-nichi-ha' 10
>>> dic['four'] 11
Traceback (most recent call last): 12
  File "<stdin>", line 1, in <module> 13
KeyError: 'four' 14
>>> dic['four'] = 4 # 文字列以外の型も使用可能 15
>>> dic['four'] 16
4 17
```

配列や参照を含む dict

`dict` には、配列や別の `dict` を登録することができる。

例として、階層ディレクトリのフォルダを `dict` で表すことを考える。あるフォルダには、(フォルダでない) ファイルと、フォルダがあるので、それぞれ `"files"` と `"folders"` で表すとする

空のディレクトリを表すオブジェクトは以下のように構築できる。



```
>>> a = {}
>>> a["files"] = []
>>> a["folders"] = []
>>> a
{'files': [], 'folders': []}
```

1
2
3
4
5

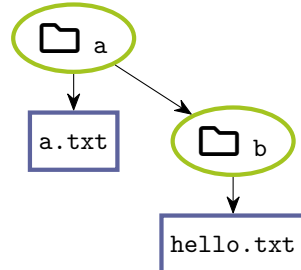
以下のように、dict の構築時に key, value を: 記号でつないで、データを指定することもできる。



```
>>> b = {"files": [], "folders": []}
>>> b
{'files': [], 'folders': []}
```

1
2
3

以下の図のような、階層関係を構築してみよう。



```
>>> b['files'].append('hello.txt')
>>> b
{'files': ['hello.txt'], 'folders': []}
>>> a['files'].append('a.txt')
>>> a
{'files': ['a.txt'], 'folders': []}
>>> a['folders'].append(b)
>>> a
{'files': ['a.txt'], 'folders': [{'files': ['hello.txt'], 'folders': []}]}
>>> b
{'files': ['hello.txt'], 'folders': []}
```

1
2
3
4
5
6
7
8
9
10
11

テスト例:



```
>>> a['files'][0]
'a.txt'
>>> a['folders'][0]
{'files': ['hello.txt'], 'folders': []}
>>> a['folders'][0]['files'][0]
'hello.txt'
```

1
2
3
4
5
6

9.2.6 JSON: 階層データの文字列表現

さて、階層構造を含むようなデータは、行でデータを区切るテキストファイルで表現することが難しい。そこでウェブを中心に広く使われている JSON という表現形式を紹介する。

Python ではまず json ライブラリを読み込む。

Python3

```
import json
```

1

これにより、Python のオブジェクトを JSON 形式の文字列に変換する `json.dumps` 関数ができるようになる。



```
>>> json.dumps([3,1,4])
'[3, 1, 4]'
```

1

2

先ほどのディレクトリ `a` のような構造を持つデータも、文字列に変換される。



```
>>> json.dumps(a)
'{"files": ["a.txt"], "folders": [{"files": ["hello.txt"], "folders":
↳ []}]}'
```

1

2

変換された文字列は、`json.loads` という手続きにより元のデータを取り出すことができる。



```
>>> json.loads('[3, 1, 4]')
[3, 1, 4]
```

1

2

問題 二分木の保存 (`write_tree`, `read_tree`)

6.1.6 項 “内部表現の世界と使用する世界の分離” では二分木を `dict` で表現した。二分木 (たとえば Q&A で作成したもの) をファイルに保存する関数 `write_tree(filename, tree)` と読み込んで返す関数 `read_tree(filename)` を作成せよ。同じ木が復元できていることを確認せよ。

ただし、二分木の表現としては以下を使うと良い。これらは 6.1.6 項のものと同じである。

Python3

```
def make_node(num, left, right):
    return {"number": num, "left": left, "right": right}

def value(tree):
    return tree["number"]

def left(tree):
```

1

2

3

4

5

6

7

```

    return tree["left"]
8
9
def right(tree):
10
    return tree["right"]
11

```

9.3 HTTP

9.3.1 HTTP での読み込み

Python では `urllib.request` を読み込んでおくと、`open` と同様にして HTTP 読み込みが可能になる。⁵⁾

Python3

```

import urllib.request
1
with urllib.request.urlopen("https://www.ecc.u-tokyo.ac.jp/") as
2
    response:
    the_page = response.read()
3
    the_html = the_page.decode('utf-8')
4
    print(the_html)
5

```



```

<!DOCTYPE html>
1
<html lang="ja">
2
<head>
3
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
4
    <link rel="shortcut icon" href="/favicon.ico">
5
    <meta name="date" content="2018-04-26T12:03:09:00">
6
    <meta name="viewport"
7
        <=> content="width=device-width,initial-scale=1.0">
8
    ...
9
</head>
10
</html>

```

例題 HTTP 読み込み

指定した URL の内容を、指定したファイルに保存する関数 `httpget(url, filename)` を作成せよ。

なお、同様の操作は `curl` というコマンドを用いて以下のように行うことができる。また `wget` という似た機能を持つコマンドも広く使われている。

5) <https://docs.python.jp/3/howto/urllib2.html>

```
$ curl -o filename 'http://...'
```

9.3.2 データの配布

データの配布を http と json で実現することも多い。

たとえば、郵便番号と住所の対応のデータは元々公開されているが、市町村の合併などで変更があるため、郵便番号を用いるソフトウェアは常に更新に気を配る必要がある。検索を容易にするために、json でデータを配布する有志のサイトもある。⁶⁾

問題 住所の表示 (showaddress; option)

便番号をあらわす文字列、**'1538902'** などから該当する URL の json を入手し、住所を表示するプログラムを書いてみよう。

なお、他サイトのサーバの負担にならないように、プログラムの動作検証を終えるまでは、担当教員が複製した以下の URL でまずは練習すること。

- <https://www.graco.c.u-tokyo.ac.jp/~kaneko/lecture/pl/1538902.json>
- <https://www.graco.c.u-tokyo.ac.jp/~kaneko/lecture/pl/1138654.json>

9.3.3 geojson

GeoJSON は、比較的新しく制定されたフォーマットで、位置を含むデータを扱う。ライブラリを組み合わせると GIS の前提知識をほぼ必要とせずに、地図にマーカを描くことができる。

https://gis-oer.github.io/gitbook/book/materials/web_gis/GeoJSON/GeoJSON.html

詳しくは別資料を参照 <https://colab.research.google.com/drive/1osriLjeM1zGSHKIV4YsYXZm-xpfgC6e9>

9.4 numpy

Python で数値計算を行う場合は、*numpy* というライブラリが用意されている。⁷⁾

以下は、numpy で 3x4 の行列を作成する例である。

Python3

```
import numpy as np
a = np.zeros((3,4))
```



```
>>> a
array([[0., 0., 0., 0.],
```

6) 例 <https://github.com/ttskch/jp-postal-code-api>

7) <https://numpy.org/doc/stable/user/quickstart.html>

```
[0., 0., 0., 0.],      3
[0., 0., 0., 0.]])    4
```

1 行目の `import numpy as np` は、`np` という別名で、`numpy` モジュールにアクセスできるようにする構文である (`numpy` を `np` という別名で扱う慣習に、本資料も倣った)。

巨大なベクトル 可読性を維持するため、大きなベクトルや行列は、適宜省略して表示される。なお、下のコード例の途中の `+=1` は、全要素に対して 1 を加える演算である。

```
>>> large_array = np.zeros(10000)      1
>>> print(large_array)                 2
[0. 0. 0. ... 0. 0. 0.]                3
>>> large_array += 1                   4
>>> large_array                         5
array([1., 1., 1., ..., 1., 1., 1.])    6
```

9.4.1 線形代数

行列の乗算には `@` 記号が用意されている⁸⁾。

$$\underbrace{\begin{pmatrix} 8 & 3 & 4 \\ 1 & 5 & 9 \\ 6 & 7 & 2 \end{pmatrix}}_A \underbrace{\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}}_b = \underbrace{\begin{pmatrix} 26 \\ 38 \\ 26 \end{pmatrix}}_c$$

の計算例を示す。

```
>>> A = np.array([[8,3,4], [1,5,9], [6,7,2]])      1
>>> b = np.array([1,2,3])                          2
>>> c = A@b                                          3
>>> c                                                4
array([26, 38, 26])                                 5
```

線形代数ライブラリは `np.linalg` にあり、 $Ax = c$ を解いて x を得るには `np.linalg.solve` を用いる。

```
>>> np.linalg.solve(A, c)                          1
array([1., 2., 3.])                                2
```

逆行列は `inv` で求める。ただし、逆行列の用途が連立式 $Ax = c$ をとくことの場合は、数値計算の誤差の観点から、前述の `np.linalg.solve(A, c)` を使う方が良い。

8) Python 3.5 以降 <https://docs.python.org/ja/3/whatsnew/3.5.html#whatsnew-pep-465>



```
>>> A=np.array([[1,2], [-3,4]]) 1
>>> np.linalg.inv(A) 2
array([[ 0.4, -0.2], 3
       [ 0.3,  0.1]]) 4
```

固有値, 固有ベクトルを求める:



```
>>> np.linalg.eig(A) 1
(array([2.5+1.93649167j, 2.5-1.93649167j]), 2
 array([[0.38729833-0.5j, 0.38729833+0.5j], 3
        [0.77459667+0.j , 0.77459667-0.j ]])) 4
```

9.4.2 numpy データの保存

複数の numpy オブジェクト (行列やベクトル) を一つのファイルに圧縮して保存することが出来る. ここでは *npz* という numpy の圧縮形式で保存する, `np.savez_compressed` を紹介する.⁹⁾ 個別にテキストファイルや json で保存するよりも効率が良く期待できる. 保存の際の最初の引数はファイル名で続けて, 名前=変数の形式で名前を指定する. 読み込みは, `np.load` で行い, 帰ってきた object に同じ名前でアクセスすると, 元のデータを得られる.

Python3

```
np.savez_compressed("filename.npz", matrix=a, vector=large_array) 1
```



```
>>> loaded_objects = np.load("filename.npz") 1
>>> loaded_objects["vector"] 2
array([1., 1., 1., ..., 1., 1., 1.]) # 保存時の large_array に相当 3
```

9.5 乱数の管理

疑似乱数に関する機能は `random module` で提供されている. 指定の確率分布に従う乱数の生成や, シャッフルなど, さまざまな機能が提供されているので, 必要に応じて標準文書を参照されたい.

<https://docs.python.org/3/library/random.html>



```
>>> import random 1
>>> random.random() # [0,1) 一様分布 2
0.43401144301524397 3
>>> random.randrange(200) # [0, 199] の整数 4
183 5
```

9) https://numpy.org/doc/stable/reference/generated/numpy.savez_compressed.html

```

>>> a = ['orange', 'apple', 'banana', 'pear']           6
>>> random.choice(a)                                   7
'banana'                                                8
>>> random.sample(a, 2)                                9
['apple', 'orange']                                    10

```

なお、numpy を使う場合は、`np.random` を使う。



```

>> import numpy as np                                  1
>> np.random.randint(1, 6, size=10)    # サイコロを 10 回ふった 2
array([5, 5, 2, 5, 5, 4, 2, 1])              3

```

9.5.1 seed

計算機シミュレーションにおける再現性を確保したい場合など、疑似乱数の系列を制御したい場合もある。`random.seed` を使うと状態を制御できる。

初めに `seed(0)` を指定して、`random.random()` で乱数列を生成したとして、



```

>>> random.seed(0)                                     1
>>> print(random.random())                             2
0.8444218515250481                                     3
>>> print(random.random())                             4
0.7579544029403025                                     5

```

リセットしたいタイミングで `seed(0)` を呼ぶと、`random.random()` で生成される列が、もともとと同じになる。



```

>>> random.seed(0)                                     1
>>> print(random.random())                             2
0.8444218515250481                                     3
>>> print(random.random())                             4
0.7579544029403025                                     5

```

`numpy.random.seed` も同様。

9.5.2 状態管理

乱数を使うモジュールが複数ある場合、(たとえば、別に関与されたプレイヤーを対戦させる場合など)、乱数列はそれぞれ管理することが望ましい。たとえば、`random.seed` を複数のモジュールから呼んでしまうと、意図する動作にはほぼならない。

推奨する方法は、`random.Random(seed)` により疑似乱数発生オブジェクトを作り、そのオブジェクトから乱数を発生させることである。以下の例では、`rng_a` がオブジェクトで、`random` や `randrange`

など、これまでに紹介した関数と同名のメソッド・機能を利用できる。この乱数の発生状態は、他の乱数生成の影響を受けない。

```

>>> rng_a = random.Random(0)
>>> rng_a.randrange(200)
98
>>> rng_a.random()
0.7579544029403025

```

`np.random.RandomState` も同様の機能を持つ。

9.6 ファイルからの実行

これまでの演習では、標準環境として jupyter を指定し、ブラウザ上で動作するインタプリタ上で実習してきた。この方法は、比較的学习に適しているが、サーバや長時間の計算など無人で動かす用途には向かない。

標準的には、「ターミナル」から、`python3` コマンドを実行する。最終課題 (第 B 章) に備えて、動作環境を確認し設定しよう。

まずは、バージョンを確認しよう。Python 3.x と表示されればこの資料の意図の通りである。

```

$ python3 --version
Python 3.10.9

```

(余談) 次のコマンドは、簡易のウェブサーバをたてて、カレントフォルダ以下をウェブアクセス可能にする。

```

$ python3 -m http.server

```

9.6.1 ソースファイルの作成と実行

VSCoide などのエディタで、次の内容の Python ソースファイルを `hello.py` という名前で作る。

```

Python3
print('Hello')

```

そのフォルダにターミナルで移動する。もしくは、VSCoide のターミナルを開く。¹⁰⁾ `cat` コマンドで、内容がエディタで保存した内容と同一であることを確認する。

10) <https://code.visualstudio.com/docs/terminal/getting-started>

```
$ cat hello.py
print('Hello')
```

ソースコードのファイル名を指定して実行する。

```
$ python3 hello.py
Hello
```

🔧 ファイル名の注意

ファイル名は標準的なライブラリと重ならないようにつけると良い。たとえば、`numpy` と関連が深い自分のプログラムがあったとして、ファイル名前、`numpy.py` ではなく `myprojectname_numpy.py` などとする。

9.6.2 引数の指定

実行時にソースファイル名に続いて、引数を渡して、プログラムの動作を制御することができる。

例題 整数の和

引数で与えられた、二つの整数の和を求めるプログラムを作って見よう

引数は、`sys.argv` という配列から入手できる。型は文字列 `str` なので、整数として使う場合は `int` で変換する。以下は実装例である。

Python3

```
import sys
1
2
3
def add(x, y):
4
    '''return sum of x and y
5
6
    >>> add(3, 5)
7
8
    ...
9
    return x + y
10
11
12
def test_add():
13
    assert add(0, 0) == 0
14
    assert add(123, 999) == 1122
15
16
17
```

```

if __name__ == '__main__':
    a = int(sys.argv[1])
    b = int(sys.argv[2])
    print(add(a, b))

```

18 行目の `if __name__ == '__main__':` のブロックがメインの計算で、この `if` の行をそのまま、以後、つけておくことを推奨する。省略可能な場合もあるが、省略すると次に紹介するテストの実行や `multiprocessing` で並列処理を行う場合などで問題が起こりうる。

9.6.3 テストの実行

ファイルに保存した Python のソースコードについて、`doctest` (8.2 節) を実行する場合は、ターミナルから次のようにタイプする。

```

$ python3 -m doctest -v filename.py

```

`pytest` 用に書かれたテスト関数を呼ぶには、以下のように実行する。

```

$ python3 -m pytest filename.py

```

9.6.4 総合問題

問題 showaddress.py

題「住所の表示」で扱った形式の `json` ファイルが、カレントフォルダにあるとする。引数で指定されたファイルをひらき、(1) 住所を 1 行で表示し、(2) 次の 1 行に同住所の文字数を表示するプログラムを作成せよ。`json` から 1 行に連結した文字列を得る機能は、関数として作成し、`doctest` または `pytest` に対応するテストを書くといい。

(option) ファイル名の代わりに URL が指定された場合は、ダウンロードしてファイルに保存する。

実行例

```

$ python3 showaddress.py 1538902.json
東京都目黒区駒場 3 丁目 8-1 東京大学大学院総合文化研究科・教養学部
33

```

9.7 この章の提出課題

Python の 1 つ以上の問題について取り組み、UTOL に提出する。テストを書きにくい問題については、実行結果 (の一部) をコピーペーストで添えることで、テストの代わりとする。

9.A Microsoft Windows での環境設定

Windows 環境で学習している場合は、自身でターミナル環境と Python 環境を用意する必要がある。本資料では以下のどちらかの方法を勧める:

1. gitbash + python,
2. Windows Subsystem for Linux WSL
3. MSYS2

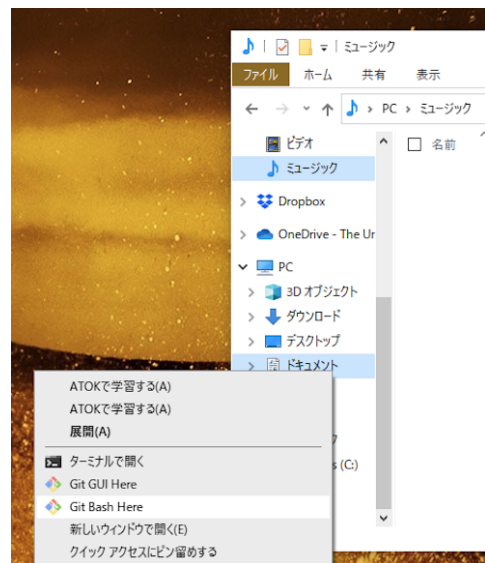
9.A.1 Gitbash

- Gitbash を <https://gitforwindows.org/> からダウンロード・インストールする。
- Python を <https://www.python.org/downloads/windows/> からダウンロード・インストールする。2025 年時点で 3.13 シリーズが最新だが、安定版としてその前のバージョンの 3.12.10 または 3.11.9 を勧める。インストーラにも種類があるが、通常の Intel または AMD の CPU なら、Windows installer (64-bit) が適するはず。

■ Download [Windows installer \(64-bit\)](#)

- ターミナルを開く

エクスプローラーの右クリックから、ソースコードを置いたフォルダを gitbash で開くメニューを使うと、ターミナルウィンドウが開き目的のフォルダに `cd` した状態となる。あるいは、gitbash アプリケーションを開いてから、ターミナルウィンドウ内で `cd /c/Users/your-name/path/to/source/` などでも良い (your-name 以下は適宜書き換えること)。



- `python` コマンドがあるので, `python your-file.py` などと実行する

9.A.2 WSL

- Powershell を「管理者権限で実行」
- `wsl --install` で Ubuntu 22.04 あるいは 24.04 をインストール
- スタートメニューに増えた, Ubuntu というアプリケーションを実行. ターミナルが開く
- ここでは `python3` というコマンドが使える. `python3 your-file.py` などと実行する
- エクスプローラーで見えるホームに移動するには, `cd /mnt/c/Users/your-name/` など.

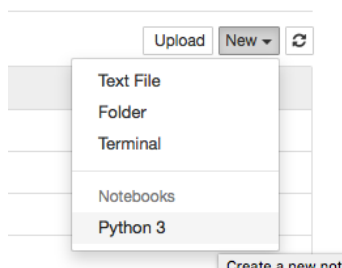
9.A.3 MSYS2

<https://www.msys2.org/> インストール 手順したがって, ターミナルが開くまで勧める (default で進めて良い).

9.B Jupyter で書いたプログラムの実行

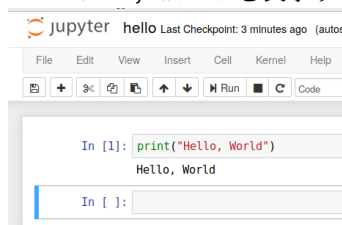
Jupyter で開発したノートブックから Python のソースコード (`.py`) を出力するには以下のように行う.

1. jupyter 上で新規ノートブックを作成する

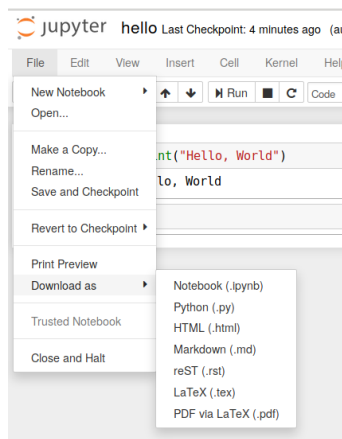


2. ノートブックを編集する

- jupyter ログの右をクリックして, ノートブックの名称を “hello” にする
- Hello, World を表示する文を書く



3. python ファイルに export する: File → Download as → Python(.py)



ダウンロードフォルダなど(ウェブブラウザの設定と操作による)に保存されているので, `cat` コマンドで確認し, つづいて, `python3` コマンドで実行する.

```

$ cat hello.py
# coding: utf-8
# In[1]:
print("Hello, World")

```

```

$ python3 hello.py
Hello, World

```

9.C ファイルの分割

大きなプログラムの開発では, 複数のファイルに分けて開発して, 統合して使用することが多い. 一つのファイルに沢山の行数を詰め込むと見通しが悪くなるため, 機能毎に適宜分割することは良い実践である.

以下の関数が, `mymessage.py` というファイルに保存されていたとする.

Python3

```

def nice_message():
    return "Hello, World!!!!"

```

上記の関数を同一ディレクトリ内の `main.py` から呼ぶ方法ためには以下のように書く.

Python3

```

import mymessage
print(mymessage.nice_message())

```

```

$ python3 main.py
Hello, World!!!!

```

この状態で、mymessage.py を (元気よく) 書き換えたとなると、

```

$ cat mymessage.py
def nice_message():
    return "Hello, Hello, Hello, Hello, Hello, World!!!!"

```

main.py を書き換えていないにもかかわらず、実行結果が変化する。確認せよ。

```

$ python3 main.py
Hello, Hello, Hello, Hello, Hello, World!!!!

```

9.D reload

jupyter 上でも `import mymessage` などとして、Python ソースファイルを読み込んで使うこともできる。一つ注意として jupyter 上で自作ファイルを `import` 後に元ファイルを変更した場合には、そのままでは新しいファイルの内容は使われない。ファイルを読み直して新しい内容を反映させるためには、`importlib` を `import` したうえで、その `reload` という機能を用いる。

```

Python3
import importlib
importlib.reload(modulename)

```

別の方法として、jupyter (の元の ipython) の拡張機能¹¹⁾を使い、次の行を実行しておくことでも良い。

```

Python3
%load_ext autoreload
%autoreload 2

```

11) <http://ipython.org/ipython-doc/stable/config/extensions/autoreload.html#module-IPython.extensions.autoreload>

第 10 章 最終課題

2026-06-24 Wed

概要

最終課題は、これまでの演習で練習した技術を組み合わせて多少規模の大きなプログラムをすることと、既存のプログラムとあるいは他者のプログラムと組み合わせて動かすことを主眼にする。

なお、付録 A 章で紹介する開発ツールについて、必須とはしないが、習熟しておくことを勧める。

10.1 最終課題の選択肢

以下のいずれかを選んで取り組む:

1. **標準ゲームのプレイヤー**: 不完全情報ゲームの一つである潜水艦ゲームの思考部分を作る。対戦フレームワークは指定する (既存の) 実装に合わせて作ること。B 章を参照。可能な範囲で、多くの人のプレイヤーと対戦して、プレイヤーの実際の行動と自分の想定の違いを把握する。
2. **標準ゲームのサーバ**: 対戦フレームワーク部分を自分で作成したコードで置き換えるまたは変更し、機能を拡張する。たとえば、マップのファイルから読み込む、通行禁止エリア通過する船の移動を禁止とする、や時間制限の指定、対戦の様子のアニメーション表示や成績管理機能などが考えられる。
3. **独自ゲーム**: 3 人程度でチームを組んで、独自のゲーム (同時着手ゲームもしくは不完全情報ゲームが望ましい) の対戦フレームワークとプレイヤーを分担して作成する。
4. **強化学習エージェント**: Gymnasium のエージェントを作る。C 章参照。大学院生などで、強化学習について習得済みである人向け。

10.2 Schedule

- 第 10 週 (本日) 構想検討, 制作
- 第 11 週 制作
- 第 12 週 発表 (原則 学部生)
- 第 13 週 発表 (原則 大学院生)

10.2.1 提出期限

- 7月8日の授業開始前まで:
 - スライド PDF 2,3 枚程度: プログラムの仕様, アイデアなど
 - ソースコード (1 次提出) この段階では一部未完成でも良い
- 7月31日予定:
 - ソースコード (最終提出) 動作するものであること
 - スライドまたは解説 発表会のコメントや他の人のアイデア, その後の進捗を踏まえたもの.
形式は plain txt, markdown, pdf のいずれかを想定.

第 11 章 最終課題作成

2026-07-01 Wed

第 12 章 最終課題発表

2026-07-08 Wed

2026-07-15 Wed

この日までにスライド (目安: 数枚) の PDF とプログラムのソースコード (できているところまで) を提出しておく。授業時間中に、構想とプログラムの説明を簡単に発表する。

付録 A 開発ツール

A.1 バージョン管理システムと最小限の git

概要

継続的な開発 (関連するファイル群に継続的に変更を加えるような状況) では, 過去の履歴を保存し, バージョン管理システムの使用を勧める (義務とはしない). さらに, ネットワークで共有することで, 自宅と大学など複数の場所での開発で連携をとったり, グループで共同開発することにも可能である. ツール類は日々新しい物が登場するので, 良いツールを見極める目を養う必要がある. ここでは git を中心に紹介する.

A.1.1 さいしょの目標

長期的に開発するソースコードや, 文書 (markdown, LaTeX, html 等) の管理について, まずは以下の目標を同時に実現しよう

- いつでも過去のバージョンを取り出せる
- 複数のファイルを一貫して管理する
- 通常は最新版だけが見えている
- 記録したくない変更は記録しない



避けたいこと

- report-new2.pdf と report-0706-1.pdf はどっちが新しいんだっけ? 何を変更したんだっけ?
- いつのまにか bug が発生して, 動かない機能があった

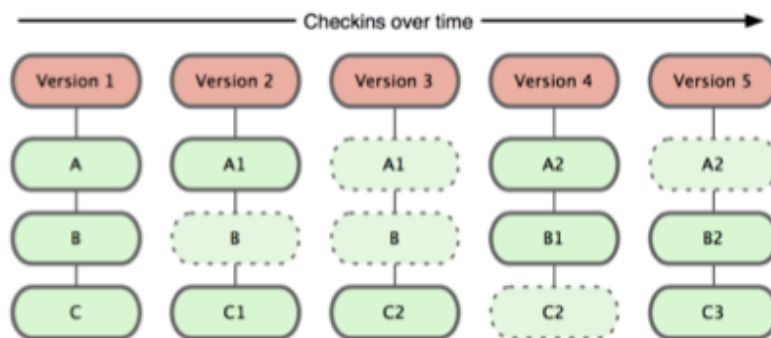


図 A.1 ProGit (図 1.5) の説明 — 縦の列の単位でバージョンを管理する. おおむね時刻と対応する.

 インストール

各自の PC で使う場合は、インストールが必要な場合もある。

- macOS: 最近は標準で入っている。
もしなければ `brew install git` (homebrew を使っている場合)
- Ubuntu: `apt install git`
- Microsoft Windows: 9.A 節で紹介した, gitbash (<https://gitforwindows.org/>) または wsl のどちらでも良い。(VSCode を使っていてそのターミナルで `git --version` などのコマンドが動作すればそれを使えば十分)

 GUI

本資料では, git の概念との対応を重視してターミナルでの `cui` コマンドを紹介する。しかし, 実用的には (基本を習得後は) GUI を使う方が便利であろう。たとえば以下の URL に, 様々な選択肢が紹介されている (有料のものもあるので注意)。VSCode を使っているならそのプラグイン, 初めて試すなら GitHub Desktop が候補と思われる。なお担当教員は, Emacs と Magit を使っているがここでの紹介は割愛する。

<https://git-scm.com/downloads/guis>

A.1.2 初期設定

ユーザ情報の設定

各計算機 (あるいはそのログインアカウント) ごとに 1 度だけ, ユーザ名と電子メールアドレスを設定する。共同作業が前提の場合は普段使うアドレスが適するが, 一人で使う分にはなんでも良い。後で変更可。



```
$ git config --global user.name "Hanako Komaba" 1
$ git config --global user.email "xxxx@g.ecc.u-tokyo.ac.jp" 2
```

リポジトリの初期作成

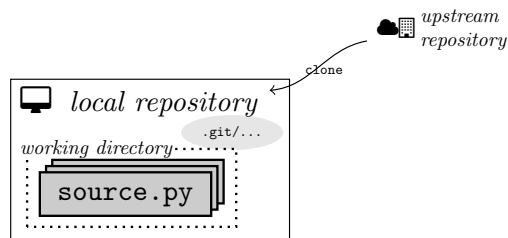
git で管理する単位 (一連のファイルやフォルダ) を **プロジェクト** と呼ぶ。また, ユーザがプロジェクト内のファイルを編集する場所を **ワーキングディレクトリ** と呼ぶ。なお, プロジェクトがフォルダを含む場合は, そのルートを実際のワーキングディレクトリとする。プロジェクトを管理するための git の単位を **リポジトリ** と呼ぶ。最初に紹介する使い方では, ワーキングディレクトリ, リポジトリそれぞれが 1:1 で対応する。大学の計算機と個人 PC の両方で同じプロジェクトを開発する場合は, 1 つのプロジェクトに複数のワーキングディレクトリが対応する。

既存のリポジトリを複製する場合

既存のリポジトリを複製して作業を始めるには `git clone` を用いる。github 等の公開リポジトリから、ソースコードを入手する場合もこれに従う

```
$ git clone URL
```

複製元のリポジトリを upstream repository (上流), 手元を local repository と呼ぶ。



`clone` 操作後に通常目にするのは、working directory 内のファイル群で、当該リポジトリの最新のものである。それ以外に、隠しフォルダ `.git` の中には、過去の履歴や、複製元の URL などが格納されている。

このように `clone` したプロジェクトに変更を加える前に、以下のように `local-changes` というブランチ (保存先) を新たに作成するとよい。

```
$ git checkout -b local-changes
```

ブランチは、自分の変更の歴史を隔離して管理する仕組みである。上流 repository は独自の開発の歴史を持つので、それと分けることが、ここでの目的。

潜水艦ゲームの導入 (B.2 節) においても、この手順に倣った。

新規リポジトリを作成する場合 (参考)

プロジェクトを新規に始める場合、以下のよう
に新規フォルダを作成後に、そのフォルダ内
で `git init` を行いリポジトリを作成する。こ
こではプロジェクト名を `project` とする。

```
$ mkdir project
$ cd project
$ git init
```

既にソースコード等をおいたフォルダを対象
として、`git` リポジトリを作ること
もできる。右図は `ProgrammingLab` というフォルダの例。

```
$ cd ProgrammingLab
$ git init
```

その後、管理するファイルを `git add` で指定し、`git commit` で最初のバージョンを登録する。これらの操作は、次に説明するファイルの変更と登録次の操作と同じ。

A.1.3 ファイルの変更と登録

ソースコードを編集・あるいはファイルを追加後に、リポジトリに記録するファイルを `git add` で指定する。

指定するファイルは、既存のものでも新規に作成したものでも良い。たとえば `hello.py` なら、右記の操作を行う。

```
$ git add hello.py
```

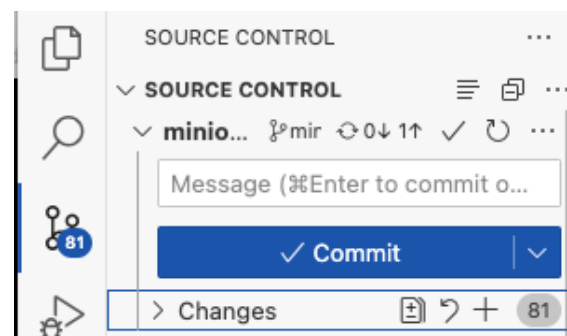
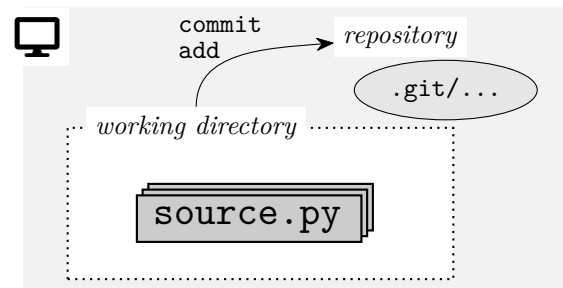
複数のファイルを指定する場合は、1 度に登録することも、何度かに分けて登録することもできる。また、間違えて指定した場合に取り消すこともできる。git add 後に、該当ファイルをもう一度編集した場合は git add を再度行う。

現時点で管理したいファイルをすべて指定したら、最後に `commit` という操作を行う。

```
% git commit -m 'msg'
```

'msg' の部分は自分のメモである。たとえば 'initial version' などとする。commit により、一連のファイルのスナップショットとバージョン ID が作成される。

通常は、add と commit 相当の操作を GUI で行うことが多い。たとえば、Visual Studio Code では、**+** ボタンで add、**✓** ボタンで commit を行う。



💡 これまでの成果

ここまでで、自身の変更を加えたバージョンが登録された。今後、現在見えているファイルをどのように変更しても、いつでもこのバージョンに戻したり、内容の差分を調査したりすることができる。

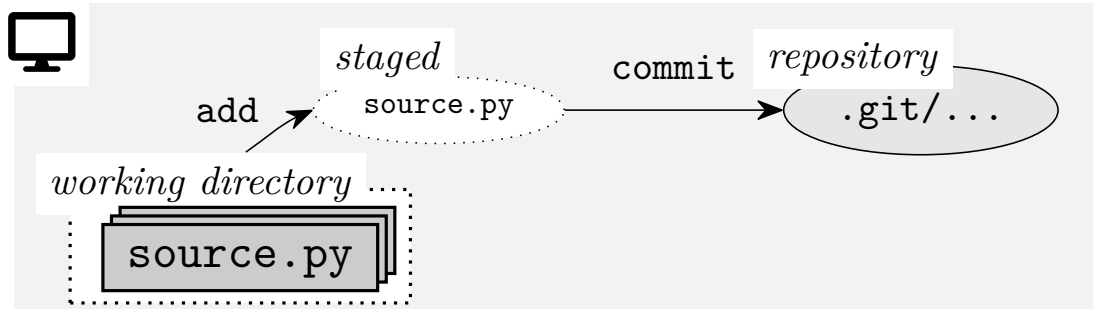
git add と commit の関係

git commit によりいつでもスナップショットをとることができる。あとから見て分かりやすい状態で commit すると良い。指針はたとえば、

- 動くコード、test をすべて pass するもの。
- 保存したいコード (臨時で書いた debug 用 print などは、あとから読んだときに邪魔になりがち)
- 前回からの commit からの目的が、単一。

3 点目について、はじめは 1 日 1 回作業終了時に commit することから始めて、慣れてきたら、機能 A の実装など区切りの良い単位で細かく commit を行うと良い。

git では意図通りの commit を作るためのサポートとして、commit の準備用の *staged* という中間段階を用いて、2 段階で commit を行う。



第 1 段階では、commit に入りたい (スナップショットをとりたい) ファイルを `git add` で指定する。これにより、まずワーキングディレクトリのファイルの内容のスナップショットが *staged* 領域に作られる。どのファイルがどの状態にあるかは、`git status` で確認できる。

複数のファイルの編集・変更を登録する場合は、1 度に `add` することも、何度かに分けて登録することもできる。

```
% git add files
```

変更内容の確認は、`git diff` で行う。stage に準備したものかどうかで、コマンドのオプションが変わる。

- `git diff` ワーキングディレクトリと stage された内容との差分 (次の commit に **含まれない**)
- `git diff --cached` stage された内容と 最新の commit との差分 (次の commit に **含まれる**)

第 2 段階として、意図通りに登録できたことを確認したら、`git commit` を行う。これにより、*staged* にあるスナップショットで commit が作成され新しくバージョン ID を付与する。図 A.1 の時系列が右に一段進むことに相当する。

```
% git commit -m 'message'
```

注意点として、`git add` 後に、該当ファイルをもう一度編集した場合、そのままではその変更は commit には含まれない。2 種類の diff で確認をすればそのような場合にも気づくことができる。なお、編集後に `git add` を再度行えば、最新の内容が stage される。

commit に入れる予定でないファイルや変更を、間違えて add した場合には、取り消すこともできる。本資料では、`git restore` を勧める。

```
% git restore --staged
↪ filename
```


A.1.4 履歴の活用

commit を活用して、様々な操作が実行可能である。それぞれの目的のために沢山のコマンドが用意されているので、当面は必要になったら調べる方針が良い。

過去の commit の一覧を見る。

```
 % git log 1
```

同 1 行で commit の一覧を見る。

```
 % git log --oneline 1
```

変更点も含めて読む

```
 % git log -p 1
```

過去の特定の時点でのファイルの内容の表示

```
 % git show バージョン文字列: ファイル名 1
```

同差分の表示

```
 % git diff バージョン文字列 ファイル名 1
```

A.2 リモートコンピュータの利用 — SSH

目的

家から大学の計算機上のプログラムを読み書きする。github を使うなど。

SSH を用いると、「ターミナル」で行う操作を (たとえば git コマンドも) 遠隔から行うことができる。情報系ではほぼ必須の技能である。また、SSH の認証部分は、github など多くのサービスで採用されている。そのため、認証部分だけでも理解しておくことが望ましい。

A.2.1 公開鍵暗号システムを用いた認証

公開鍵暗号システムでは、ユーザ毎が **鍵ペア** (公開鍵, 秘密鍵) を作成して用いる。サーバは秘密鍵の所有者のログインを許可し、所有しないもののログインを拒絶する。加えて、通信データを暗号化する。

公開鍵 公開して用いる。SSH ログインの場合は、ログイン **先** のサーバに置く。ログイン先のサー

バが、ログインを試みるものが行った電子署名をこの公開鍵を用いて検証する。サーバでの置き場所は、標準的には `~/.ssh/authorized_keys`。

秘密鍵 ユーザの手元で1つのファイルを秘密に保つ。ssh でログインを試みる際に、秘密鍵を用いて署名を作成しサーバに提出する。標準的には `~/.ssh/id_method`。 (method の部分は方式の名前で、rsa や ecdsa が入る)

鍵ペアの作成

ssh-keygen コマンドにより、公開鍵  と秘密鍵  のペアを作成する。¹⁾ 下記の実行例は、手元のノート PC (等) でのユーザ名が hanako の場合。

 `~/.ssh/`
 `id_rsa.pub` (`id_ecdsa.pub`)
 `id_rsa` (`id_ecdsa`)

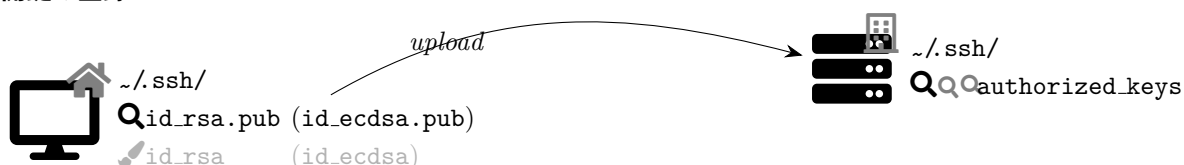
```
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/hanako/.ssh/id_rsa):
# 上記の質問は秘密鍵の保存場所 お勧めは何もタイプせずにリターン
Enter passphrase (empty for no passphrase):
# 上記の質問はパスフレーズ 暗記できるものを入れておくことがお勧め
```

Notice: 注意

もし、秘密鍵が盗まれると (ノート PC 盗難やウイルス感染を含む)、盗んだ人はログインしてあなたの権限を行使できる。その場合、まず間違いなくそこを踏み台に他のサーバを攻撃するので、被害はユーザ個人に留まらない。そのようら可能性が分かった際は、速やかにログイン先サーバの管理者に届け出ること。最後の砦がパスフレーズだが、時間がたてば破られてしまう。

鍵ペア作成における一次資料としては openssh のマニュアル <https://man.openbsd.org/ssh-keygen> を参照されたい。

公開鍵の登録



1) 暗号化と復号の文脈では、公開鍵が錠前  秘密鍵が  ととえることもあるが、今回は署名と検証に使うので、専用検査キット  と特殊インク  の意図でたとえた。

作成した **公開鍵** を，login 先のサーバに登録する．ECCS や github などのサービスでは，Web から行うことができる．ECCS の場合は「SSH サーバ 公開鍵アップロード」 https://portal.ecc.u-tokyo.ac.jp/key_upload を用いる．

大学研究室など多くの通常のサーバでは，`~/.ssh/authorized_keys` が公開鍵を登録すべき場所となる．管理者に依頼するか，ECCS の場合は自身で端末にログインし，該当ファイルに公開鍵をコピーペーストしても良い．なお，`~/.ssh/authorized_keys` には複数の鍵を登録できる．テキストファイルで，1 行に 1 つの鍵を登録する．

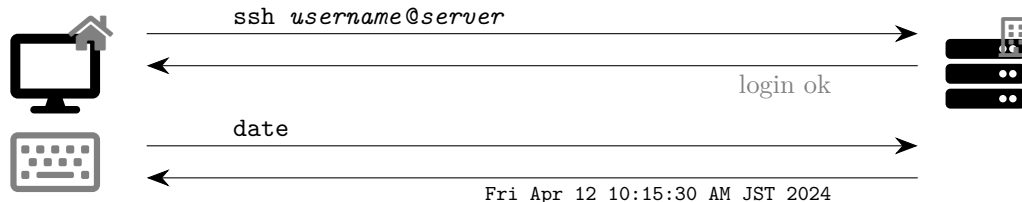
ログイン

ログインには `ssh` コマンドを使う `@` の前がユーザ名，後にホスト名を書く．なお，github 等のサービスで `ssh` の公開鍵の認証を用いる場合は，ここで説明するログインには対応していない．次の節に進めたい．

次の例は，ログイン先でのユーザ名が `9876543210` で，`ssh01.ecc.u-tokyo.ac.jp` にログインする例である．

```
$ ssh 9876543210@ssh01.ecc.u-tokyo.ac.jp
```

このとき，秘密鍵を使うためにパスフレーズを聞かれたらタイプする．ログインに成功したら，リモートでコマンドを実行することができる．下図はログイン後に `date` コマンドをログイン先で実行し，実行結果を手元のターミナルで受け取った例である．



ECCS 環境に `ssh` ログインする方法の詳細は次の URL を参照 <https://www.ecc.u-tokyo.ac.jp/system/outside.html#ssh>

発展

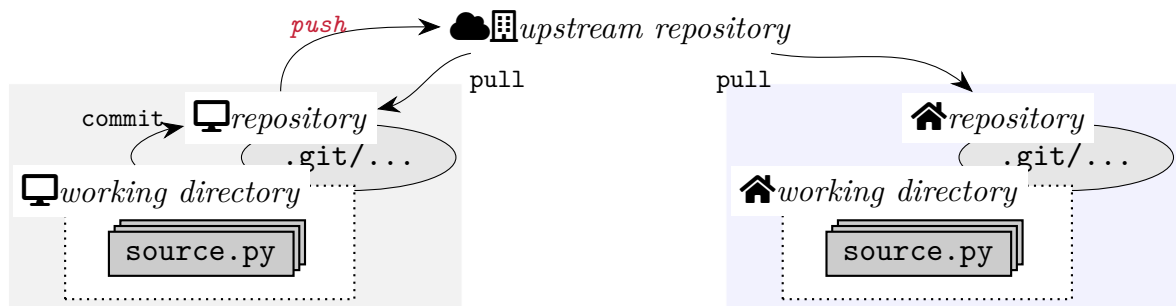
何度もログインする際に，パスフレーズをいちいち打たずに行うには，`ssh-agent` や `ForwardAgent` などの仕組みを調べると良い．またホスト名の短縮表記やユーザ名の省略記法については，`.ssh/config` の設定方法を調べると良い．ファイルのコピーは `scp`, `rsync` など調べると良い．

A.3 リモートリポジトリの利用と共同開発

目的

ノート PC とデスクトップなど複数の環境のどこでも開発する。グループで、作業を共有する。研究室で、卒論やソースコードを教員に見せる。などの状況を想定する。

ここで紹介する使い方では、共通の **上流** (*upstream*) リポジトリを作成し、全リポジトリの commit 集約・管理する。各作業場所では、これまで同様ローカルに `git` のリポジトリを持つとともに、その内容を上流リポジトリと同期する。手元の変更を上流に送る操作を *push*, 上流に存在する変更を手元に取り込む操作を *pull* と呼ぶ。



A.3.1 上流リポジトリの作成

ファイルシステム上に作成する場合と、外部サービスを利用する場合に分けて説明する。前者は、`git` コマンドやファイル操作を使える環境なら場所を問わない。つまり、手元のコンピュータでも ECCS でも良い。

ファイルシステム上の場所を決め、リポジトリを設定する。右記は、`~/git` に `projectname` 用のリポジトリを作成する例である。オプション `--bare` に注意。

```

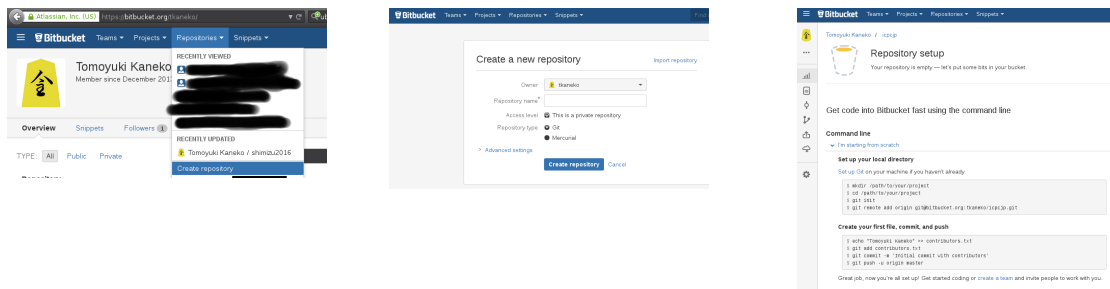
$ mkdir ~/git          1
$ cd ~/git             2
$ git init --bare      3
↪ projectname.git

```

外部サービスを使う場合は、リポジトリはウェブ等のユーザインターフェースで作成することが多い。

利用可能な外部サービスとしては `github` (<https://github.com/>) が現在最も有名である。学生特典もある。他に、`bitbucket` (<https://bitbucket.org/>) や `gitlab` (<https://about.gitlab.com/>) も選択肢で、教員としては、特定のサービスを勧めるわけではない。一般にどのサービスでもアカウントを作成する必要がある。その際には何らかの契約が行われるので、`terms and conditions` (`terms of service`, `terms of use`) などの文書を熟読すること。通常は、サービス提供側の免責が書いてあることが多いが、利用者の義務が課せられていないかどうかを注意して読むこと。一般に自分が同意した

ファイルは、保存しておくことを推奨。



A.3.2 上流リポジトリの活用

これまで開発してきた手元のリポジトリと上流リポジトリをつなげる方法、つづいて、上流リポジトリを共有する新しい作業場所を作る方法を説明する。

まず、上流リポジトリ (がどこか) を特定するためには (一般の意味の) URL を用いる。よく使うものは次の 2 通り。

- 同じ計算機のファイルシステムの場合、ファイルパスを使う。
例 `~/git/projectname.git`
- リモートの計算機を ssh login で使う場合や github のように ssh での認証をする場合は、リモートのユーザ名, @, リモートのサーバドメイン名, :, リモートでのファイルパス, をつなげる。
例 `git@github.com:tkaneko/submarine-py.git`,
`git+ssh://9876543210@ssh01.ecc.u-tokyo.ac.jp:/home/9876543210/git/projectname.git`

正式には、`git+ssh:` のようなプロトコルの指定を URL の先頭に書くが、多くの場合省略可能。

ローカルのリポジトリに上流を指定するには、右図の 2 つのコマンドを使う。 `git remote add` ままでが動作の指示で、`origin` は上流を表す名称 (当面このまま使う)。

```
$ git remote add origin URL      1
$ git push -u origin main        2
```

`main` は、**ブランチ** と呼ばれる歴史の名前に相当し、最初の `git init` の際に標準のブランチとして自動で設定された名前である。 `git` に馴染んだら、開発バージョンと安定バージョンなどブランチを分けることが有用である。²⁾



main v.s. master

default の branch 名称は、新しめの環境では `main` が標準的である。古くは `master` が使われていた。

2) <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>

上流の場所は、`git remote -v` で確認できる。出力の `fetch` と `push` はそれぞれアップロードとダウンロードに対応する。今回は同じ URL が表示されるはずである。

```
$ git remote -v
origin URL (fetch)
origin URL (push)
```

新しい環境に作業場所を設定するには、`git clone` を用いる。

```
$ git clone URL
```

💡 これまでの成果

ここまでで、複数の開発環境を、共通の上流リポジトリを通じて、同期させる準備ができた。

今後、典型的な開発は以下のようになる

- ワーキングディレクトリで通常の開発を行い、`commit` する。この時点では、そのコミットは手元でのみ使える。
- `push` により、上流リポジトリに新しいコミットを送る。
- 別のワーキングディレクトリでは、新たに編集する前に `pull` により、上流リポジトリの新しいコミットをローカルに反映させる

⚠ Notice: 注意

`pull` する前に手元のファイルを編集したり、手元の編集履歴を `push` する前に別の場所で `push` が行われて上流の履歴が書き換わると、編集履歴が二股に別れる。そのようなコミット自体は保存されているので、後から対応可能である。とはいえ、分かれた履歴の統合手順を学ぶまでは、上記の手順に従うことがお勧め。

A.3.3 複数人での開発

リポジトリは、通常は個人用に初期設定される。共同作業を行う場合は、読み書きの権限を適切な人で共有する必要がある。

共通の外部サービスを使っている場合は、ユーザごとに `read`, `write` などの権限を設定できる。研究室などでファイルシステムを使う場合は、OS の「グループ」を使ったアクセス制限が可能である。ただ `eccs` の `student` グループは学生全員となってしまうため、目的に適さないことが多いと思われる。

参考書籍

Pro Git (初めの 1-2 章に目を通せば十分使える) がお勧め

- 和訳の PDF, epub など <http://progit-ja.github.io/>,
- 和訳の HTML <http://git-scm.com/book/ja>

付録 B 最終課題: 潜水艦ゲーム

この章では、潜水艦ゲームの思考部分の作成という課題について紹介する。

B.1 目標

- 必須
 - ランダムプレイヤー (random_player.py) より、はっきり強いプレイヤーを作る
 - 何かしら自分の考えた戦略によって行動する
 - いろいろな大きさのマップに対応する (最大 40 マス程度と仮定して良い)
- 努力目標
 - なるべく強くする
 - simple_player.py (後日 UTOL に掲載) に勝ち越す

B.2 準備

ターミナルで、git, python が使えることを確認する。Mac の場合はどれも標準で使える。Microsoft Windows の場合は、第 9 章のファイルからの実行 (9.A 節) で紹介した手順により、git と python (または python3) コマンドが使えているはず。

続いてインストールを行う

1. 配布物一式をダウンロードして、適当な場所で展開する。

	\$ git clone https://github.com/tkaneko/submarine-py.git	1
	\$ cd submarine-py	2

2. 必要に応じて¹⁾ git checkout main で main というブランチ (後述) を選択

	\$ git checkout main	1
---	----------------------	---

3. ソースコードの変更準備として、local-changes というブランチ (保存場所) を新たに作成する。
(必須ではないが推奨)

1) フォルダにファイルが見当たらない場合は必要



```
$ git checkout -b local-changes
```

1

4. インストール ターミナルを1つ開いて、パッケージをインストールする。オプションの `-e` とそれにつづくドット `.` が今後の開発で重要。²⁾



```
$ uv pip install --user -e .
```

1

あるいは



```
$ pip3 install --user -e .
```

1



Virtualenv の準備が必要な場合

新しい Python (3.12 以降) では、パッケージのインストールの前に virtualenv の準備が必須である。そのようなエラーが出た際には、以下のコマンドで準備をし、もう一度インストールすると良い。

```
$ python3 -m venv venv
$ source venv/bin/activate
$ python3 -m pip install -e .
```

1

2

3

なお `uva` を導入済みの場合は、`uv venv` と `source .venv/bin/activateb`, `uv pip install -e .` などが同等のコマンドである。

^a <https://docs.astral.sh/uv/>

^b あるいは `source .venv/Scripts/activate`

B.3 動作確認と対戦

まず、人間が AI と対戦するまでを、動作確認しよう。その後で、`README.md` の説明を読むと良い。

1. サーバ起動 同ターミナルで以下を実行する



```
$ python3 sample/server.py
```

1

これによりサーバープログラムが起動し、プレイヤープログラムからの接続を待つ。

2) 標準コードを編集した場合に再インストール無しで反映されるかどうかが変わる。変更しなければ動作に差はない。

2. プレイヤーの接続 ターミナルをもう 1 つ (別のウィンドウまたはタブを) 開いて, 以下を実行する

```
$ python3 sample/random_player.py localhost 2000 1
```

これによりランダムに動く AI プログラムをサーバープログラムと接続する.

3. ターミナルをさらにもう 1 つ (別のウィンドウまたはタブを) 開いて, 以下を実行する.

```
$ python3 sample/manual_player.py localhost 2000 1
```

これは, ターミナルからのキーボード入力を通じて, 人がゲームをプレイするプログラムである.

```

0  1  2  3  4  1
----- 2
0  3
----- 4
1  5
----- 6
2  7
----- 8
3  9
----- 10
4  11
----- 12
please input x, y in ([0, 4] x [0, 4]) 13
w 14
x = 0 15
y = 0 16
c 17
x = 4 18
y = 4 19
s 20
x = 2 21
y = 2 22
0  1  2  3  4 23
----- 24
0  w3 25
----- 26
1  27
----- 28
2  s1 29
----- 30
3  31

```

-----		32
4	c2	33
-----		34



うまく動かない時の調査

- 3つのターミナルを注視し、エラーメッセージが出ている場合は、よく読む。一つが異常終了すると、巻き添えで全てが止まることがあるので、どれが最初かを推理する。たとえば、起動時の引数が足りない場合などはエラーメッセージから追跡できる。
- ファイアーウォールが通信を止めている場合がある。(安全な環境 e.g., インターネットから切り離れた状態で) ファイアーウォールを一旦停止して、動かしてみる。これで動く場合は、ファイアーウォールの設定を工夫して、ファイアーウォール動作状態で動くようにする。サーバが接続を待ち受ける許可を出したり、port 番号 (上記の例では 2000 番) での通信を許可するなどが必要と思われる。
- IPv4 v.s. IPv6: `random_player.py` は IPv4 を前提に書かれているが、サーバ側は OS の設定に依存するため IPv6 で接続を待つことがある。

B.4 独自の思考プレイヤー作り方

`sample/random_player.py`などを参考に、自動で行動するプレイヤーを作成する。

B.4.1 サンプルプログラムを読む

1. README.md を読んで、全体の概要をつかむ。(ゲームのルール、ディレクトリの構造など)。ウェブブラウザで <https://github.com/tkaneko/submarine-py/blob/main/README.md> を読んでも良い。
2. `doc/client_doc.md`, `doc/document.md` を読んで、通信の流れなどの仕様を理解する。
3. `src/submarine_py/player_base.py`, `sample/random_player.py` や `sample/manual_player.py` のを軽く読んで見る。共通の `Player` class を継承していること、 subclasses で実装が必要な method が何かを把握する
4. 自分の AI プレイヤー `sample/xxxx_player.py` を作る

B.4.2 ヒント

- 人とランダムプレイヤーでゲームを遊んでみる。
- 人間同士で遊んでみる。紙でも遊べるが、このプログラムを利用する方が簡便。ゲームの特徴を理解して、人間ならまずどういう作戦を立てられるかを検討する。
- プレイヤープログラムがサーバーから情報 (json) を受け取るのは、「自分の行動終了後」及び「相

手の行動終了後」で、このタイミングで Player クラスの update メソッドが実行される。

- `random_player.py` はゲームのルールに違反しないように実装されているが、もしゲームのルールに違反した行動をサーバーに送信するとその時点で敗北扱いになる。それを防ぐために、自分の行動をサーバーに送信する前に、ルール違反がないかどうかをチェックするようにすること。そのための補助メソッドが `player_base.py` に存在するので、適宜活用すると良い。

B.5 Python パッケージの読み方

既存ソフトウェアを信頼できるという前提なら、重要なところに時間を使うこと、常識的に理解できそうな部分は「流し」で、どこが本質か手早く見分けられると良い。そのような読み方の手がかりをいくつか紹介する。

B.5.1 フォルダ構成

潜水艦ゲームのプロジェクトは、典型的な Python パッケージに倣って、右図のようなディレクトリ構造を持つ。ルート直下の `pyproject.toml` (と `setup.py`) が、パッケージの内容と `pip install` 時の動作を定める。授業では、詳細の説明は割愛する。³⁾

`src/submarine.py` が、`submarine.py` パッケージの本体である。その中の、`__init__.py` が `import submarine.py` したときに提供される機能を定義し、各機能の実装は、同フォルダの他のファイルに置かれる。たとえば `__init__.py` 内の `from .ship import Ship` という行は、同じフォルダの `ship.py` から `Ship` という機能を使う（ここでは提供する）という意味である。

```
.
├── pyproject.toml
├── README.md
├── sample
│   ├── random_player.py
│   └── ...
├── setup.py
├── src
│   ├── submarine.py
│   │   ├── __init__.py
│   │   ├── field.py
│   │   └── ...
└── tests
    ├── test_client.py
    └── ...
```

B.5.2 よく使うモジュール

大規模ソフトウェアでしばしば必要になる機能について、Python での標準的な実現方法で、かつ、このゲームで利用されているものを紹介する。

3) <https://packaging.python.org/ja/latest/tutorials/packaging-projects/>

pytest

潜水艦ゲームのパッケージでは、`python -m pytest`などで、テストを実行することができる。テスト用のコードは、フォルダ `tests` 以下に格納されている。また `pytest.ini` の設定により、自動で `doctest` も実行される。

argparse

プログラムの起動時に引数を与えて、動作を調整する方法を 9.6.2 項で紹介した。標準ライブラリ `argparse` は、より高度な機能を提供する。以下のサンプルは、2つの整数を引数 `a`, `b` としてとり、その和を返す。またオプションで `--scale` があたえられたら、それを乗ずる。

Python3

```

import argparse 1
2
3
def task(a, b, scale): 4
    return (a + b) * scale 5
6
7
if __name__ == '__main__': 8
    parser = argparse.ArgumentParser( 9
        description="argparse sample", 10
    ) 11
    parser.add_argument("a", type=int) 12
    parser.add_argument("b", type=int) 13
    parser.add_argument("--scale", type=int, default=1) 14
15
    args = parser.parse_args() 16
    print(task(args.a, args.b, args.scale)) 17

```

読むべき行は、12-14 行目のオプションの指定で、各行が一つの引数に対応している。また、解析したオプションの利用方法は、17 行目のようにオプション名に対応した変数名を用いる。残りの、9-11 行目と 16 行目は定型なので、実用上はコピーペーストして適宜編集すれば良い。

いくつかの実行例を示す。

```

$ python samples/argparse-sample.py 3 5 1
8 2

```

```

$ python samples/argparse-sample.py 3 5 --scale 100 1
800 2

```

必須の引数が不足している場合はエラーになる。自動生成された、簡易的な使い方が示される。

```

$ python samples/argparse-sample.py 3
usage: argparse-sample.py [-h] [--scale SCALE] a b
argparse-sample.py: error: the following arguments are required: b

```

ヘルプ `--help` の対応は自動生成される。これにより詳細な説明を加えることもできる (配布プログラムを参照)。

```

$ python samples/argparse-sample.py --help
usage: argparse-sample.py [-h] [--scale SCALE] a b

argparse sample

positional arguments:
  a
  b

options:
  -h, --help            show this help message and exit
  --scale SCALE

```

例題 `--help` の表示

sample/ におかれている `server.py` や `random_player.py` などについて、`--help` をつけて起動し、引数やオプションの説明を表示してみよう。

標準パッケージ以外では、`click`⁴⁾ はさらに高機能で使用例も多い。

logging

`logging` module は、名前の通りの機能を提供する。動作把握の際は `print` の代わりに使うと有用である。以下の `logging-sample.py` で 5-11 行目の `logging` を含む行が、ほぼ `print` に相当する。引数は文字列 1 つである必要があるので、変数の値を表示する際には `f'text {a}'` という *formatted string*⁵⁾ という文法を用いている。

Python3

```

import logging

def sample_logs():
    logging.debug("I'm looking for chocolate")
    logging.info("I'm going to my lab")

```

4) <https://click.palletsprojects.com/en/stable/>

5) <https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals>

```

a = [3, 1, 4]
logging.info(f"sample for printing value of a variable: {a}")
logging.warning("I feel a bit hungry")
logging.error("Where is my phone?")
logging.critical("No more energy to act")

if __name__ == '__main__':
    level = logging.DEBUG
    # level = logging.WARNING
    logging.basicConfig(level=level)
    sample_logs()

```

このソースコードを実行すると以下の表示を得る。

```

$ python3 logging-sample.py
DEBUG:root:I'm looking for chocolate
INFO:root:I'm going to my lab
INFO:root:sample for printing value of a variable: [3, 1, 4]
WARNING:root:I feel a bit hungry
ERROR:root:Where is my phone?
CRITICAL:root:No more energy to act

```

ここで変数 `level` を次のように変更すると、この 1 行の変更のみで、表示が簡潔になる

Python3

```

level = logging.WARNING

```

```

$ python3 logging-sample.py
WARNING:root:I feel a bit hungry
ERROR:root:Where is my phone?
CRITICAL:root:No more energy to act

```

必ず必要な表示以外は、`print` に代えて `logging` を用い、重要さに応じて `debug`, `info`, `warning`, `error`, `critical` を使い分けると良い。そのようにしておくで、デバッグ中は詳細を表示するなどを簡単に切り替えることができる。また、方法は割愛するが、記録先をファイルに切り替える、時刻を付記するなども簡単に実現できる。

💡 潜水艦ゲームでの logging

このプロジェクトでは、起動時に引数 `--verbose` を与えると `logging.debug` まですべての情報を表示し、それ以外は `logging.info` までを出力するように書かれている。

B.5.3 関数と引数

ソースコードをより読みやすく、また管理を容易にするために、Python の機能をいくつか紹介する。

以下の例で 関数 `add_and_scale(a, b, c)` は $(a + b) \cdot c$ を返す。

Python3

```
def add_and_scale(a: int, b: int, scale: int = 1) -> int: 1
    '''return sum of a and b, optionally multiply it by scale 2
    3
    >>> add_and_scale(1, 2) 4
    3 5
    >>> add_and_scale(1, 2, 100) 6
    300 7
    ''' 8
    return (a + b) * scale 9
```

1. type annotation 関数の引数や戻り値について、ソースコードを書いた人の意図を表記することができる。引数のあとのコロンと型、たとえば `a: int` の部分を、引数に対する type annotation という。同様に `def` の末尾のコロンの直前の `-> int` は、関数の戻値の型に対する annotation である。
2. default value 引数 `scale` は、指定しなければ 1 と解釈される。



```
>>> add_and_scale(1, 2) 1
3 2
>>> add_and_scale(1, 2, 100) 3
300 4
>>> add_and_scale('1', '2') # 異なる型でも、... 5
12 6
>>> add_and_scale('1', '2', 10) 7
12121212121212121212 8
```

関数呼び出しの際に、変数名を指定すること、また (その場合に) 記述の順序を入れ替えることができる。



```
>>> add_and_scale(a=1, b=2, scale=10) 1
30 2
>>> add_and_scale(scale=10, a=1, b=2) 3
30 4
```


B.6 2025 年度の変更

2025 年度は以下の変更が行われた:

- Python package としてインストール可能に
- サーバ `sample/server.py` が Python に移植され, Ruby のコードは不要となった
- ゲーム開始前のプロトコルを拡張し, ゲームのマップの大きさなどを指定可能とした.
詳しくは `python3 sample/server.py --help` で指定可能なオプションの一覧を参照

付録 C 最終課題: 強化学習 Gymnasium

この章では、最終課題のオプションの一つである、Gymnasium について紹介する。歴史としては OpenAI Gym として始まり、OpenAI から Farama Foundation に開発が引き継がれ、名称も gymnasium に変化した。

強化学習は、試行錯誤を通じて学ぶ方法で、いま注目されている分野の一つである。例:
<https://www.youtube.com/watch?v=TmPfTpjtdgg>

Gymnasium は、さまざまな問題 (環境) を統一的に扱うための、インターフェースの一つ。
<https://gymnasium.farama.org/>

C.1 インストール

```
 $ pip3 install --user 'gymnasium[classic-control,toy-text]' 1
```



旧版利用の場合

かつては gym という名称で配布されていた

```
$ pip3 install --user gym 1
```

C.2 動作確認

python3 を起動して `import gymnasium as gym` できれば成功。

Python3

```
import gymnasium as gym 1
env = gym.make('CartPole-v1', render_mode='human') 2
env.reset() 3
# env.render() 4
```

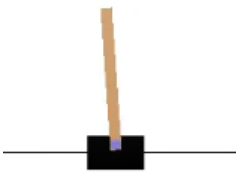
絵が出る。



旧版利用の場合

```
import gym
env = gym.make('CartPole-v0')
env.reset()
env.render()
```

1
2
3
4



C.3 ランダムプレイ

動作確認として、ランダムにプレイするエージェントの動作を確認する。初めに、環境 `env` を作成する。'CartPole-v1' 以外にもさまざまな環境が用意されている (難しすぎるものもあるので注意)。エージェントの行動は、`step` の引数として伝える。行動の選択枝は、`env.action_space` で得られる。`env.make` 時に `render_mode='human'` と指定しておくことで `render()` で GUI 上に描画される。描画は把握に有用だが必須ではない。

Python3

```
env = gym.make('CartPole-v1', render_mode='human')
env.reset()
for _ in range(100):
    env.render()
    env.step(env.action_space.sample())
```

1
2
3
4
5

C.4 賢い行動をさせる

`env.reset()` や `env.step()` を呼び出す毎に、状態と報酬を返り値として得られる。(上記のランダムプレイでは捨てているが)、それらを適切に活用することで、賢い行動を出来るようになる。

詳細は公式文書を参照: <https://gymnasium.farama.org/>¹⁾

1) 旧版 <https://www.gymnasium.dev/>

C.4.1 初めの一步 (案)

FrozenLake-v1²⁾で (よい確率で³⁾) ゴールに到達するエージェントを作成せよ.

これは API に慣れる (特に observation を活用する) 練習なので, (学習せずに), 行動選択方法を直接プログラムに書き込んで良い.

C.4.2 強化学習

経験から学習するには, **強化学習**を用いると良い. ニューラルネットワークを使うことが最近の流行だが, はじめは, 表 (tabular) の Q 学習または Policy Gradient を勧める. 詳しくは教科書を参照: <http://incompleteideas.net/book/the-book-2nd.html>

他の人のコードを参考にした場合は, 出典と参考にした内容について, 提出物内できちんと説明すること.

C.4.3 環境の選択

さまざまな環境が用意されていて⁴⁾, それぞれ難易度が異なる. とくに入力が画像で与えられるものは, はじめは避けること. (画像認識の訓練を GPU で行う経験を積んだのちに取り組む方が無難)

2) https://gymnasium.farama.org/environments/toy_text/frozen_lake/

3) 床は滑る/回転するので, 良い行動を選択していても運が悪いと穴に落ちる.

4) ゲーム以外にも, たとえば GUI の操作 <https://github.com/Farama-Foundation/miniwob-plusplus>

参考文献

- [1] 森畑明昌 「Python によるプログラミング入門 アルゴリズムと情報科学の基礎を学ぶ」, 東京大学出版会, 2019 年
- [2] Peter Van Roy and Seif Haridi, “Concepts, Techniques, and Models of Computer Programming” MIT Press, 2004, <https://ieeexplore.ieee.org/servlet/opac?bknumber=6267353>
- [3] 計算機プログラムの構造と解釈 <http://sicp.iijlab.net/fulltext/>
- [GHJV95] Erich Gamma, Richrad Helm, Ralph Johnson, and John Vlissides. *Design Patterns*. Addison-Wesley, 1995.
- [4] Edmonds, J. (2008). How to Think About Algorithms. Cambridge: Cambridge University Press. doi:10.1017/CBO9780511808241
- [5] 秋葉 拓哉, 岩田 陽一, 北川 宜稔, 「プログラミングコンテストチャレンジブック」第二版, マイナビ, 2012 年, <https://book.mynavi.jp/ec/products/detail/id=22672>

索引

`__init__`, 112

`argparse`, 196

`assert`, 43, 44, 62

`base64`, 157

binary search, 90

`commit`, 182

composition, 121

`deepcopy`, 81

design patterns, 144

`dict`, 160

`docstring`, 138

`doctest`, 138

`dunder`, 117

`elif`, 30

`else`, 30

encapsulation, 116

`evaluate`, 21

expression, 21

Fibonacci numbers, 91

`for`, 13

formatted string, 197

Google Colaboratory, 10

`help`, 137

identity, 115

`if`, 16, 30

`IndexError`, 49

inheritance, 129

`inorder`, 106

`jupyter`, 9

`koch`, 93

`logging`, 197

`matplotlib`, 74

memoization, 122

merge sort, 95

module, 155

`NameError`, 48

`node`, 103

`npz`, 166

`numpy`, 164

overriding, 130

package, 155

`postorder`, 106

`preorder`, 106

`print`, 10, 11

`pull`, 187

`push`, 187

`pytest`, 43

recursion, 86

`return`, 25

`slice`, 96

`staged`, 183

statement, 10

`this`, 135

`type`, 137

`ulimit`, 101

UML, 125

`unittest`, 43

`upstream`, 187

utf-8, 11, 157

value, 21

variable, 23

variant, 137

virtual, 141

while, 96

値, 21

値渡し, 81

インスタンス, 112, 116

インターフェース, 125

インデント, 13

上書き, 130

オブジェクト, 115

改行, 12

仮想関数, 141

型, 21

カプセル化, 116

関数, 24

クラス, 111

クラス図, 125

継承, 129

コッホ曲線, 93

コンストラクタ, 135

再帰, 86

参照, 80

式, 21

条件分岐, 16

真偽値, 27

スタック領域, 101

制御変数, 13

整数, 12

節点, 103

漸化式, 86

代入, 50

抽象化, 23

抽象メソッド, 140

定数, 17

デザイン・パターン, 144

テストケース, 44

内部状態, 115

二項演算子, 22

二次元配列, 66

二重ループ, 15

二分木, 103

二分探索, 90

二分探索木条件, 107

根, 103

バイト列, 156

ハッシュ, 160

引数, 25

評価, 21

符号化方式, 156

フレーム, 100

ブロック, 13

文, 10, 11

変数, 23

末尾再帰, 87

メソッド, 112, 116

メモ化, 122

メンバ関数, 135

メンバ初期化子, 135

メンバ変数, 135

文字列, 28

ユニットテスト, 43

ライフゲーム, 73

リスト内包表記, 65

ループ不変条件, 52

例外, 135

連想配列, 160